

DIPLOMARBEIT

Reinforcement Learning applied to the Game of Abalone

ausgeführt am Institut für
Ökonometrie, Operations Research und Systemtheorie
der Technischen Universität Wien

unter der Anleitung von
Univ.-Doz. Prof. Dr. A. Mehlmann

durch
Frederik René Schorr

Bergheidengasse 29
1130 Wien

Wien, im Februar 1999

To my little sister Rebekka

Abstract

This master's thesis applies artificial intelligence methods to ABALONE, a two-player strictly competitive game of perfect information. It assumes no prior knowledge of artificial intelligence. Reinforcement learning (RL) is the problem faced by an agent that learns behaviour through trial-and-error interactions with a dynamic environment. This work gives a firm mathematical foundation of RL in combination with neural networks. A feedforward neural network is conceived that is able to learn on its own to play ABALONE, without the aid of an intelligent teacher. Experiments are conducted to determine its level of play and the influence of various learning parameters. The work is completed by the development and analysis of advanced game-searching algorithms.

Keywords: Reinforcement Learning, Neuro-Dynamic Programming, Neural Network, Abalone, Game, Game-searching.

Contents

1	Introduction	1
1.1	Multilayer Perceptrons	2
1.2	Game-Searching Theory	5
1.3	Value of a Strictly Competitive Game	6
1.4	Genetic Algorithms	7
1.5	Case-Based Reasoning	9
2	Neuro-dynamic Programming	12
2.1	Dynamic Programming	13
2.1.1	Principal elements	13
2.1.2	Infinite Horizon Problems	15
2.2	Stochastic Shortest Path Problems	16
2.2.1	General Theory	16
2.2.2	Value Iteration	18
2.2.3	Policy Iteration	19
2.2.4	Temporal Difference-Based Policy Iteration	21
2.2.5	Dynamic Sequential Games	23
2.3	Neuro-Dynamic Simulation Methods	23

2.3.1	Simulation and Training	24
2.3.2	Lookup Table Representation	25
2.3.3	Approximate Policy Evaluation	26
2.3.4	Approximate Policy Evaluation using $TD(\lambda)$	27
2.3.5	Approximate Optimistic $TD(\lambda)$ Policy Iteration	29
2.3.6	Approximation in Dynamic Games	32
3	Malin – An Abalone Playing Program	33
3.1	First steps	34
3.2	Installation	34
3.3	Start a New Game	35
3.4	Playing a Game	36
3.5	Tools	38
3.6	How to ... ?	39
3.7	Files	39
4	Game-Searching Algorithms	41
4.1	Game-Playing Algorithms	41
4.1.1	Position Evaluator	41
4.1.2	Move Generator	42
4.1.3	Game-Searching Algorithms	43
4.1.4	Deadlock Situation	44
4.2	Position Evaluation by Feature Extracting	45
4.2.1	Three-Player Games	47
4.3	Search Algorithms	48
4.3.1	Brute Force Negamax Search	48

4.3.2	Quiescence Search	50
4.3.3	α - β Search	53
4.3.4	Killer Moves	56
4.3.5	Iterative Search	58
4.3.6	Hash Tables	60
4.3.7	Comparison	63
5	Neuro-Dynamic Game-Playing Algorithms	72
5.1	Theory	73
5.2	Multilayer Perceptron	76
5.3	Optimistic TD(λ)	77
5.3.1	Online	78
5.3.2	Offline	83
5.4	Reinforcement Player	84
5.5	Features	87
5.6	State-Space Exploration	90
5.7	Experimentation	92
6	Conclusion	98
6.1	Further research	98
6.2	Aftereffects	99
A	Abalone – Rules of the Game	101
B	Malin – Detailed Design	107
	Bibliography	111

List of Figures	115
List of Tables	117
List of Algorithms	118
Index	119

Preface

The game of ABALONE has accompanied me throughout my studies. In 1991, with seventeen, I developed my first ABALONE program; it focused primarily on graphical gadgets but was not able to play by itself. This gap was filled by the second version, accomplished during my first years at university. Although its algorithms completely lacked any theoretical foundation and were – to be euphemistic – original, achieved the program beginner level. The third attempt was baptized MALIN 3 and has become the subject of this master’s thesis. At this point Ms-Dos & Turbo C have been replaced by Windows 98 & Visual C++, and a number of advanced scientific game-searching algorithms were applied to ABALONE.

Despite the high playing level of the new algorithms, they still lacked the ability to profit from previous experiences. My research in matters of artificial intelligence remained fruitless until the discovery of the *reinforcement learning* paradigm. Tesauro’s [Tes95] success in breeding a neural network beating all previous Backgammon programs, motivated the creation of a new generation of ABALONE players – which proved to outperform all its predecessors!

I would like to thank Alexander Mehlmann, who not only gave me the decisive hint, but always supported and encouraged me throughout the last year.

Vienna, February 1999

Chapter 1

Introduction

The basic paradigm of *reinforcement learning* is as follows (see Tesauro [Tes95]). The learning agent observes an *input state* or input pattern, it produces an *output signal* (most commonly thought of as an *action* or *control signal*), and it receives an scalar *reward* or reinforcement *feedback signal* from the environment indicating how good or bad its output was. The goal of learning is to generate the optimal actions leading to maximal reward. In many cases the reward is also delayed (i.e., is given at the end of a long sequence of inputs and outputs). In this case the learner has to solve what is known as the *temporal credit assignment* problem (i.e., it must figure out how to apportion credit and blame to each of the various inputs and outputs leading to the ultimate final reward signal).

The reinforcement learning paradigm has held great intuitive appeal, and has attracted considerable interest for many years because of the notion of the learner being able to learn on its own, without the aid of an intelligent teacher, from its own experience at attempting to perform a task. In contrast, in the more commonly employed paradigm of supervised learning, a “teacher signal” is required that explicitly tells the learner what the correct output is for every input pattern.

The first large-scale success was Tesauro’s famous Backgammon-playing program TD-Gammon [Tes94]. TD-Gammon is a neural network that trains itself to be an evaluation function for the game of Backgammon by playing against itself and employing the reinforcement learning paradigm. TD-Gammon has greatly surpassed all previous computer programs in its ability to play Backgammon and in at least some

cases, appears to surpass the positional judgement of world-class human players. The tremendous success of TD-Gammon motivated the application of reinforcement methods to the game of ABALONE. Although the prospects were less promising than for Backgammon (which is in contrast to ABALONE a *non-deterministic* game), the developed reinforcement players turned out to play extremely well, beating its conventional counterparts by far!

This work assumes *no* prior knowledge of artificial intelligence; however, some basic mathematical notions, particularly of optimization theory, dynamic programming and probability theory will be helpful. Familiarity with the programming language C will facilitate the understanding of the pseudo-code used to demonstrate various algorithms.

The rest of this chapter reviews briefly some interesting techniques, concepts or theoretical results related to game-playing. Chapter 2 gives a firm mathematical foundation of the reinforcement learning paradigm in combination with neural networks, summarized under the title *Neuro-Dynamic Programming*. Chapter 3 presents the *User Guide* of the developed ABALONE playing computer program baptized MALIN, while Chapter 4 concentrates on advanced *Game-Searching Algorithms*. These are extended in Chapter 5 (*Neuro-Dynamic Game-Playing Algorithms*) to develop step by step a neural network which learns to play ABALONE.

This paper as well as the developed ABALONE playing program can be found at <http://www.math.tuwien.ac.at/OR/Mehlmann/Diplomarbeiten/schorr.htm>.

1.1 Multilayer Perceptrons

Neural networks are the most popular tool developed by the artificial intelligence community. Neural networks are composed of many nonlinear computational elements (*neurons*) operating in parallel and arranged in patterns reminiscent of biological neural nets. The neurons or nodes are connected via weights (*synapses*) that are typically adapted during use to improve performance. Neural networks are used for many purposes; in our context we only need a network *approximating a function* $f : \mathbb{R}^n \rightarrow \mathbb{R}$. This short introduction focuses on such a neural network, the *multilayer perceptron*, and is based on Dawid [Daw96] and Bertsekas and Tsitsiklis [BT96].

Each neuron h is connected via the synapses $r(i, h)$ to numerous other neurons i ; a

neuron's output x_h is computed by

$$x_h = \sigma \left(\sum_{i=1}^I r(i, h) x_i \right), \quad (1.1)$$

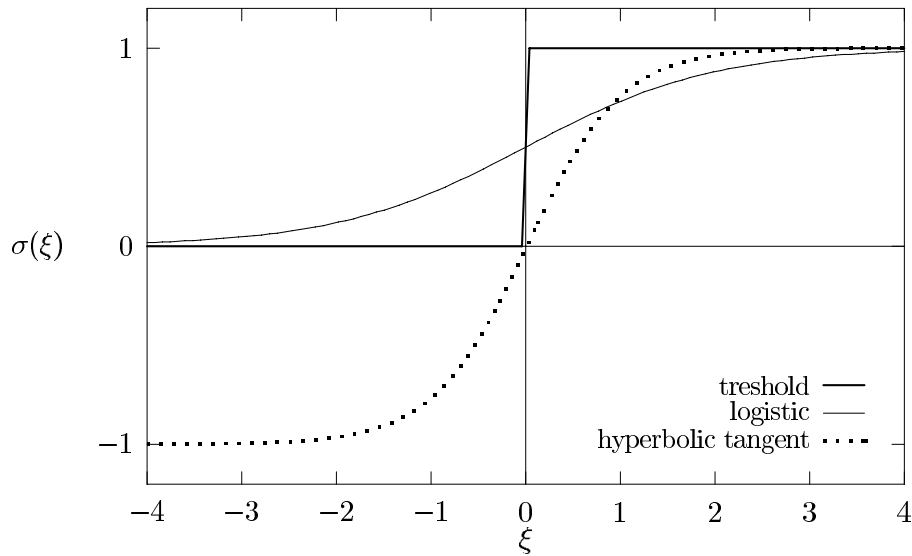
where x_i , $i = 1, \dots, I$ are the outputs of all neurons connected with neuron h , $\sigma(\cdot)$ is the *activation* or *sigmoidal function*. If the weight $r(i, h)$ is positive the synapse is called excitatory, if the weight is negative the synapse is called inhibitory. The sum of input signals $\sum_{i=1}^I r(i, h) x_i$ is called the *activation level* of the neuron x_h . Figure 1.1 depicts several common choices for the activation function; the threshold function, the logistic function

$$\sigma(\xi) = \frac{1}{1 + e^{-\xi}},$$

and the hyperbolic tangent function

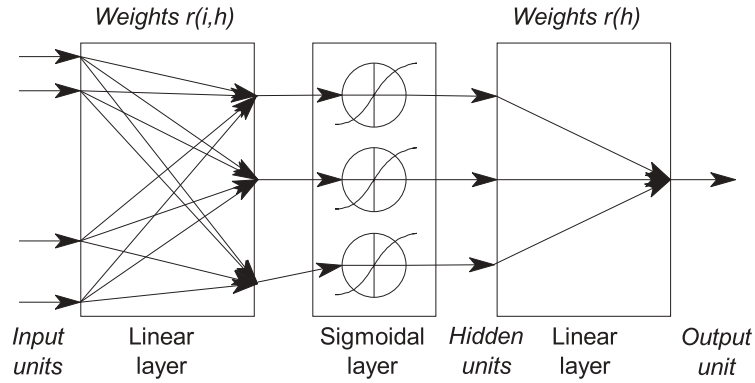
$$\sigma(\xi) = \tanh(\xi) = \frac{e^{\xi} - e^{-\xi}}{e^{\xi} + e^{-\xi}}.$$

Figure 1.1: Several activation functions



The architecture or topology of a neural network is determined by the way the neurons are interconnected. The number of network topologies is virtually infinite, we will here only cope with *feedforward networks* or *multilayer perceptron* with a *single hidden layer*; see Figure 1.2. Under this architecture the weight vector r consists of the

Figure 1.2: A multilayer perceptron with one hidden layer



A 4–3–1 perceptron.

The 4 input neurons are transformed into 3 hidden neurons;
these are linear combined into the output neuron.

coefficients $r(i, h)$ and $r(h)$. An input state j is encoded as a vector with components $x_i(j)$, $i = 1, \dots, I$. These input neurons $x_i(j)$ are transformed through a linear and a sigmoidal layer, resulting in the H hidden neurons $y_h(j, r)$

$$y_h(j, r) = \sigma \left(\sum_{i=1}^I r(i, h) x_i(j) \right), \quad h = 1, \dots, H.$$

The hidden neurons $y_h(j, r)$ are linear combined using the weights $r(h)$ to produce the final output

$$z(j, r) = \sum_{h=1}^H r(h) y_h(j, r) = \sum_{h=1}^H r(h) \cdot \sigma \left(\sum_{i=1}^I r(i, h) x_i(j) \right).$$

We note that it is common practice to provide the constant 1 as an additional input to the linear layer, or equivalently to fix one of the components of x at the value 1. This provides a tunable constant bias to each of the inputs of the hidden units, and expands the range of mappings that the architecture can effectively approximate.

There is an important result that relates to the approximation capability of the multilayer perceptron. It can be shown that this network, with a good choice of the weights r , can approximate arbitrarily closely any function $f : S \mapsto \mathbb{R}$ that is continuous over a closed and bounded set S , provided that the number of hidden units H is sufficiently large (see Hornik et al [HSW89]). For this, a single hidden layer is sufficient.

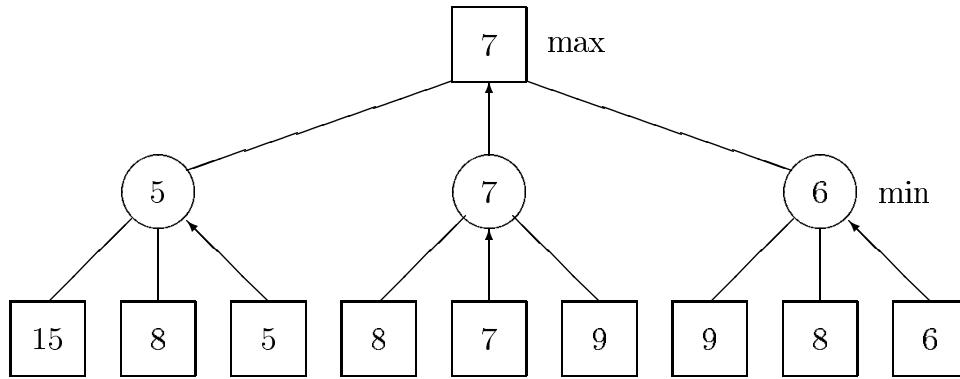
Section 5.2 demonstrates the implementation of a simple multilayer perceptron.

1.2 Game-Searching Theory

This section summarizes briefly analytical investigations of the performance of game-searching algorithms (see Pearl [Pea84]). Detailed explanations and performance comparisons of the employed algorithms can be found in Section 4.3.

The model most frequently used for evaluating the performance of game-searching methods consists of a uniform tree of depth n (n even) and degree M , where the terminal positions are assigned random values independently drawn from a common distribution F . We shall refer to such a tree as a (n, M, F) -tree, see Figure 1.3. The

Figure 1.3: A $(2, 3, F)$ -tree



A game-tree of depth $n = 2$ and degree $M = 3$.

expected number of terminal nodes examined during the search and its branching factor have become standard criteria for the complexity of the search method.

Definition 1.1 (Branching Factor) Let A be a deterministic algorithm which searches the (n, M, F) -game and let $I_A(n, M, F)$ denote the expected number of terminal positions examined by A . The quantity

$$R_A(M, F) = \lim_{n \rightarrow \infty} \sqrt[n]{I_A(n, M, F)} \quad (1.2)$$

is called the *branching factor* corresponding to the algorithm A .

The α - β search algorithm is the most commonly used procedure in game-playing applications (see Section 4.3.3). It can be shown that the number of terminal nodes examined by α - β must be at least $M^{\lfloor n/2 \rfloor} + M^{\lceil n/2 \rceil} - 1$ but may, in the worst case reach the entire set of M^n terminal nodes. For moderate values of M ($M \leq 1000$) the α - β branching factor can be approximated by

$$R_{\alpha-\beta}(M, F) = 0.925 \cdot M^{0.747}. \quad (1.3)$$

Roughly speaking, a fraction of only $0.925 \cdot M^{0.747}/M \approx M^{-1/4}$ of the legal moves will be explored by α - β . Alternatively, for a given search time allotment, the α - β pruning allows the search depth to be increased by a factor $\log M / \log R_{\alpha-\beta} \approx 4/3$ over that of an exhaustive minimax search. Note that these results ignore the impacts of possible *move ordering* as introduced in Section 4.3.4. Effectively, the branching factor of the best search algorithm developed in this work is bounded by $R_{\text{Hash}} \leq M^{0.581}$ (see Section 4.3.7).

Although search algorithms other than α - β have been introduced (SSS* by Stockman [Sto79], SCOUT by Pearl [Pea84]), α - β has been proven to be asymptotically optimal of all search algorithms. This, and its modest memory requirements, make α - β the only used search algorithm in this (and the most other) game-playing programs.

1.3 Value of a Strictly Competitive Game

Game theory is a mathematical discipline analyzing *rational* actions of individuals whose actions affect each other. Game theory proved to be very useful in economic applications, but it provides few insights into very complex games, such as Chess or ABALONE. However, there is an interesting result concerning the possible outcomes of a game, which will be stated below. For an introduction to game theory see Mehlmann [Meh97], or Binmore [Bin92], for combinatorial game theory [BCG82].

Let G be a *two-player, strictly competitive game of perfect information*. This simply means that the players' aims are diametrically opposed and that each player knows everything about what has happened in the game so far. The set U_G contains all possible outcomes of the game G , $U_G = \{u_1, u_2, \dots, u_K\}$. Further it will be assumed that player I is not indifferent between any pair of outcomes and that the outcomes

u_k in the set U_G are labeled so that

$$u_1 \prec_1 u_2 \prec_1 \cdots \prec_1 u_K,$$

where $a \prec_1 b$ means that player I prefers b to a .

Definition 1.2 (Value a of Strictly Competitive Game) An outcome v will be said to be a *value* of a finite, strictly competitive game G if and only if player I can force an outcome in the set $W_v = \{u : u \succeq_1 v\}$ and player II can force an outcome in the set $L_v = \{u : u \preceq_1 v\}$.

Theorem 1.1 Any finite, strictly competitive game of perfect information without chance moves has a value. *No proof*

ABALONE is a two-player, strictly competitive game of perfect information. If the rules are altered slightly in order to avoid deadlock situations, only two possible outcomes remain, $U_A = \{u_1, u_2\}$, where u_i denotes the win of player i , and thus $u_1 \succ_1 u_2$. This leaves two possibilities for the value v of ABALONE. If $v = u_1$, player I wins every game, if $v = u_2$ player II wins every game. However, although Theorem 1.1 proves that ABALONE has a value, it is still unknown. Note, that a “symmetry argument” aiming to prove that the value v cannot be u_2 , is erroneous. The situation in Chess is similar.

1.4 Genetic Algorithms

Genetic algorithms (GA) were developed as a tool to find solutions of optimization problems in poorly understood large spaces. They are based on genetic processes of biological organisms, particularly on the principle of natural selection that has become famous as *survival of the fittest*. This short overview is based on Dawid [Daw96], an excellent introduction can be found in Michalewicz [Mic96]; the implementation was facilitated by the genetic algorithms software library GALib [Gal96].

A genetic algorithm is a search algorithm based on the principles of natural evolution. A *population* of strings is considered, where each string encodes an admissible input of the surrounding system. This input may be a solution of an optimization problem or a game-playing strategy. Every input gets some reward from the surrounding system,

the so called *fitness value* of the string. Using these fitness values a new population of strings is generated by applying some genetic operators to the old population. Starting with a randomly initialized population, this iteration is carried out for a given number of periods. Besides the principle of natural selection, GAs imitate not only the spreading of genetic material in a population but also the generation of new genetic material by mutations.

Three genetic operators make up for the behaviour of a population of strings.

Selection operator The selection operator determines which of the strings in the current population will be allowed to inherit their genetic material to the next generation. The standard selection operator, called proportional selection, does this by carrying out n random draws with replacement out of the current population P_t . The probability that a given member of the current population is chosen at a draw is proportional to the fitness of the string. By the use of this (or other) selection operators an intermediate population, the *mating pool* is produced. The following two operators, crossover and mutation, are applied to this mating pool in order to generate the new population P_{t+1} .

Crossover The crossover operator is the key operator to generate new individuals in the population. Inspired by the example of nature, crossover is intended to join the genetic material of strings with high fitness in order to produce new and better individuals. In the simplest case of one-point crossover, one crossover point is drawn randomly between 1 and $l - 1$, where l is the constant number of bits per string. Two parent strings are chosen from the mating pool, and their bits to the right of the crossover point are swapped, producing a pair of offspring.

Mutation The mutation operator allows the GA to find solutions, which contain bit values that are non-existent in the initial population. The parameter governing this operator is called mutation probability μ . After the application of crossover each bit in any string is inverted with probability μ . Mutation probabilities are in general of order 10^{-3} . While the selection operator reduces the diversity in the population, does the mutation operator increase it again. The higher the mutation probability, the smaller is the danger of premature convergence.

For the application to the game of ABALONE an extension of genetic algorithms, called *evolutionary programming* (see [Mic96]) was employed. A population is a set

of ABALONE computer players; a players string corresponds to the parameters of its position evaluator. The goal is to determine the best position evaluator, which is equivalent to the best player. The problem is the definition of a players fitness.

In function approximation tasks the fitness of a string is equivalent to its difference from the target function $f(\cdot)$. In the case of game-playing on the contrary, no “best strategy”, which could be used as a reference, is available. Instead the fitness of a player is defined as the percentage of games it wins in a tournament against a set of “representative opponents”. These opponents are typically all members of the current population P_t . The problem is that GAs rely on a very large number of iterations (generations) and that this kind of fitness evaluation is too time consuming. There are also theoretical doubts, whether a fitness definition depending exclusively on the current population will generate the desired results. Additionally the outcome of a single ABALONE game depends to a substantial amount on randomness, which forges the results of the “tournament-fitness”.

Experimentation based on [Gal96] underpinned the above objections. Due to the time-consuming fitness evaluation, the size and the number of populations had to be kept fairly small. This rendered the genetic algorithm almost a random process, yielding no substantial ameliorations in the quality of play. This approach has therefore been abandoned in favor of self-trained neural networks, see Chapter 2.

1.5 Case-Based Reasoning

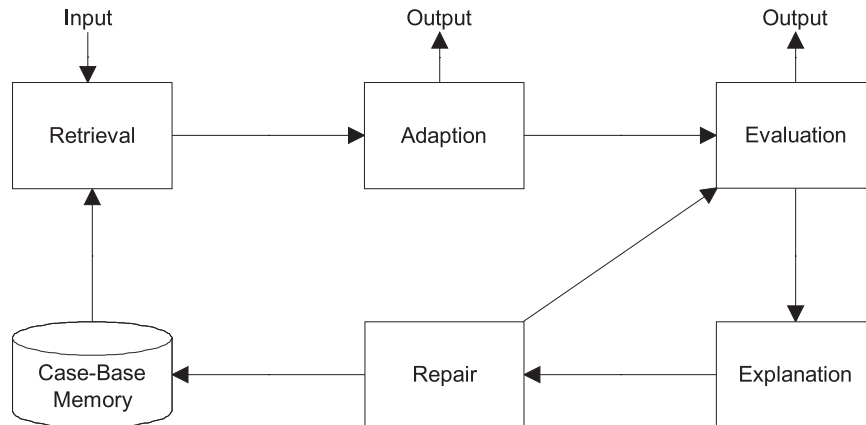
The following introduction to *case-based reasoning* (CBR) is based on Reiser [Rei94], who applied CBR to the game of ABALONE.

Case-based reasoning consists of concluding from preceding examples, of adapting foregoing solutions to solve new problems, or of retrieving old examples to illuminate aspects of the current situation [KR89].

Case-based reasoning is the attempt to simulate the “human” approach to problem solving by machines. Human behaviour is not exclusively governed by strictly defined rules (algorithms) but to a large extent by experience. Many skills are even primarily based on acquired experience – diagnosis and treatment of diseases, cooking, complex

management tasks, ...

Figure 1.4: Components of a case-based reasoning system



Case-Base Memory, Retrieval, and Adaption are mandatory,
Evaluation, Explanation and Repair are optional.

Figure 1.4 introduces the components of a typical case-base system.

Case-Base Memory Each case consists of a description of a problem and its solution. The memory organisation ranges from a simple array (and sequential search), to sophisticated search trees.

Retrieval The retrieval algorithm searches the case-base memory for the “most similar” case(s) to the current one.

Adaption The cases found by the retrieval algorithm have to be adapted for the current problem. Firstly the differences are detected, and then the retrieved solutions are mutated.

Evaluation (optional) This component examines if the derived solution is indeed a valid solution. This is not always possible.

Explanation (optional) Analyzes and describes eventually arising errors, thus providing the Repair component with important information.

Repair (optional) Repairs errors and ensures that they do not occur again.

Case-based reasoning has several advantages.

- It is not necessary to repeat already achieved deductions. Once a solution is found, it is stored in memory and reused at the next occasion.
- CBR is easily combined with other AI-methods, such as rule-based deduction.
- Knowledge engineering is often based on a set of examples, rather than algorithms. This knowledge can effortlessly be integrated in the case-base memory.
- It imitates humans, which perhaps facilitates its intuitive understanding.

Reiser [Rei94] proposes an ABALONE playing program based on case-based reasoning, completed by iterative α - β search. A case is a board position (problem) along with its best move (solution). A case-base is calculated offline, that is before the game starts. Whenever the move m_j for a given position j is to be calculated, the position j is looked up in the case-base memory. If a similar position j' is found, the corresponding move $m_{j'}$ is used. If no position j' similar to j can be found, or if j is an “unstable” position, an ordinary search algorithm is invoked. The systems response time is improved by keeping a history pointer j_h in the case-base memory which speeds up the retrieving of a suitable position j' .

Reiser claims that this algorithm outperforms ordinary search algorithms, if the admitted time budget is very small (“Blitz-tournaments”). However, it seems as if extensive α - β search remains still superior. This is why this approach was *not* pursued any further in this work.

Chapter 2

Neuro-dynamic Programming

Reinforcement learning is the problem faced by an agent that must learn behaviour through trial-and-error interactions with a dynamic environment. It dates back to early days of cybernetics and work in statistics, psychology, neuroscience, and computer science. In the last five to ten years (the early nineteen-nineties), new methods have attracted rapidly increasing interest in the machine learning and artificial intelligence communities. These methods are aiming to provide effective suboptimal solutions to complex problems of planning and sequential decision making under uncertainty, that for a long time were thought to be intractable. They claim to deal effectively with the dual curses of dynamic programming and stochastic optimal control: Bellman's *curse of dimensionality* (the exponential computational explosion with the problem dimension is averted through the use of parametric approximate representations of the cost-to-go function), and the *curse of modeling* (an explicit system model is not needed, and a simulator can be used instead). Furthermore, the methodology has a logical structure and a mathematical foundation. It draws on the theory of function, the theory of iterative optimization and neural network training, and the theory of dynamic programming. In view of the close connection with both neural networks and dynamic programming, Bertsekas and Tsitsiklis [BT96] settled on the name *neuro-dynamic programming* (NDP), which describes better the nature of the subject than the older and more broadly applicable name *reinforcement learning*.

This chapter is essentially a summary of Bertsekas and Tsitsiklis' excellent textbook on neuro-dynamic programming [BT96], focusing on topics relevant for game-playing algorithms. Its application to the game of ABALONE is discussed in Chapter 5, but

the presented theory is also applicable to non-deterministic games like Backgammon.

Section 2.1 gives a short introduction to *dynamic programming*; Section 2.2 concentrates on *stochastic shortest path problems*, which will later be used to model the game ABALONE. Finally Section 2.3 (labeled *neuro-dynamic simulation methods*) combines neural networks with dynamic programming.

2.1 Dynamic Programming

This section considers systems where decisions are made in stages. The outcome of each decision is not fully predictable but can be anticipated to some extent before the next decision is made. Each decision results in some immediate cost but also affects the context in which future decisions are to be made and therefore affects the cost incurred in future stages. We are interested in decision making policies that minimize the total cost over a number of stages. Such problems are challenging primarily because of the tradeoff between immediate and future costs. *Dynamic programming* (DP) provides a mathematical formalization of this tradeoff.

2.1.1 Principal elements

The principal elements of a problem in dynamic programming are,

1. *A discrete-time dynamic system whose state transition depends on a control.* Throughout this chapter, we assume that there are n states, denoted by $1, 2, \dots, n$, plus possibly an additional termination state, denoted by 0 . When at state i , the control must be chosen from a given finite set $U(i)$. At state i , the choice of a control u specifies the transition probability $p_{ij}(u)$ to the next state j .
2. *A cost that accumulates additively over time* and depends on the states visited and the controls chosen. At the k th transition, we incur a cost $\alpha^k g(i, u, j)$, where g is a given function, and α is a scalar with $0 < \alpha \leq 1$, called the *discount factor*. The meaning of $\alpha < 1$ is that future costs matter to us less than the same costs incurred at the present time. Discounted problems are of no interest for the analysis of typical games, we will therefore restrict this introduction to undiscounted models, where $\alpha = 1$.

If we are in state i and we choose control u , we move to state j with given probability $p_{ij}(u)$. The control u depends on the state i and the rule by which we select the controls is called a *policy*. Simultaneously with a transition from i to j under control u , it is not enough to look at the magnitude of the cost $g(i, u, j)$; we must also take into account the desirability of the next state j . We thus need a way to rank or rate states j . This is done by using the optimal cost (over all remaining states) starting from state j , which is denoted by $J^*(j)$ and is referred to as the optimal *cost-to-go* of state j . These costs-to-go can be shown to satisfy some form of *Bellman's equation*

$$J^*(i) = \min_u E[g(i, u, j) + J^*(j) \mid i, u], \quad \text{for all } i,$$

where j is the state subsequent to i , and $E[\cdot \mid i, u]$ denotes expected value with respect to j , given i and u . Generally, at each state i , it is optimal to use a control u that attains the minimum above. Thus controls are ranked based on the sum of the expected cost of the present period and the optimal expected cost of all subsequent periods.

We are interested in *stationary policies*, that is, functions μ mapping states into controls with $\mu(i) \in U(i)$ for all states i . Let us denote by i_k the state at time k . Once a stationary policy μ is fixed, the sequence of states i_k becomes a *Markov chain* with transition probabilities

$$P(i_{k+1} = j \mid i_k = i) = p_{ij}(\mu(i)).$$

We can distinguish between *finite horizon problems*, where the cost accumulates over a finite number of stages, say N , and *infinite horizon problems*, where the cost accumulates indefinitely. In N -stage problems the expected *cost of a stationary policy* μ , starting from an initial state i , is

$$J_N^\mu(i) = E \left[G(i_N) + \sum_{k=0}^{N-1} g(i_k, \mu(i_k), i_{k+1}) \mid i_0 = i \right],$$

where $G(i_N)$ is a terminal cost for ending up with final state i_N , and the expected value is taken with respect to the probability distribution of the Markov chain $\{i_0, i_1, \dots, i_N\}$. This distribution depends on the initial state i_0 and the stationary policy μ .

2.1.2 Infinite Horizon Problems

In infinite horizon problems, the total expected cost starting from an initial state i and using a stationary policy μ is

$$J^\mu(i) = \lim_{N \rightarrow \infty} E \left[\sum_{k=0}^{N-1} g(i_k, \mu(i_k), i_{k+1}) \mid i_0 = i \right].$$

The optimal cost-to-go starting from state i , is denoted by

$$J^*(i) = \min_{\mu} J^\mu(i).$$

The costs $J^*(i)$, $i = 1, \dots, n$, can be viewed as the components of a vector J^* , which is referred to as the *optimal cost-to-go vector*.

Three principal classes of infinite horizon problems can be distinguished:

Stochastic shortest path problems We assume that there is an additional state 0, which is a cost-free termination state; once the system reaches that state it remains there at no further cost. The structure of the problem is assumed to be such that termination is inevitable, at least under an optimal policy. Thus, the objective is to reach the termination state with minimal expected cost $J^\mu(i_0)$. The problem is in effect a finite horizon problem, but the length of the horizon may be random and may be affected by the policy being used.

Discounted problems Here, we assume that the discount factor $\alpha < 1$. We try to minimize the total cost

$$J^{\mu, \alpha}(i) = \lim_{N \rightarrow \infty} E \left[\sum_{k=0}^{N-1} \alpha^k g(i_k, \mu(i_k), i_{k+1}) \mid i_0 = i \right].$$

Note that the absolute one-stage cost $|g(i, u, h)|$ is bounded from above by some constant M , because of our assumption that i , u , and j belong to finite sets; this makes the cost-to-go $J^{\mu, \alpha}(i)$ well defined because it is the infinite sum of a sequence of numbers that are bounded in absolute value by the decreasing geometric progression $\alpha^k M$.

Average cost per stage problems Minimization of the total cost $J^\mu(i)$ makes sense only if $J^\mu(i)$ is finite for at least some policies μ and some initial state i . If this is not the case we can try to minimize the *average cost per stage*, given by

$$\lim_{N \rightarrow \infty} \frac{1}{N} J_N^{\mu, \alpha}(i),$$

which frequently turns out to be well defined as a limit and finite.

The rest of this chapter concentrates on stochastic shortest path problems; Chapter 5 shows how the game of ABALONE can be fitted into this model.

2.2 Stochastic Shortest Path Problems

Here, we assume that there is no discounting ($\alpha = 1$) and, to make the cost-to-go meaningful, we assume that there exists a *cost-free termination state*, denoted by 0. Once the system reaches that state, it remains there at no further cost, that is,

$$p_{00} = 1, \quad g(0, u, 0) = 0, \quad \forall u \in U(0).$$

We are interested in problems where reaching the termination state is inevitable, at least under an optimal policy. Thus, the essence of the problem is how to reach the termination state with minimum expected cost.

A special case of the stochastic shortest path problems is worth mentioning. *The deterministic shortest path problem*, obtained when each control at a given state leads with certainty to a unique successor state. This problem is usually defined in terms of a directed graph consisting of n nodes plus a destination node 0, and a set of arcs (i, j) , each having a given cost c_{ij} . The cost of a path consisting of a sequence of arcs is the sum of the costs of the arcs of the path. The objective is to find among all paths that start at a given node and end at the destination, a path that has minimum cost; this is also called a *shortest path*. If we identify nodes with states, the outgoing arcs from a given node with the controls associated with the state corresponding to the node, and the costs of the arcs with the costs of the corresponding transitions, we obtain a special case of the problem of this section.

2.2.1 General Theory

In order to guarantee the inevitability of termination under an optimal policy, we introduce certain conditions that involve the notion of a *proper policy*; that is a stationary policy that leads to the termination state with probability one, regardless of the initial state. A stationary policy that is not proper is said to be *improper*.

Throughout this section we assume the following.

Assumption 2.1 There exists at least one proper policy.

Assumption 2.2 For every improper policy μ , the corresponding cost-to-go $J^\mu(i)$ is infinite for at least one state i .

When specialized to the classical (deterministic) shortest path problem, Assumption 2.1 is equivalent to assuming the existence of at least one path from each node to the destination in the underlying graph, while Assumption 2.2 is equivalent to assuming that all cycles in the underlying graph have positive cost. These are the typical conditions under which deterministic shortest path problems are analyzed.

Note that a simple condition that implies Assumption 2.2 is that the expected one-stage cost $\sum_{j=0}^n p_{ij}(u)g(i, u, j)$ is strictly positive for all states $i \neq 0$ and $u \in U(i)$. Another important case where Assumptions 2.1 and 2.2 are satisfied is when *all* policies are proper, that is when termination is inevitable under all stationary policies.

A useful mapping

The form of DP algorithm motivates the introduction of two mappings that play an important theoretical role and provide a convenient shorthand notation in expressions that would be too complicated to write otherwise.

For any vector $J = (J(1), \dots, J(n))$, we consider the vector TJ obtained by applying one iteration of the DP algorithm to J ; the components of TJ are

$$(TJ)(i) = \min_{u \in U(i)} \sum_{j=0}^n p_{ij}(u) (g(i, u, j) + J(j)), \quad i = 1, \dots, n,$$

where we will always use the convention that $J(0) = 0$ instead of viewing $J(0)$ as a free variable. Note that T can be viewed as a mapping that transforms a vector J into the vector TJ . Furthermore TJ is the optimal cost-to-go vector for the one-stage problem that has one-stage cost g and terminal cost J .

Similarly, for any vector J and any stationary policy μ , we consider the vector $T_\mu J$ with components

$$(T_\mu J)(i) = \sum_{j=0}^n p_{ij}(\mu(i)) (g(i, \mu(i), j) + J(j)), \quad i = 1, \dots, n.$$

We will denote by T^k the composition of the mapping T with itself k times; that is, for all k we write

$$(T^k J)(i) = (T(T^{k-1} J))(i), \quad i = 1, \dots, n.$$

Thus, $T^k J$ is the vector obtained by applying the mapping T to the vector $T^{k-1} J$. The components of $T_\mu^k J$ are defined similarly. The following proposition gives the main results regarding stochastic shortest path problems. The proof can be found in [Ber95].

Theorem 2.1 Consider the stochastic shortest path problem under Assumptions 2.1 and 2.2.

1. The optimal cost-to-go vector J^* has finite components and satisfies $J^* = TJ^*$. Furthermore, J^* is the only solution of the equation $J = TJ$.
2. We have $\lim_{k \rightarrow \infty} T^k J = J^*$, for every vector J .
3. A stationary policy μ is optimal if and only if $T_\mu J^* = TJ^*$.
4. For every proper policy μ and every vector J , the associated cost-to-go vector J^μ satisfies $\lim_{k \rightarrow \infty} T_\mu^k J = J^\mu$. Furthermore, $J^\mu = T_\mu J^\mu$, and J^μ is the only solution of this equation. *No proof*

We now discuss a number of different methods for computing the optimal cost-to-go vector J^* .

2.2.2 Value Iteration

The DP iteration that generates the sequence $T^k J$ starting from some J is called *value iteration* and is a principal method for calculating the optimal cost-to-go vector J^* . Generally, the method requires an infinite number of iterations. However, under special circumstances, the method can terminate finitely. A prominent example is the case of a deterministic shortest path problem, but there are other more general circumstances where termination occurs.

In particular, let us assume that the transition probability graph corresponding to some optimal stationary policy μ^* is *acyclic*. By this we mean that there are no cycles

in the graph that has as nodes the states $0, 1, \dots, n$, and has an arc (i, j) for each pair of states i and j such that $i \neq 0$ and $p_{ij}(\mu^*(i)) > 0$. Then it can be shown that the value iteration method will yield J^* after at most n iterations when started from the vector J given by

$$J(i) = \infty \quad i = 1, \dots, n.$$

In the value iteration method, the estimate of the cost-to-go vector is updated for all states simultaneously. An alternative is to update the cost-to-go at one state at a time, while incorporating into the computation the interim results. As long as $J(i)$ is iterated infinitely often for each state i , the validity of this method can also be proven.

Theorem 2.2 (Asynchronous Value Iteration Convergence)

Consider the algorithm that starts with an arbitrary initial vector J_0 and at the k th iteration, chooses an index $i_k \in \{1, \dots, n\}$, replaces the i_k th component of the current vector J_k with $(TJ_k)(i_k)$, and leaves all other components of J_k unchanged; that is

$$J_{k+1}(i) = \begin{cases} (TJ_k)(i), & \text{if } i = i_k, \\ J_k(i), & \text{otherwise.} \end{cases}$$

Assume that all components are chosen for iteration infinitely often. Then, the generated sequence of cost-to-go vectors J_k converges to J^* . *No proof.*

2.2.3 Policy Iteration

In general, value iteration requires an infinite number of iterations to obtain the optimal cost-to-go vector. There is an alternative to value iteration, called *policy iteration*, which always terminates finitely. In this algorithm, we start with a proper policy μ_0 and we generate a sequence of new proper policies $\mu_1, \mu_2, \dots$. Given the policy μ_k , we perform a *policy evaluation step*, that computes $J^{\mu_k}(i)$, $i = 1, \dots, n$, as the solution of the (linear) system of equations

$$J(i) = \sum_{j=0}^n p_{ij}(\mu_k(i)) (g(i, \mu_k(i), j) + J(j)), \quad i = 1, \dots, n, \quad (2.1)$$

in the n unknowns $J(1), \dots, J(n)$ (recall that $J(0)$ is set to zero). We then perform a *policy improvement step*, which computes a new policy μ_{k+1} as

$$\mu_{k+1}(i) = \arg \min_{u \in U(i)} \sum_{j=0}^n p_{ij}(u) (g(i, u, j) + J^{\mu_k}(j)), \quad i = 1, \dots, n, \quad (2.2)$$

or equivalently as the policy satisfying

$$T_{\mu_{k+1}} J^{\mu_k} = T J^{\mu_k}.$$

The process is repeated with μ_{k+1} used in place of μ_k , unless we have $J^{\mu_{k+1}}(i) = J^{\mu_k}(i)$ for all i , in which case the algorithm terminates with the policy μ_k . The following theorem establishes the validity of policy iteration.

Theorem 2.3 (Policy iteration convergence) The policy iteration algorithm generates an improving sequence of proper policies, that is $J^{\mu_{k+1}}(i) \leq J^{\mu_k}(i)$ for all i and k , and terminates with an optimal policy. *No proof*

In practice the policy iteration algorithm typically requires very few iterations, although there are no theoretical guarantees for this.

When the number of states is large, solving the linear system in the policy evaluation step by direct methods such as Gaussian elimination is time-consuming. One way to get around this difficulty is to solve the linear system iteratively by using value iteration. In fact, we may consider solving the system approximately by executing a limited number of value iterations. The resulting method is called the *modified policy iteration algorithm*.

To formalize this method, let J_0 be an n -dimensional vector such that $TJ_0 \leq J_0$. Let $m_0, m_1, \dots$ be positive integers, and let the vectors $J_1, J_2, \dots$ and the stationary policies $\mu_0, \mu_1, \dots$ be defined by

$$T_{\mu_k} J_k = TJ_k, \quad J_{k+1} = T_{\mu_k}^{m_k} J_k, \quad k = 0, 1, \dots \quad (2.3)$$

Thus a stationary policy μ_k is defined by from J_k according to $T_{\mu_k} J_k = TJ_k$, and the cost-to-go J^{μ_k} is approximately evaluated by $m_k - 1$ additional value iterations, yielding the vector J_{k+1} , which is used in turn to define μ_{k+1} .

Theorem 2.4 (Modified Policy Iteration Convergence)

Assume that $TJ_0 \leq J_0$ and let (J_k, μ_k) be the sequence generated by Eq. (2.3). Then J_k converges to J^* . *No proof.*

Note that if $m_k = 1$ for all k in the modified policy iteration algorithm, we obtain the value iteration method, while if $m_k = \infty$ we obtain the policy iteration method, where the policy evaluation step is performed iteratively by means of value iteration.

It seems often to be best to take m_k larger than 1 according to some heuristic scheme. A key idea is that a value iteration involving a single policy (evaluating $T_\mu J$ for some μ and J) is much less expensive than an iteration involving all policies (evaluating TJ for some J), when the number of controls available at each state is large.

2.2.4 Temporal Difference-Based Policy Iteration

We mentioned earlier that when the number of states is large, it is usually preferable to carry out the policy evaluation step of the policy iteration method by using value iteration. A drawback of this approach, however, is that the value iteration algorithm may converge very slowly. This is true for many stochastic shortest path problems with a large number of states, as well as for discounted problems with a discount factor that is close to 1. We are thus motivated to use an approach whereby the discount factor is effectively reduced in order to accelerate the policy evaluation step. Remember that for stochastic shortest path problems the discount factor equals 1.

The idea underlying the approach is that the discount factor can be reduced without altering the cost-to-go of a given policy only if the expected value of the one-stage cost under that policy is equal to 0. We thus introduce a transformation that asymptotically induces this one-stage cost structure. The transformation is based on the notion of *temporal differences* (TD for short), which will be of major importance in the context of the simulation-based methods of section 2.3.

We now describe a policy-iteration like algorithm that maintains a policy-cost vector pair (J_k, μ_k) . We can view J_k as an approximation to the cost-to-go vector J^{μ_k} . In the typical iteration, given (J_k, μ_k) , we calculate μ_{k+1} by the policy improvement step

$$T_{\mu_{k+1}} J_k = TJ_k.$$

To calculate J_{k+1} , we define the TD associated with each transition (i, j) under μ_{k+1} :

$$d_k(i, j) = g(i, \mu_{k+1}(i), j) + J_k(j) - J_k(i). \quad (2.4)$$

We also consider a parameter λ from the range $[0, 1]$. We view $d_k(i, j)$ as the one-stage cost of policy μ_{k+1} for an λ -discounted DP problem with the transition probabilities $p_{ij}(\mu_{k+1}(i))$ of the original problem, and we calculate the corresponding cost-to-go

vector. This vector, denoted by Δ_k , has components given by

$$\Delta_k(i) = E \left[\sum_{m=0}^{\infty} \lambda^m d_k(i_m, i_{m+1}) \mid i_0 = i \right], \quad \forall i. \quad (2.5)$$

The vector J_{k+1} is then obtained by

$$J_{k+1} = J_k + \Delta_k. \quad (2.6)$$

We refer to the method as the λ -policy iteration method.

Note that when $\lambda = 1$, we obtain $J_{k+1} = J^{\mu_{k+1}}$, so the 1-policy iteration method coincides with the standard policy iteration method. However, for $\lambda < 1$, the two methods are different, and in fact it can be seen from Eqs. (2.4)-(2.6) that the 0-policy iteration method coincides with the value iteration method for the original problem.

The motivation for the λ -policy iteration method is that the λ -discounted policy evaluation step can be much easier when $\lambda < 1$ than when $\lambda = 1$. In particular, when value iteration is used for policy evaluation, convergence is faster when $\lambda < 1$ than when $\lambda = 1$. Similarly, when policy evaluation is performed using a simulation approach, as in the NDP methods to be discussed in section 2.3, the variance of the cost samples that are averaged by simulation is typically smaller when $\lambda < 1$ than when $\lambda = 1$, as can be seen from the definition of Δ_k , cf. Eq. (2.5). As a result, the number of cost samples that are required to evaluate by simulation the cost-to-go of a given policy within a given accuracy may be much smaller when $\lambda < 1$ than when $\lambda = 1$.

On the negative side, the asymptotic convergence rate of the sequence J_k produced by the λ -policy iteration method deteriorates as λ becomes smaller. However, this disadvantage may not be very significant within the NDP approximation context to be discussed later. In particular, when the policy evaluation step cannot be performed with high accuracy, a reasonable practical objective is to obtain a policy whose cost is within the best “achievable” tolerance from the optimum, rather than to obtain the optimal cost-to-go function and an optimal policy. Under these circumstances, the asymptotic rate of convergence of J_k may not be crucial, and using $\lambda < 1$ often requires a comparable number of policy iterations to attain the same performance level as using $\lambda = 1$.

Theorem 2.5 (TD-Based Policy Iteration Convergence)

Assume $\lambda \in [0, 1)$, $TJ_0 \leq J_0$ and let (J_k, μ_k) be the sequence generated by the λ -policy iteration algorithm. Then J_k converges to J^* . *No proof*

2.2.5 Dynamic Sequential Games

In this subsection we introduce a generalization of the stochastic shortest path DP model to the case where there is an opponent who aims to maximize the cost-to-go. We more precisely focus on an important subclass, where the *Minimizer* and *Maximizer* take turns in selecting the controls u and v respectively. The choice of each player is selected with full knowledge of preceding choice of the other player and the state transition resulting from that choice. We refer to this type of problem as a *dynamic sequential game* with perfect information.

To formalize this concept we introduce a state space of the form $\underline{S} \cup \overline{S}$. At a state i of $\underline{S}$, only the Minimizer is allowed to choose a $u \in U(i)$ and the system then moves to a state $j \in \overline{S}$ with probability $\underline{p}_{ij}(u)$. Similarly, at a state i of $\overline{S}$, only the Maximizer is allowed to choose a $v \in V(i)$ and the system then moves to a state $j \in \underline{S}$ with probability $\overline{p}_{ij}(v)$. We continue to assume that the sets $\underline{S}$, $\overline{S}$, $U(i)$, and $V(i)$ are all finite.

The mapping T can be defined by

$$(TJ)(i) = \begin{cases} \min_{u \in U(i)} \sum_{j \in \overline{S}} \underline{p}_{ij}(u) (g(i, u, j) + J(j)), & \forall i \in \underline{S}, \\ \max_{v \in V(i)} \sum_{j \in \underline{S}} \overline{p}_{ij}(v) (g(i, v, j) + J(j)), & \forall i \in \overline{S}, \end{cases} \quad (2.7)$$

As in the one-player game there exists a cost-free and absorbing termination state. If we assume that this state is reached with probability 1 under all policies for the Minimizer and the Maximizer, the validity of various value iteration and policy iteration methods can be proven. For example a value iteration method starting from some vector J and sequentially generating $TJ, T^2J, \dots$ will converge to

$$\lim_{k \rightarrow \infty} T^k J = J^*,$$

where $J^*(i)$ is the *game-theoretic value* of the game at state i (see Section 1.3).

2.3 Neuro-Dynamic Simulation Methods

The computational methods for dynamic programming problems that were described in Section 2.2 apply when there is an explicit model of the cost structure and the

transition probabilities of the system. However, such a model is often not available but instead the system and the cost structure can be simulated. By this we mean that the state space and the control space are known and there is a computer program that simulates, for a given control u , the probabilistic transitions from any given state i to a successor state j according to the transition probabilities $p_{ij}(u)$, and also generates the corresponding transition cost $g(i, u, j)$. It is then of course possible to use repeated simulation to calculate (at least approximately) the transition probabilities of the system and the expected cost per stage by averaging, and then to apply the dynamic programming methods discussed in Section 2.2.

The methodology discussed in this section, however, is geared towards an alternative possibility, which is much more attractive when one contemplates approximations: the transition probabilities are not explicitly estimated, but instead the cost-to-go function of a given policy is progressively calculated by *generating several sample system trajectories* and associated costs.

We are interested in problems with a large number of states where a tabular description of J^* is impossible. We will therefore replace the optimal cost-to-go $J^*(j)$ with a suitable approximation $\tilde{J}(j, r)$, where r is a vector of parameters. The function $\tilde{J}(\cdot, r)$ will be called the *approximate cost-to-go function*, and the value $\tilde{J}(j, r)$ will be called the *approximate cost-to-go* of state j . The general form of $\tilde{J}$ is known and is such that once the parameter vector r is fixed, the evaluation of $\tilde{J}(j, r)$ for any state j is fairly simple. Any type of universal approximator of nonlinear mappings can be used, i.e., radial basis functions, multilayer perceptrons, wavelets, polynomials, splines, ... In the following we will use the term *neural network* for all of these approximators.

Approximating cost-to-go functions involving few parameters will be called *compact representations*, while the tabular description of J^* will be called the *lookup table representation*. In a lookup table representation, the values $J^*(j)$ for all states j are stored in a table.

2.3.1 Simulation and Training

Some of the most successful applications of neural networks are in the areas of pattern recognition, nonlinear regression, and nonlinear system identification. In these applications the neural network is used as a universal approximator; the input-output

mapping of the neural network is matched to an unknown nonlinear mapping F of interest using a least-squares optimization, known as *training the network*. To perform training, one must have training data, that is, a set of pairs $(i, F(i))$, which is representative of the mapping F that is approximated.

It is important to note that in contrast with these neural network applications, in the DP context there is no readily available training set of input–output pairs $(i, J^*(i))$ that could be used to approximate J^* with a least squares fit. The only possibility is to evaluate (exactly or approximately) by simulation the cost-to-go functions of given (suboptimal) policies, and to try to iteratively improve the policies based on the simulation outcomes. This creates analytical and computational difficulties that do not arise in classical neural network training contexts. Indeed the use of simulation to evaluate approximately the optimal cost-to-go function is a key new idea that distinguishes the methodology of neuro-dynamic programming from earlier approximation methods in DP.

Simulation offers another major advantage: it can implicitly identify the *most important* or *most representative* states of the system. It appears plausible that these states are the ones most often visited during simulation, and for this reason the scoring function will tend to approximate better the optimal cost-to-go function for these states, and the suboptimal policy obtained will on the average perform better.

2.3.2 Lookup Table Representation

A *lookup table representation* of the cost-to-go function J keeps in memory a separate variable $J(i)$ for each state i . Consider for example the policy evaluation step (2.1) used in the policy iteration method:

$$J(i) = \sum_{j=0}^n p_{ij}(\mu(i)) (g(i, \mu(i), j) + J(j)), \quad i = 1, \dots, n \quad (2.8)$$

where μ is a fixed stationary policy and we wish to calculate the corresponding cost-to-go vector. Instead of (2.8) we want to update $J(i)$ by simulation. Suppose that we perform a simulation run using the policy μ that generates the state trajectory $(i_0, i_1, \dots, i_N)$ where i_N is the termination state 0. Then we can update the estimates

$J(i_k)$ by using for each $k = 0, \dots, N - 1$, the formula

$$\begin{aligned} J(i_k) &:= J(i_k) + \gamma(i_k) (g(i_k, i_{k+1}) + g(i_{k+1}, i_{k+2}) + \dots + g(i_{N-1}, i_N) - J(i_k)) \\ &= J(i_k) - \gamma(i_k) \left(J(i_k) - \sum_{m=k}^{N-1} g(i_m, i_{m+1}) \right), \end{aligned} \quad (2.9)$$

where the stepsize $\gamma(i_k)$ is allowed to change from one iteration to the next. There are many choices for the stepsizes $\gamma(i_k)$ under which the algorithm remains sound.

It can be shown that the values $J(i)$ generated by (2.9) are guaranteed to converge to $J^\mu(i)$, with probability 1, provided that each state is visited by infinitely many trajectories and the stepsize diminishes towards zero at a suitable rate. Similar results are available for simulation algorithms corresponding to value iteration methods (Section 2.2.2) and temporal difference methods (Section 2.2.4).

The number of states in the problem we are interested in, is much too high for a lookup table representation of J , for the rest of this chapter we will therefore consider *compact representations*.

2.3.3 Approximate Policy Evaluation

Generally, we are interested in approximations of the cost-to-go function J^μ of a given policy μ , or of the optimal cost-to-go function J^* . This is done using a function which, given a state i , produces an approximation $\tilde{J}(i, r)$ of $J^\mu(i)$ or $J^*(i)$. The approximating function involves a parameter vector r , and may be implemented using a neural network, a feature extraction mapping, or any other suitable architecture. The parameter r is determined by optimization, often using some type of least square framework.

This subsection demonstrates how the policy evaluation step carried out for a lookup table representation in Section 2.3.2 is implemented with an approximate cost-to-go function $\tilde{J}(\cdot, r)$. Suppose that we have a fixed a proper policy μ , a set of representative states $\tilde{S}$, and that for each $i \in \tilde{S}$, $c(i)$ is a sample of the cost $J^\mu(i)$. r is a parameter vector to be determined by solving the least squares optimization problem

$$\min_r \sum_{i \in \tilde{S}} \left(\tilde{J}(i, r) - c(i) \right)^2. \quad (2.10)$$

This least squares problem can be solved by an incremental gradient update rule,

$$r := r - \gamma \nabla_r \tilde{J}(i, r) \left(\tilde{J}(i, r) - c(i) \right), \quad (2.11)$$

where γ is a stepsize. In order to approximate $J^\mu(i)$ by $\tilde{J}(i, r)$, we repeat the update rule 2.11 for all $i \in \tilde{S}$.

Given a sample state trajectory $(i_0, i_1, \dots, i_N)$ generated using the policy μ , the sample cost $c(i_k)$ is defined by

$$c(i_k) = \sum_{m=k}^{N-1} g(i_m, \mu(i_m), i_{m+1}).$$

The parameter vector r is then updated by

$$r := r - \gamma \sum_{k=0}^{N-1} \nabla_r \tilde{J}(i_k, r) \left(\tilde{J}(i_k, r) - \sum_{m=k}^{N-1} g(i_m, \mu(i_m), i_{m+1}) \right). \quad (2.12)$$

This update rule is called an *offline* variant because the parameter vector r is only updated at the end of the trajectory. If the approximate cost-to-go $\tilde{J}(\cdot, r)$ is already sufficiently close to J^μ the policy evaluation step is completed, otherwise a new sample state trajectory is generated and the procedure repeated. Note the formal similarities between the policy evaluation algorithm for lookup tables (2.9) and compact representations (2.12).

2.3.4 Approximate Policy Evaluation using TD(λ)

In the preceding subsection we discussed one possible method for approximate policy evaluation, here we develop an alternative implementation, in terms of temporal differences (TD). We will see that an algorithm called TD(1) can be interpreted as a type of incremental gradient method for solving the least squares problem associated with the cost-to-go approximation. We then modify TD(1) using a parameter λ , and we obtain a family of methods corresponding to the TD methods of Section 2.2.4. The motivation for TD(λ) is not as clear as in Section 2.2.4, and in fact we will argue that as λ becomes smaller, there is a tendency for the quality of the cost-to-go approximation to deteriorate. Nonetheless, there are reasons for interest in TD(λ), for $\lambda < 1$: there is empirical evidence that for some problems, particularly those where the cost-to-go have large variance, it converges faster and leads to better performance than that obtained from TD(1).

Throughout this subsection, we assume that a policy μ has been fixed. For this reason, we do not need to show the dependence of various quantities on μ , and we employ the notation $g(i, j)$.

TD(λ) for $\lambda = 1$

In the approximate policy evaluation step discussed in Section 2.3.3, we pick an initial state i_0 and generate a sample trajectory $(i_0, i_1, \dots, i_N)$. Once this trajectory is available we update the parameter vector r according to the incremental gradient iteration (2.12).

We define temporal differences d_k by

$$d_k = g(i_k, i_{k+1}) + \tilde{J}(i_{k+1}, r) - \tilde{J}(i_k, r), \quad k = 0, \dots, N-1, \quad (2.13)$$

and using the fact $\tilde{J}(i_N, r) = \tilde{J}(0, r) = 0$, the iteration (2.12) becomes

$$r := r + \gamma \sum_{k=0}^{N-1} \nabla_r \tilde{J}(i_k, r) (d_k + d_{k+1} + \dots + d_{N-1}). \quad (2.14)$$

Instead of waiting for the end of the trajectory to update r as above, we may perform the part of the update involving d_k as soon as d_k becomes available. This leads to the update equation

$$r := r + \gamma d_k \sum_{m=0}^k \nabla_r \tilde{J}(i_m, r), \quad k = 0, 1, \dots, N-1. \quad (2.15)$$

For example, following the state transition (i_0, i_1) , we set

$$r := r + \gamma d_0 \nabla_r \tilde{J}(i_0, r). \quad (2.16)$$

Following the state transition (i_1, i_2) , we set

$$r := r + \gamma d_1 \left(\nabla_r \tilde{J}(i_0, r) + \nabla_r \tilde{J}(i_1, r) \right), \quad (2.17)$$

and similarly for subsequent transitions.

If all updates of the vector r are performed at the end of a trajectory, the method is denominated *offline*. If on the other hand, an update is performed subsequent to each transition, according to Eqs. (2.15)-(2.17), we have an *online* method.

Note that in order to expect any kind of convergent behaviour, the stepsize γ should diminish over time. A popular choice is to let $\gamma = c/(m + d)$ in the course of the m th trajectory, where c and d are some positive constants. We should also stress the importance of proper sampling. In particular, the method used for picking the initial states can be crucial to the quality of the cost-to-go approximation constructed by TD(1). One generally expects the approximation to be better for those states that are more frequently visited, and the identity of these states depends strongly on the initialization of the simulated trajectories.

TD(λ) for general λ

While TD(1) is equivalent to approximate policy evaluation, will we now generalize from TD(1) to TD(λ), where λ is a parameter in $[0, 1]$. In an offline version of TD(λ), a trajectory $i_0, i_1, \dots, i_N$ is simulated and afterwards the parameter vector r is updated by letting

$$r := r + \gamma \sum_{m=0}^{N-1} \nabla_r \tilde{J}(i_m, r) \sum_{k=m}^{N-1} d_k \lambda^{k-m}.$$

In the online version, the terms involving d_k are used for a particular update of r , as soon as d_k becomes available. More specifically, for $k = 0, \dots, N - 1$, following the state transition (i_k, i_{k+1}) , we set

$$r := r + \gamma d_k \sum_{m=0}^k \lambda^{k-m} \nabla_r \tilde{J}(i_m, r).$$

While this method has received wide attention, its convergence behavior is unclear. In general, when $\lambda < 1$, TD(λ) resembles an incremental gradient method for the objective function $\sum_i \left(\tilde{J}(i, r) - J^\mu(i) \right)^2$, except that some error terms are added to the gradient.

2.3.5 Approximate Optimistic TD(λ) Policy Iteration

In policy iteration a sequence (r_t, μ_t) is generated by subsequently performing a policy evaluation and a policy improvement step. In a evaluation step we fix the policy μ_t and calculate r_t so that $\tilde{J}(\cdot, r_t)$ approximates J^{μ_t} ; this is the subject of Section 2.3.3 and 2.3.4. The policy improvement step updates the policy by switching to the new

policy μ_{t+1} which is greedy with respect to $\tilde{J}(\cdot, r_t)$. An alternative is to perform a policy update earlier, before the approximate evaluation of J^{μ_t} converges. An extreme possibility, is to use a simulation-based method for approximating J^{μ_t} and to replace μ_t with μ_{t+1} subsequent to the simulation of *every state trajectory* (offline variant) or even subsequent to *every state transition* (online variant). The resulting type of method, which we will call *optimistic policy iteration*, is often used in practice, sometimes with success, even though there is little understanding of its convergence behaviour. It is also the method we will use in Chapter 5, when implementing an ABALONE reinforcement learner.

Online Optimistic TD(λ)

We will describe here the TD(λ)-based variant of optimistic policy iteration that updates the current policy after every state transition (i_k, i_{k+1})

At a typical iteration, we are at some state i_k and we have a current parameter vector r_k . We select $u_k \in U(i_k)$ such that

$$u_k = \arg \min_{u \in U(i_k)} \sum_{j=1}^n p_{i_k j}(u) \left(g(i_k, u, j) + \tilde{J}(j, r_k) \right), \quad (2.18)$$

we generate the next state i_{k+1} by simulating a transition according to the transition probabilities $p_{i_k j}(u_k)$, and finally we perform the TD(λ) update

$$r_{k+1} = r_k + \gamma_k d_k \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_k),$$

where d_k is the temporal difference

$$d_k = g(i_k, u_k, i_{k+1}) + \tilde{J}(i_{k+1}, r_k) - \tilde{J}(i_k, r_k). \quad (2.19)$$

In fact, this method is not easily implemented because we need to evaluate a different gradient $\nabla \tilde{J}(i_m, r_k)$ for each k and m . For this reason, one uses instead the update rule

$$r_{k+1} = r_k + \gamma_k d_k \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_m). \quad (2.20)$$

The gradient $\nabla \tilde{J}(i_m, r_m)$ is evaluated just once, when state i_m is reached. If we define the *eligibility vector*

$$e_k = \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_m), \quad (2.21)$$

the algorithm takes the simpler form

$$r_{k+1} = r_k + \gamma_k d_k e_k, \quad (2.22)$$

The eligibility vector can also be defined iteratively,

$$e_{k+1} = \lambda e_k + \nabla \tilde{J}(i_{k+1}, r_{k+1}). \quad (2.23)$$

Note that if we set $\lambda = 0$, we get an algorithm of the form

$$r_{k+1} = r_k + \gamma_k d_k \nabla \tilde{J}(i_k, r_k).$$

Offline Optimistic TD(λ)

For the offline variant, suppose that we update the policy at the end of each trajectory. Let r_t be the value of the parameter vector at the beginning of the t th trajectory. We pick an initial state i_0 and simulate a trajectory $i_0, i_1, \dots, i_N$, using the policy μ_t defined by

$$\mu_t(i) = \arg \min_{u \in U(i)} \sum_j p_{ij}(u) \left(g(i, u, j) + \tilde{J}(j, r_t) \right), \quad \forall i.$$

Of course, we do not need to generate the policy for all states i ; it is only when the trajectory reaches a state i that we compute $\mu_t(i)$ as above. Then, at the end of the trajectory, we update the parameter vector r by letting

$$\begin{aligned} r_{t+1} &= r_t + \gamma_t \sum_{k=0}^{N-1} d_k \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_t) \\ &= r_t + \gamma_t \sum_{k=0}^{N-1} \nabla \tilde{J}(i_k, r_t) (d_k + \lambda d_{k+1} + \dots + \lambda^{N-1-k} d_{N-1}). \end{aligned} \quad (2.24)$$

2.3.6 Approximation in Dynamic Games

The NDP methodology with cost-to-go function approximation is not very well developed at present for dynamic games (see Section 2.2.5). However, there have been reports of success with some of the natural extensions of the one-player NDP methods to sequential games [Tes95]. An interesting and straightforward possibility is to implement an approximate version of the optimistic policy iteration algorithm. For example, a TD(λ)-based method that parallels the online method given in Section 2.3.5 operates as follows. At the typical iteration, we are at some state i_k and we have a current parameter vector r_k . If $i_k \in \underline{S}$, we select $u_k \in U(i_k)$ such that

$$u_k = \arg \min_{u \in U(i_k)} \sum_{j \in \bar{S}} \underline{p}_{i_k j}(u) \left(g(i_k, u, j) + \tilde{J}(j, r_k) \right),$$

we generate the next state i_{k+1} by simulating a transition according to the transition probabilities $\underline{p}_{i_k j}(u_k)$, and finally we perform the TD(λ) update

$$r_{k+1} = r_k + \gamma_k d_k \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_m),$$

where d_k is the temporal difference

$$d_k = g(i_k, u_k, i_{k+1}) + \tilde{J}(i_{k+1}, r_k) - \tilde{J}(i_k, r_k).$$

If $i_k \in \bar{S}$, we use the corresponding formulas where u_k is replaced by

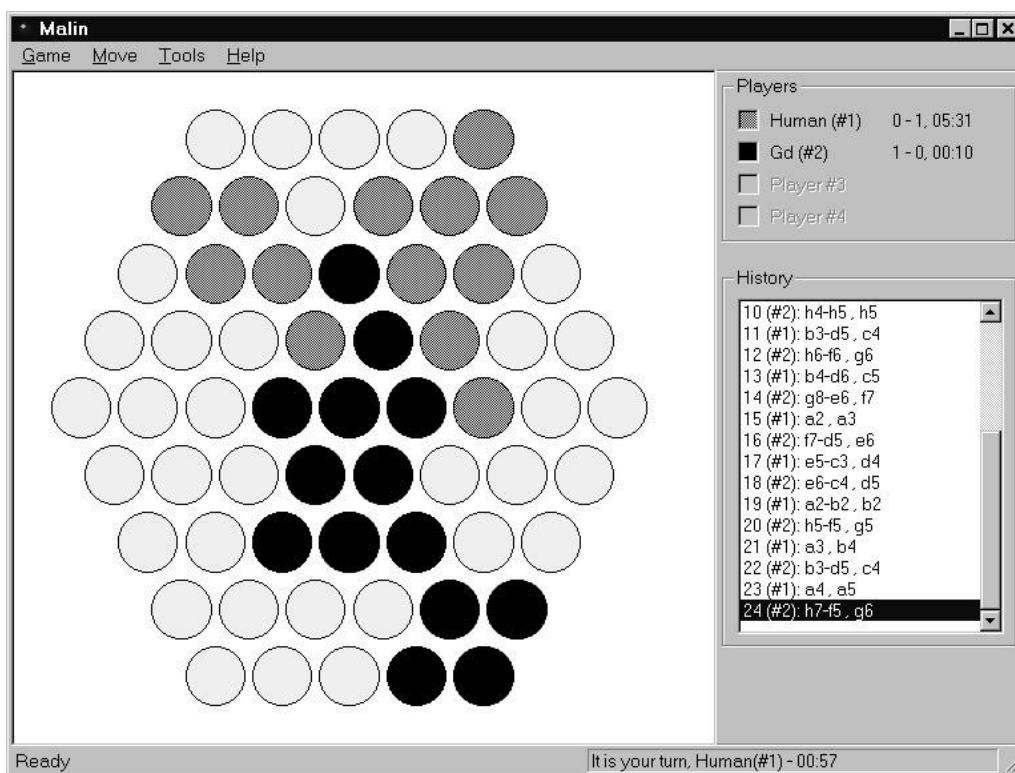
$$v_k = \arg \max_{v \in V(i_k)} \sum_{j \in \underline{S}} \bar{p}_{i_k j}(v) \left(g(i_k, v, j) + \tilde{J}(j, r_k) \right),$$

and the next state i_{k+1} is generated by simulating a transition according to the transition probabilities $\bar{p}_{i_k j}(v_k)$. Similar to the one-player case, no guarantees of success can be offered for this type of method.

Chapter 3

Malin – An Abalone Playing Program

Figure 3.1: MALIN in action



The player Human has already lost one marble against the (computer) player Gd. According to the history pane, 12 moves (24 plies) have been played. MALIN is waiting since 57 seconds for Human to play.

MALIN is an ABALONE playing computer program running on Microsoft Windows platforms. It is primarily designed as a testbed for different game-playing algorithms (including neural networks) and *not* as a commercial program. Nevertheless the program features a graphical user interface which makes it fairly easy to use.

By the way, the name MALIN is derived from the French adjective *malin*, which could be translated to English by *clever*, *bright*, *astute* and to German by *schlau*, or *gewitzt*.

3.1 First steps

Have a look at the Screen-snapshot 3.1. The largest part of the MALIN window is dedicated to the game board. From the **Players** pane on the upper right side we know that the gray marbles belong to player Human and that he has already lost 1 marble (although he is a very slow player!). The history pane below lists the 24 plies (making 12 complete moves) which have already been played. Finally pay attention to the status bar in the bottom line – for the moment MALIN is idle and waits for you to make a move.

To make a move, mouse-click on the desired marbles (up to three) – this will highlight them. Then try to move them, just as you would do with any other window: Mouse-click one of the highlighted marbles, tear it (mouse button still down) in the desired direction and release finally the mouse button. Did not work out? You can cheat by using the menu item **Move – Take Back** to undo your move ...

If you want to sleep over the next move, use the menu items **Game – Save** and **Game – Open** respectively. By default MALIN proposes you to challenge a computer opponent, but you can use **Game – New** to setup a new game with two, three or four players, each of them being human or one of dozens of different computer players. By the way, MALIN is a fully multithreaded application (*multi-tasking*), so there is no problem in interrupting MALIN while it is thinking about the next move.

3.2 Installation

MALIN comes with a standard installation utility: execute **Setup.exe** and let you

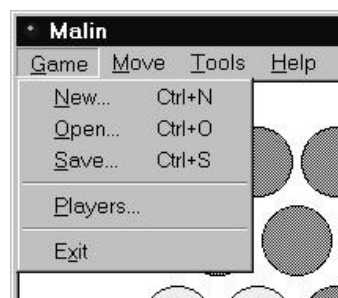
guide by the installation wizard. Basically you only have to determine the directory where the program files will be copied to. To remove MALIN from your system use the Microsoft Windows 98 remove utility: **Start – Settings – Control Panel – Add/Remove Programs**. This ensures that the tiny little entry in your system registration is also deleted.

3.3 Start a New Game

Now let's start to play the Game! By default (after starting up MALIN) you are proposed a game against a computer opponent, so you are ready to go! However, sometimes the default computer player will be too strong, or you might just want to see two computer players fighting against each other, ... By the way, MALIN does not know *one* computer player whose strength is adjustable, but knows *many* different computer players, who are by convention organized into families. The first generation got the generic name **Aba**, the latest generation of MALIN computer players shares the root **Gunilla**.

To setup a new game select **Game – New** (Snapshot 3.2). The dialogbox **New Game**

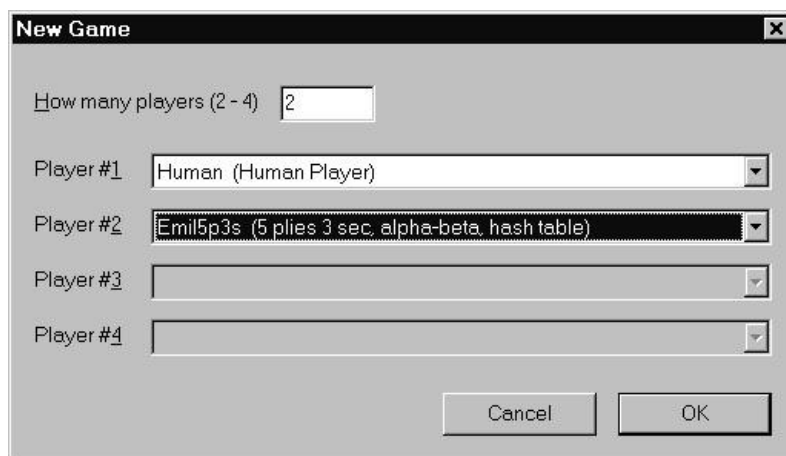
Figure 3.2: The Game menu



(Snapshot 3.3) pops up. You have to decide on the number of players (between two and four). Below you have to choose the players from drop-down-lists. Each entry in the drop-down-list contains a player along with a short description in parentheses, usually characterizing the strength of the player. For example

Emil (5 plies 3 sec alpha-beta hashtable)

Figure 3.3: The New Game dialog box



tells you that the player **Emil** thinks at least 5 plies (half-moves) in advance, and if the time budget of 3 seconds is not already scooped out, even farther. The algorithm used is alpha-beta pruning speeded up by a hashtable.

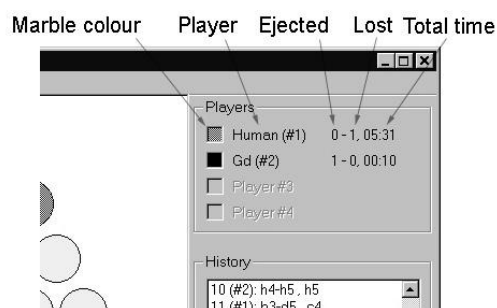
If you want to change players during game, you can use **Game – Players**. A dialog box very similar to 3.3 pops up and allows you to change any player. You might for example want to look which move a expert computer player would select in a tricky situation.

3.4 Playing a Game

Once you started to play a game, you have lots of status information displayed in the main window. The **Players** pane in the upper right for example (see Snapshot 3.4) holds the name of the players and their score. The “checkbox” contains the marble-color of the player, followed by his name. The number of enemy marbles kicked from the board (**Ejected**) is matched against the number of lost marbles. In two player games it would not be necessary to display the number of lost marbles (they equal the opponents number of ejected marbles), but in three or four player games it can be useful. Finally the total thinking time is displayed.

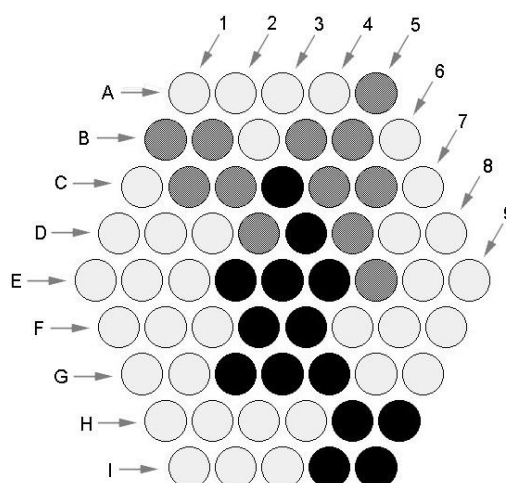
The **History** pane below records the game, ply by ply. Each entry holds one move: when it was played, whose turn it was and which marbles where moved. The board

Figure 3.4: The Players pane



notation used is somewhat similar to chess, but not that easy to read because of the hexagonal form of the ABALONE board. See Figure 3.5 for an explanation.

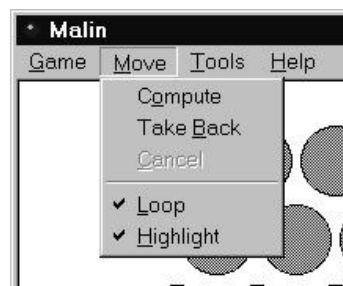
Figure 3.5: MALIN board notation



The empty place in the upper left corner would be notated **A1**, the two marbles in the lower right corner **I8-I9**.

Now let's have a look at the **Move** menu in Snapshot 3.6. One menu entry – **Cancel** – is not available for the moment; this is because MALIN is idle, waiting for the user to move. But imagine MALIN busy thinking about the next move for a computer player; if you want interrupt it to save the game, then you will **Cancel** the game. To tell MALIN to take up thinking, select **Compute**; this will obviously only work if it is a computer player's turn. **Take Back** allows you to cheat – you can effectively take back

Figure 3.6: The Move menu



moves until the very beginning of the game!

Finally you will find two checkboxes at the bottom of the **Move** menu. They both affect the behaviour of computer players. If **Loop** is turned on (default), MALIN will always make computer players play when it is their turn, if it is turned off, you have to make MALIN play manually by selecting **Compute**. When **Highlight** is on (default), the computer players highlight their selected marbles (for one second) before moving them. This can be too time consuming when two computer opponents face each other.

3.5 Tools

The **Tools** menu is mainly designed for developing purposes and will therefore not be discussed in detail. It contains a submenu which handles the creation and modification of computer players, but these functions require detailed knowledge of the internal MALIN design. The **Tournament** tool allows to select several computer players for a tournament competition. MALIN then plays everybody against everybody although it is not displaying the game; the results are written to **Malin.log**. Finally the **Train Neural Net** entry takes a MALIN script file as argument, which is used to specify the training program for a neural computer player. However, this is beyond the scope of this User Guide.

3.6 How to ... ?

How do I know if a player is a computer or human player?

The only *human* player is called Human, all other players are computer players. However, the status bar always tells you whether MALIN expects you to move or does the job itself.

Why do many computer players not accept three or four player games?

Because playing against more than one player requires different programming techniques. All really good MALIN computer players are exclusively optimized for two player challenges, but you can always try the Aba or Boub computer player families.

What are these strange codes in the history pane?

Each entry in the History list box, corresponds to one move and records when it was played, whose turn it was and which marbles were selected. See Figure 3.5 to understand the move notation.

How do I get advice of an expert computer player?

There is no menu entry like “Give advice”, but you can always use the Game – Players utility to change your (Human) player into a computer expert, let him make one move and change back to the Human player.

How about having several instances of MALIN running at the same time?

No problem, your CPU will be occupied, that is all.

Does anything bad happen if I close the MALIN window while it is busy?

No, MALIN cancels thinking and shuts itself down correctly.

What is left on my system, after I have run the remove utility?

The program files and the entry in the system registration are deleted, only the four dynamic libraries stay in your Windows system directory.

3.7 Files

Program Files These files are located in the directory chosen during setup.

<code>Malin.exe</code>	The MALIN executable
<code>*.ar</code>	Initialization files
<code>Malin.log</code>	The log file, used for example by the tournament tool

Shared Dynamic Libraries The dynamic libraries are as always stored in the Windows system directory. They are not deleted by the remove utility.

<code>Msvcrt.dll</code>	C++ runtime library
<code>Msvcirt.dll</code>	C++ runtime library
<code>Msvcp60.dll</code>	Microsoft Visual C++ 6.0
<code>Mfc42.dll</code>	Microsoft Foundation Classes 4.2

Players The player files have the extension `ap` and are stored in the subdirectory `Player`.

Games The game files have the extension `ag` and are stored in the subdirectory `Game`.

Chapter 4

Game-Searching Algorithms

This chapter covers the development of standard ABALONE-playing algorithms. Each algorithm consists of three main components, the *position evaluator*, the *move generator*, and a *game-searching algorithm*. While the position evaluator and the move generator remain unchanged for the majority of the presented algorithms, is the search algorithm refined step by step. Detailed experimentation results can be found in Section 4.3.7. The results of this chapter form the foundation for the development of reinforcement players in Chapter 5.

4.1 Game-Playing Algorithms

This section treats the general structure of game-playing programs. It applies to standard algorithms developed in this chapter as well as to the reinforcement players of Chapter 5.

4.1.1 Position Evaluator

The position evaluator is a function that assigns a value to a board position, possibly between -1 and $+1$, where -1 would signify a loss of White, 0 a equilibrium position and $+1$ that White wins. Basically, two different techniques are employed in this work. The first relies on *hand-crafted feature extracting*, the second on the *approximation capabilities of a neural network*.

Feature extracting has often been called an art (instead of a science); effectively it is a very heuristic task to filter out the significant *features* of a position and to assign them values according to their importance. A feature typically used to reward the grouping of marbles, adds (subtracts) one point to the position value for every white (black) marble that is surrounded by at least three marbles of its own color. The number of handcrafted features can vary from a very small number (three) up to virtually infinite, limited only by imagination and computing speed. Once values have been assigned for each feature, they are (linearly) combined and eventually normalized to build the position's value.

The most important features are the number of enemy marbles already kicked from the board (as well as the number of lost marbles), the position of the marbles, particularly their distance from the center of the hexagonal ABALONE board, and last not least the compactness of the marble formation (measured by the number of marbles placed beside marbles of the own color).

The approach taken when using neural networks is fundamentally different. Although hand-crafted features are still used to speed up the task of learning, no values are anymore attributed to features manually. Instead the network has to learn on its own which reward policy leads to success. After several thousands of training games, a well prepared neural network is effectively able to outperform hand-crafted feature extractors by far.

4.1.2 Move Generator

Even with very sophisticated position evaluators it will still not be enough to restrict one-self to the static evaluation of a position. Every human ABALONE or Chess player knows that it is often (or perhaps always) necessary to “look into the future”, to try to anticipate the moves of one's opponent, how oneself will react, how he will then respond, ... Suppose we try to figure out *all* of our possible moves, then *all* responses to *all* our moves, then ... – obviously the number of move sequences will increase exponentially with the *search depth*. The average number of possible moves in a typical ABALONE position is $M \approx 58$, consequently we will not be able to think many plies (half-moves) ahead, before being hopelessly lost. Instead even a novice ABALONE or Chess player considers only a few moves at every ply to inspect more thoroughly; by

this means good ABALONE players can think up to eight plies (four complete moves) ahead.

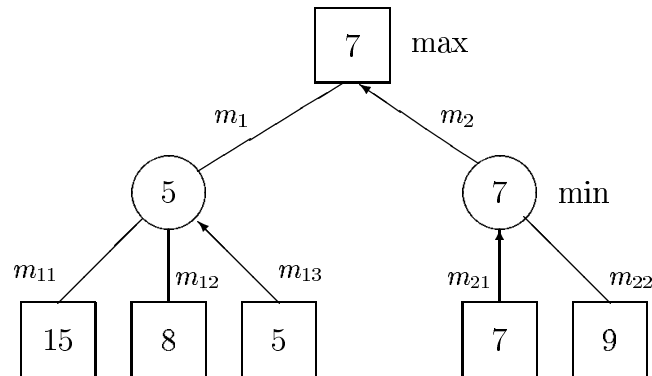
The *move generator* decides at a given stage in the lookahead search, which moves are worth being considered more in detail. For example, *brute force* algorithms always select *all* moves for further inspection, until a given search horizon (for example 5 plies) is reached. On the other hand *heuristic selection* algorithms, try to do it the same way the human mind does, and do only continue to search a fraction of all possible moves, the fraction being between 5% and 75%. The problem is apparently to know *which* moves are worth further consideration, and which can be ignored. This is far from being a trivial problem, on the contrary it is so complex that finally brute force approaches, intellectually perhaps less appealing, made the race in almost all good game-playing programs available today. The first MALIN players are also based on heuristic move selection, but as they are clearly inferior to the succeeding brute force players they are not treated further in this work.

4.1.3 Game-Searching Algorithms

Once we know to evaluate a position and how to select the moves worth deeper examination, there remains still the question of the best way of searching all the resulting move sequences for the best outcome. The idea underlying (almost) all search algorithms is the *minimax* principle. It states that we always have to select the best move when it is our turn, but always the *worst* when it is our *opponents* turn.

The move sequence derived by this principle will lead to the best possible outcome we can *enforce*. Perhaps an even better outcome would have been *possible*, but in that case we would have had to count on our opponent to commit an error. Take a look at the *game tree* in Figure 4.1 to understand the difference. The player *Maximizer* (rectangles) faces the *Minimizer* (circles). The numbers in the rectangles and circles are the payoff for the Maximizer – the higher the better. Because ABALONE is a strictly competitive game, the minimizers effort to maximize his own payoff is equivalent to minimizing his opponents payoff. The *payoffs* (numbers) in the lowest row are evaluated by the position evaluator as discussed above. The highest possible value is 15, but if the Maximizer plays m_1 , the Minimizer will answer with m_{13} , leading to an outcome of 5! Accordingly, on the right half of the game tree, the Minimizer can

Figure 4.1: The minimax principle



The rectangles are positions where it is the *Maximizers* turn, the circles correspond to the *Minimizers* positions.

The numbers in the rectangles and squares are the utility of the Maximizer, or equivalently the loss of the Minimizer.

m_{xy} uniquely identifies a move.

enforce an outcome of 7, thus leaving the Maximizer the choice between 5 or 7, which finally results in the minimax value 7. This is the minimum value that the Maximizer can *enforce*.

By a little modification of this minimax algorithm its complexity can be divided by two; the resulting enhancement is known as the α - β algorithm and is extensively used in the MALIN players. See Section 1.2 for theoretical analysis of α - β and Section 4.3.3 for a detailed explanation.

4.1.4 Deadlock Situation

In some games like Backgammon, it is almost impossible to end up in a *deadlock* situation, that is, when the best action for each player is to repeat a move already played once, and thus to get lost in an endless loop. However, in ABALONE this happens fairly often, particularly in the opening phase of the game, after 5–10 moves have been played. Humans can easily avoid this situation, once we realize the deadlock, we simply play the second best move to get out of the trap. The problem is critical when two computer players compete against each other. The solution adapted for all MALIN players is the recording of all moves played during the game. Before the move

generator returns the list of possible moves, it excludes the deadlock moves and hence avoids an endless loop.

4.2 Position Evaluation by Feature Extracting

A central issue in game-playing is the employed position evaluator, already introduced in Section 4.1. All MALIN computer players except the very first and the very last (based on neural networks, Chapter 5) use the same position evaluator – which is the subject of this section.

The design of the position evaluator is kept very simple, in order to attain high computation speed. Only four features are employed, the number of kicked (and lost) marbles feature (resulting in the partial sum V^k), the distance-to-center feature V^d , the neighbour feature V^n , and the game-over feature V^o .

Kicked and lost marbles

$$V^k = 1000 \cdot \begin{cases} 2k_1 - k_2 & \text{if } k_1 > k_2, \\ 2k_2 - k_1 & \text{if } k_1 < k_2, \\ 0 & \text{else,} \end{cases}$$

where k_i , $i = 1, 2$ is the number of marbles kicked by player i . This nonlinear function “prefers” high scores, i.e., 5:3 gets a higher rating than 4:2. The result is weighted by the factor 1000.

Distance-to-center If a marble occupies the (only) central space of the hexagonal ABALONE board (E5 following the notation of Figure 3.5), its player gets 12 units, for a marble in one of the six corners (A1, A5, E9, I9, I5, E1) 5 units are deduced. The values of the spaces in between are (heuristically) interpolated.

$$V^d = \sum_s d_c(s) o(s),$$

where s is a space on the ABALONE board, $d(\cdot) : \text{board} \rightarrow [-5, 12]$ is the distance-to-center evaluation function, and $o(\cdot)$ is the occupation function,

$$o(s) = \begin{cases} +1 & \text{if a white marble (player I) is occupying the space } s, \\ 0 & \text{if the space } s \text{ is free,} \\ -1 & \text{if } s \text{ is occupied by a black marble.} \end{cases}$$

Neighbours For every pair of neighbouring marbles of the same color, 2 units are awarded.

$$V^n = 2 \cdot \frac{1}{2} \sum_s o(s) \sum_{\bar{s}} \delta_{o(s)o(\bar{s})},$$

where $\bar{s}$ is a space neighbouring s , and $\delta_{xy} = 1$ if $x = y$ and 0 otherwise.

Game over If one player has kicked six marbles the game is over and the value is set to infinity,

$$V^o = \begin{cases} +\infty & \text{if player I has won the game,} \\ 0 & \text{if the game is still going on,} \\ -\infty & \text{if player II won the game.} \end{cases}$$

The position evaluates then to

$$V = V^k + V^d + V^n + V^o.$$

Of course the constants mentioned above (i.e. 1000 units for a kicked marble) are easily modifiable by the user (MALIN menu item **Tools – Player – Modify**).

Several other features are possible, but did not work out very well.

Intruder If a marble is surrounded by at least five enemy marbles, bonus points are awarded. The idea is to disturb the construction of compact marble formations by introducing an alien marble in the middle.

The problem is, that this “intruding” *can* be beneficial but does not have to. Suppose the enemy is dominating the center of the ABALONE board, then it is certainly a good idea to place a marble inside his formation, because this may dramatically reduce his opportunities to push your marbles. But when we find one of our marbles near the board border surrounded by enemy marbles, we are almost sure to loose it.

Loneliness If a marble is all alone, touching no neighbours, value is decreased because a marble is much more valuable when it is integrated in a compact formation.

This feature is often mentioned by human players, when asked how they evaluate an ABALONE position and is unambiguously an useful feature. However, it unnecessarily slows down the position evaluation because the neighbour feature already takes account of lonely marbles – by simply not awarding any neighbour bonus points.

With enemy at border If the marble is placed right beside the border and an enemy marble is neighbouring, value is decreased.

This feature is too general to be really useful and was therefore abandoned.

Push enemy The possibility of pushing enemy marbles in the next move increases a position's value, because it tends to irritate the opponent.

Having the opportunity to push the opponents marbles, may sometimes be a good intermediate goal, but it sometimes produces unwanted results and was therefore abandoned too.

Attack enemy If an enemy marble could be kicked off the board during the next move, bonus points are awarded.

This is certainly a very useful feature, perhaps one of the most important. The reason why it is no longer used, is a modification of the search algorithm, called *quiescence search*, see Section 4.3.2. Basically it means, that unstable positions are never evaluated directly but always expanded until a stable position is reached. The position evaluator is therefore never run on positions where marbles could be kicked during the next move (this is the definition of an unstable position).

4.2.1 Three-Player Games

In a two-player challenge the position evaluator returns one value V , which is by definition the utility of the first player, and because ABALONE is strictly competitive, the loss of the second player. In a three-player challenge, it will be necessary to remember more information than a single value to characterize a position. Note also, that an game-playing algorithm designed for *three, four, or more* player challenges cannot rely on the minimax principle, which is exclusively applicable to two-player games.

Consider the two-player case; a position's value is composed of two partial sums $V = v_1 - v_2$, where v_1 is attributed to the marbles of the first player, and v_2 to the second. For example, v_1 is increased for every marble *player I* has kicked from the board, but v_2 gets points for every marble *of player II* near the center of the board. According to the minimax principle (see Figure 4.1) player I will always try to maximize V , while

player two does the contrary by trying to minimize V . This is equivalent to player I maximizing V_1 and player II maximizing V_2 , where

$$\begin{aligned} V_1 &= +v_1 - v_2, \\ V_2 &= -v_1 + v_2. \end{aligned}$$

This way all players maximize “their” utility, which simplifies the algorithm; on the other hand do we need to store the two values v_1 and v_2 instead of only V .

The generalization to the p -player case is straightforward. Instead of generating a single value V for a board position, p values $v_1, v_2, \dots, v_p$ are calculated, where v_i consists of the points attributed to the marbles of player i . The utility V_i of player i can then be defined by

$$V_i = \frac{1}{p-1} (-v_1 - \dots - v_{i-1} + (p-1) \cdot v_i - v_{i+1} - \dots - v_p).$$

Whenever its player i ’s turn, he will try to maximize “his” utility V_i . Note that this definition is completely heuristic; for example, the utilities V_1, V_2, V_3 , in a three-player challenge can as well be defined by

$$\begin{aligned} V_1 &= v_1 - 0.5v_2 - 0.3v_3 \\ V_2 &= -0.3v_1 + v_2 - 0.5v_3 \\ V_3 &= -0.5v_1 - 0.3v_2 + v_3. \end{aligned}$$

4.3 Search Algorithms

4.3.1 Brute Force Negamax Search

This section introduces the core algorithm, upon which *all* following game-searching algorithms are based. Algorithm 4.1 is equivalent to the minimax principle, but is coded slightly different. While the minimax algorithm distinguishes between player I and II, alternatively maxi- and minimizing, does this version not need this distinction. The position values are always maximized but instead they are multiplied with -1 after every lookahead-stage, giving the algorithm its name – *negamax algorithm*.

The term *brute force* refers to the method of move generation. While a heuristic move selection algorithm only expands a (small) fraction of all possible moves, do brute force algorithms simply inspect *all* moves.

Algorithm 4.1 (Brute Force Negamax Search)

```

int Evaluate (Position position, int depth)
{
    Input:
        position ... Object holding the marble positions on the Abalone board,
                   the score, and whose turn it is.
        depth ... Holds the actual search depth in plies (starting with 0).
        g_depthMax ... Global variable holding the search depth limit in plies.
    Output:
        g_moveBest ... Holds the best move for the initial start position.
    Return value: The positions value after a (g_depthMax - depth)-ply
                   lookahead with respect to the player whose turn it is.

    ? already reached search depth limit (g_Xxx signifies global variable)
    if (depth >= g_depthMax)
    {
        The position is evaluated with respect to the player whose turn it is
        return PositionEvaluator (position);
    }

    Generate all possible moves
    moveVector = BruteForceMoveGenerator (position);

    int valueBest = -infinity;
    Loop through all moves
    for (move in moveVector)
    {
        Move, and store new board position
        newPosition = Move (position, move);

        Expand the move by calling Evaluate recursively
        value = -Evaluate (newPosition, depth + 1);

        if (value > valueBest)    Hold track of the best move
        {
            valueBest = value;
            moveBest = move;
        }
    }

    If lowest level, store the final result
    if (depth == 0) g_moveBest = moveBest;

    return valueBest;           Return the value of the best move
}

```

And this is the function which initializes the recursion.

Algorithm 4.2 (Brute Force Negamax Search – Initialization)

```

Move CalculateMove (Position position, int lookahead)
{
    Input:
        position ... Object holding the Abalone board.
        lookahead ... Defines how many plies deep the lookahead shall be carried out.
    Returns the best move.

    g_depthMax = lookahead;
    Evaluate (position, 0);
    return g_moveBest;
}

```

Note the simplicity of the algorithm due to its recursive character. `Evaluate` always returns the positions value V with respect to the player whose turn it is. The MALIN players contain additionally a random element, which does not necessarily choose the *first* move returning `valueBest`, but *arbitrarily* one move among the best.

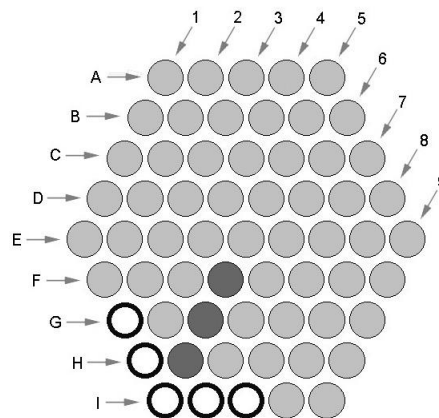
4.3.2 Quiescence Search

This section introduces an important enhancement of the minimax/negamax algorithm, called *quiescence search*. The basic idea is to avoid the evaluation of unstable positions. The characterization of an unstable position can be very difficult, depending on the game (i.e. see [Bra83] for the game of Chess). However, in ABALONE the definition of a unstable game is very straightforward.

Definition 4.1 (Unstable position) If a marble can be kicked off the board during the next move, the position is said to be *unstable*.

See Figure 4.2 for an example. If the static position evaluator tries to analyze this situation it will be unable to estimate correctly how many marbles will remain on the board; the evaluator can only take into account how many marbles *already* have been kicked. The quality of the evaluation of such a position is very low, because the positions value will change dramatically within one or a few moves; this causes very bad move selection decisions. Quiescence search will avoid the evaluation of such positions by continuing to expand all critical (“kicking”) moves as far as the position

Figure 4.2: An unstable position



It is Blacks turn.

becomes stable. This enhancement improves substantially the game-playing quality of the algorithm. Algorithm 4.3 is based upon the brute force negamax Algorithm 4.1.

Algorithm 4.3 (Quiescence Search)

```

int Evaluate (Position position, int depth)
{
    At which search stage are we ?
    if (depth >= g_depthMax)      Quiescence search
    {
        Generate only the moves which kick a marble off the board
        moveVector = KickingMoveGenerator (position);
    }
    else if (depth > 0)           Regular search
    {
        Generate all possible moves
        moveVector = BruteForceMoveGenerator (position);
    }
    else                          Lowest search level (depth == 0)
    {
        Generate all possible moves except those already played once
        in exact the same position
        moveVector = ExcludeHistoricMoveGenerator (position);
    }

    ? no more moves
    if (moveVector.size() == 0)
    {

```

```

    The position is evaluated with respect to the player whose turn it is
    return PositionEvaluator (position);
}

int valueBest = -infinity;
for (move in moveVector)      Loop through generated moves
{
    newPosition = Move (position, move);
    int value = -Evaluate (newPosition, depth + 1);
    if (value > valueBest)
    {
        valueBest = value;
        moveBest = move;
    }
}
if (depth == 0) g_moveBest = moveBest;
return valueBest;
}

```

The only difference to Algorithm 4.1 is the move generation section at the beginning of `Evaluate`, which now depends on `depth`, the recursion parameter, holding the number of plies already expanded. While the basic minimax Algorithm 4.1 simply stops further investigation when `depth` equals `g_depthMax` (the lookahead limit in plies), does the quiescence Algorithm 4.3 continue to expand all moves which kick a marble off the board. This ensures that the `PositionEvaluator` is never applied to an unstable position.

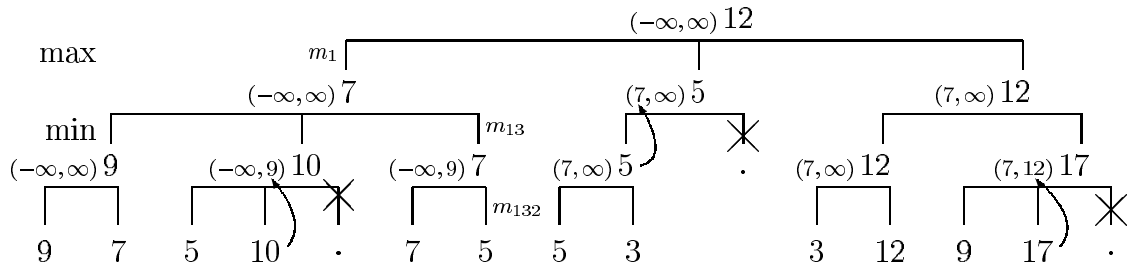
Suppose that we carry out a 3-ply lookahead to determine the value of a given start position and that we encounter the position of Figure 4.2 while it is Blacks turn and `depth` is already 3. Without quiescence search the position would simply be evaluated by the static `PositionEvaluator`. With quiescence search the move `F5-H5 → G5` will be stored in the `moveVector` and the lookahead goes on with *only one, instead of all* moves generated. This way `Evaluate` will be called with `depth = 4`; and generate the two moves `G3-H4 → H4` and `I7-I6 → I6` for player White. By continuing this way, `Evaluate` inspects move sequences with up to 7 plies – 3 plies of regular search and 4 plies of quiescence search for the position in Figure 4.2 (until only one black marble remains). Strictly speaking, is Algorithm 4.3 no brute force search anymore, because quiescence search can be interpreted as “heuristic move generation”.

The MALIN player family Cäsar relies on the quiescence Algorithm 4.3.

4.3.3 α - β Search

While quiescence search improves the overall result by selectively investigating deeper than the initial lookahead limit proposes, does the following enhancement speed up the algorithm without changing the overall value of the examined position. The game tree is still searched in the recursive manner introduced by the minimax / negamax Algorithm 4.1, but information of the already searched branches is kept in memory in order to produce cut-offs. A cut-off is a portion of the game tree which is not searched; of course these cut-offs must not alter the overall resulting value. This enhancement is known as the α - β *algorithm*, see Section 1.2 for an theoretical analysis. Figure 4.3 depicts an game tree which we will use to explain the idea.

Figure 4.3: The α - β algorithm in a minimax formulation



The tree of a 3-ply lookahead problem.

The numbers are positions values, “.” signifying that an evaluation was not necessary.

Only the positions at the “bottom” of the search tree are actually analyzed by the position evaluator, the other values are determined by maximum (minimum) selection over the descendents respectively.

The α - β couples are depicted in parentheses. Note that during an maximum (minimum) selection only the upper (lower) bound grants cut-offs.

m_x , m_{xy} , and m_{xyz} uniquely identify the branches of the search tree.

The search tree is analyzed from the left to the right, accordingly starting with the branches m_1 , m_{11} , and m_{111} where the desired lookahead depth is reached and the position evaluator is invoked for the first time, returning 9. For the moment we ignore quiescence search. The position m_{112} (more precisely the position reached after the move m_{112}) evaluates to 7. As it is the maximizers turn, m_{11} is therefore assigned the value 9. At position m_1 it is the minimizers turn and therefore m_{11} (value 9) will compete against the positions m_{12} and m_{13} in a minimum selection. We continue the

analysis of the search tree and evaluate the positions m_{121} to 5 and m_{122} to 10; by now we know that m_{12} is *at least* 10 – which guarantees that m_{11} is better than m_{12} !! It is therefore not necessary to evaluate m_{123} because it will not change the value of m_1 – the portion of the search tree beginning with m_{123} can therefore be *cut-off*.

By continuing this scheme we evaluate the position m_1 to 7, which is therefore a lower bound for the value V of the initial start position. Therefore we can cut-off all branches below m_{22} : after evaluation of m_{21} we know that m_2 values *at most* 5. Note the intervals depicted beside the positions values, the lower bound is called α value, the upper bound β value. The cut-off explained above is called an α -cut-off, because 5 was below the α value 7.

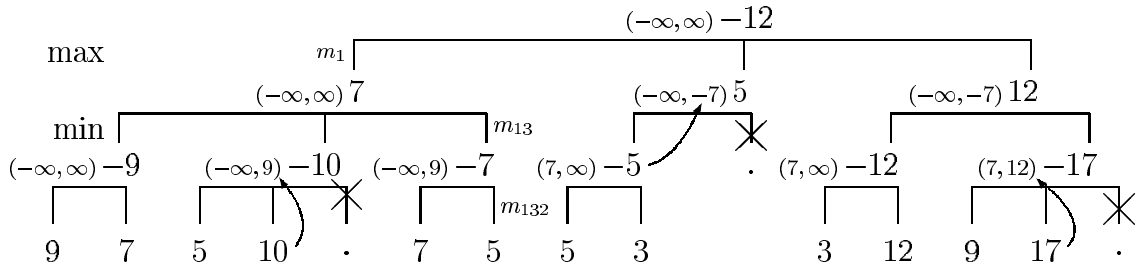
Now, why does m_{311} not induce a cut-off? After all it is outside the $(\alpha, \beta) = (7, \infty)$ interval? First have a look at m_{32} , we already know that when its value gets above 12, it is of no further use. Therefore m_{322} evaluating to 17 generates a β -cut-off. Both m_{322} and m_{311} compete in a *maximum* selection, but only m_{322} is *above the β value*. On the other hand does it not matter whether m_{311} is below the α value. In the case of m_{21} it is inverse, it is the *minimizers* turn and we only care if a value is *below the α value*. The reason why we always keep the α and the β value in memory, is that at position m_{3211} of a (not depicted) 4-ply lookahead we could once again make use of the α value, and so on ...

Negamax Formulation

While the presented minimax formulation of the α - β algorithm already tends to tie up (recursively) one's brain, is the negamax formulation even worse. Figure 4.4 presents once more the search tree known from Figure 4.3, but this time we always maximize, therefore only inducing β -cut-offs.

The positions values are not anymore depicted with respect to a *constant* player but depending on the player whose turn it is (to be more precise, with respect to the “actual” players opponent). To see the equivalence with Figure 4.3 multiply all values in the odd rows by -1 and mirror the (α, β) interval in the even rows at the origin. Note also that the (α, β) intervals placed beside the value of a position m_{xy} belong to the positions $m_{xyz}, \forall z$.

However, even if things seem to get complicated, the important thing is to see the

Figure 4.4: The α - β algorithm in a negamax formulation

The numbers are position values with respect to the *opponent* player,

“.” signifying that an evaluation was not necessary.

Only the positions at the “bottom” of the search tree are actually analyzed by the position evaluator.

The α - β couples are depicted in parentheses beside the *parents* of the positions they belong to. Only β -cut-offs are possible.

m_x , m_{xy} , and m_{xyz} uniquely identify the branches of the search tree.

equivalence with the minimax formulation (Figure 4.3) on one hand, and with Algorithm 4.4 on the other hand.

Algorithm 4.4 (α - β Search)

```

int Evaluate (Position position, int depth, int alpha, int beta)
{
    Input:
        alpha ... The lower bound, never used for cut-offs.
        beta ... The upper bound.

    if (depth >= g_depthMax) moveVector = KickingMoveGenerator (position);
    else if (depth > 0) moveVector = BruteForceMoveGenerator (position);
    else moveVector = ExcludeHistoricMoveGenerator (position);

    if (moveVector.size() == 0) return PositionEvaluator (position);

    int valueBest = -infinity;
    for (move in moveVector)      Loop through all moves
    {
        newPosition = Move (position, move);

        Call Evaluate with appropriate alpha beta values
        int value = -Evaluate (newPosition, depth + 1,
                               -beta, -Max(alpha, valueBest));
    }
}

```

```

    if (value > valueBest)      Hold track of the best move
    {
        valueBest = value;
        moveBest = move;

        If Beta cut-off stop loop
        if (value >= beta) break;
    }
}
if (depth == 0) g_moveBest = moveBest;
return valueBest;
}

```

The algorithm is derived from the quiescence Algorithm 4.3 and is initialized by `Evaluate (position, 0,  $-\infty$ ,  $\infty$ )`. Only three changes have been put into effect; the parameter definition of `Evaluate`, the recursive call of `Evaluate` and the test for a β -cut-off.

The *Charles* family is based on exactly this α - β algorithm.

4.3.4 Killer Moves

The α - β algorithm reduces the number of position evaluations substantially. Without α - β we expect M^n positions to be evaluated in a n -ply lookahead, where M is the average number of possible moves. The α - β Algorithm 4.3 *can* reduce the number of evaluations in the best case to

$$M^{\lfloor \frac{n}{2} \rfloor} + M^{\lceil \frac{n}{2} \rceil} - 1,$$

but in the worst case it remains M^n , see Section 1.2. What does this depend upon? Take once more a look at Figure 4.3; if the move m_3 had been analyzed *before* the move m_1 a larger part of the search tree could have been cut-off (i.e. the moves m_{12} and m_{13}). The order in which the moves are analyzed is therefore important. In the best case, the move which yields the best value is always selected first, consequently inducing a maximum of cut-offs. Obviously this is not possible (why would we carry out a search otherwise?), but a good idea might be to introduce *heuristic move ordering*.

Elmenreich [Elm98] proposes to evaluate intermediate positions before further analyzing them. For example, the moves m_1 , m_2 , and m_3 in Figure 4.3 are statically evaluated before deciding which one is expanded first. Tables are kept in memory to

make use of the similarity of many moves. However, this requires additional calls to the position evaluator and has not been pursued in MALIN.

Instead heuristic move ordering is accomplished by the use of *killer move* tables. A killer move is a very good move, so good he “kills” a great portion of the search tree by inducing a lot of cut-offs. The idea is that in a typical ABALONE position there are only a few really good moves and that furthermore in positions which are not very different, the good moves remain the same. This is because of several reasons (see [Elm98]),

- in ABALONE marbles are only moved by exactly one space;
- in one move no more than 5 marbles can be displaced, usually only 2 or 3 marbles are moved;
- the evaluation of a board position depends primarily on *where* the marbles lie and not on the patterns they form.

The killer algorithm remembers the move which is currently considered best at each level of the search tree and tries this move first at each new position. Actually, MALIN does not only keep the best move at each level in memory, but uses the best *six* moves at each level (see Figure 4.10). Suppose the branch m_{13} in Figure 4.3 corresponds to the move $G6-E6 \rightarrow F6$, then we will consider it a killer move, because it was the best move at position m_1 . It will be therefore be stored in the killer move table for level 1. We assume further that $G6-E6 \rightarrow F6$ is also a valid move from position m_3 , and that it corresponds to the branch m_{32} . Then the killer move Algorithm 4.5 will start expanding m_{32} before the other branches, hoping to increase the number of cut-offs (unfortunately this is not the case in this example).

Algorithm 4.5 (Killer Moves)

```
int Evaluate (Position position, int depth, int alpha, int beta)
{
    Input/Output: g_killer ... Object holding the killer move tables.

    if (depth >= g_depthMax) moveVector = KickingMoveGenerator (position);
    else if (depth > 0) moveVector = BruteForceMoveGenerator (position);
    else moveVector = ExcludeHistoricMoveGenerator (position);

    if (moveVector.size() == 0) return PositionEvaluator (position);
```

```

Sort the moves according to the killer table for level 'depth'
g_killer.sort(depth, moveVector);

int valueBest = -infinity;
for (move in moveVector)      Loop through all moves
{
    newPosition = Move (position, move);
    int value = -Evaluate (newPosition, depth + 1,
        -beta, -Max(alpha, valueBest));
    if (value > valueBest)
    {
        valueBest = value;
        moveBest = move;
        if (value >= beta) break;
    }
}

Add move to the killer move table for level 'depth'
g_killer.add(depth, moveBest);

if (depth == 0) g_moveBest = moveBest;
return valueBest;
}

```

The algorithm is derived from the α - β Algorithm 4.4; two lines make up for the difference, `g_killer.sort(depth, moveVector)` which orders the `moveVector` according to the killer move table, and `g_killer.add(depth, moveBest)` declaring that `moveBest` is a good move. Note that all calls to the killer object are parametrized with `depth`, in order to determine which killer move table to use (there is one for every lookahead-stage). For each move in one of the killer tables, a counter is kept which determines its “killing quality”; a call to `add(depth,move)` increases the counter, which rapidly decreases in the course of time. `sort(depth, moveVector)` effectively performs only a partial sort, stopping after the six best moves have been found and shifted to the front of the move vector.

4.3.5 Iterative Search

From Algorithm 4.5 it follows that an important portion of the search tree has to be searched before the killer move tables are able to provide good move ordering.

The idea of *iterative searching* is to conduct the search in stages, starting with shallow searches and gradually increasing the depth of search until the desired lookahead depth is reached or time runs short. Hence, the algorithm starts with a 1-ply search, building up the killer move table for level 1, then continues with a 2-ply search, ... until a full n -ply search has been carried out. At a first glance this seems to increase the number of position evaluations, but Frey [Fre83] has illustrated that the better move ordering counterbalances the repeated search stages. Even when evaluation are counted for all iterations combined, the total is still less than that observed with a single search equal in depth to that of the last iteration. Note, that iterative search without a move ordering algorithm (like killer moves) would not make any sense at all.

The recursive algorithm developed in the last subsection stays the same, only the initialization is changed.

Algorithm 4.6 (Iterative Search – Initialization)

```

Move CalculateMove (Position position, int lookahead)
{
    Empty the killer move tables
    g_killer.init();

    Start with a 1-ply search, ... , up to a full 'lookahead'-ply search
    for (int i = 1; i <= lookahead; i++)
    {
        Set the search depth
        g_depthMax = i;
        Evaluate (position, 0, -infinity, infinity);
    }
    return g_moveBest;
}

```

This algorithm could be refined by heuristically adapting the (α, β) window from $(-\infty, \infty)$ to some smaller interval ([Fre83] refers to this technique as non-speculative forward pruning). This will induce more cut-offs, but in the worst case the interval was too small and the calculations have to be restarted. This approach has not been pursued any further. The David family is based on killer moves with iterative search.

4.3.6 Hash Tables

Even if the number of examined positions can be substantially reduced by using an *iterative killer move α - β algorithm* (Algorithms 4.5, 4.6) the position evaluator still remains a bottle-neck to the overall performance. Moreover, when performing a n -ply search where $n \geq 3$, it is very probable that positions are evaluated *several times* at different locations in the search tree. This can happen when the move sequence $m^1, m^2, m^3, m^4, \dots$, played alternately by player I and II, is permuted to $m^3, m^2, m^1, m^4, \dots$ (which is not always possible). Hence, if enough memory is available, overall speed could be increased by storing the values of often needed positions. This is of course not limited to the results of static position evaluations – remembering evaluations based upon a n -ply lookahead will be even more precious.

The memory organisation required, is in terms of computer science, a *sparse array*, because the number of possible ABALONE board positions is very large. As a fast random access for a number of a priori unknown entries is needed, a linked list or a pointer array are not feasible. The compromise in size and speed is a *hash table*. The utilization of hash tables for this purpose is the subject of [Elm98].

The basic idea is to use an ordinary table of fixed size H and to assign uniformly each board position a *hash index* h_i ranging from $[0, H - 1]$. A position is then stored at offset h_i , in the worst case overwriting another position with the same h_i . The size H of the hash table and the *uniform* distribution of h_i over all possible positions are crucial to the performance.

For each evaluated position, the following data is stored in the table at offset h_i .

- The positions value V .
- The lookahead depth n upon which the calculation of V is based.
- The α - β quality q of the evaluation (indicates whether V is exact or only approximated due to α or β -cut-offs).
- The positions *hash value* h_v (the hash index h_i and the hash value h_v together (almost) uniquely identify the position).

In order to ensure that the hash index h_i is uniformly distributed, we choose H to be

a prime number (i.e. 4423, 9941, 11213, 21701) and calculate

$$h_i = \left(\sum_{s=0}^{60} 3^s \cdot \bar{o}(s) \right) \bmod H,$$

where s enumerates the 61 spaces on the ABALONE board, and $\bar{o}(\cdot)$ is an occupation function,

$$\bar{o}(s) = \begin{cases} 0 & \text{if the space } s \text{ is free,} \\ 1 & \text{if a white marble (player I) is occupying the space } s, \\ 2 & \text{if } s \text{ is occupied by a black marble.} \end{cases}$$

For the calculation of the hash value h_v a large prime number P has to be chosen arbitrarily (i.e. $P = 72117691$),

$$h_i = \left(\sum_{s=0}^{60} 5^s \cdot \bar{o}(s) \right) \bmod P.$$

The hash value h_v of an evaluated position is stored at offset h_i in the hash table. To avoid a collision with another position having the same index h_i , several hash tables of size H are kept in parallel; if the entry in the first table is already occupied by a position with different hash value h_v , the second table is used, ... if all hash tables are already full, one entry is overwritten. The entry to overwrite is selected with a heuristic algorithm, which favors entries with a small lookahead depth n . MALIN uses typically 4 hash tables in parallel. Theoretically it is possible that two positions share h_v and h_i , but the probability is so small ($\approx 10^{-9}$) that it can be neglected.

When the value of a position with hash index h_i is to be looked up, the hash tables are searched for an entry at offset h_i with correct hash value h_v^s (the s signifies *stored*). The stored value V^s is only accepted if the lookahead depth n^s it is based on, is greater than the requested lookahead depth n , $n^s \geq n$.

If the result of a n -ply lookahead search returns a value in the interval (α, β) it can safely be reused at any other node in the search tree, because this is the *exact* minimax value of the position. If on the other hand a positions value V is outside the (α, β) bounds, it is only known that $V \leq \alpha$, or $V \geq \beta$ respectively. However, this *can* under certain circumstances provide useful information, and is therefore also stored in the

hash table. The α - β quality q can accordingly take three values,

$$q = \begin{cases} \text{lesser} & \text{if } V \leq \alpha, \\ \text{exact} & \text{if } V \in (\alpha, \beta), \\ \text{greater} & \text{if } V \geq \beta. \end{cases}$$

When retrieving a value V^s from the hash table and the quality $q^s \neq \text{exact}$, it depends on the actual (α, β) interval, if V^s is accepted. The hash table algorithm is derived from Algorithms 4.5 and 4.6.

Algorithm 4.7 (Hash Table)

```

int Evaluate (Position position, int depth, int alpha, int beta)
{
    Input/Output: g_hash ... Object holding the hash tables.

    Search hash table for position (calculated at least with a
    (g_depthMax - depth)-ply lookahead) and retrieve its value and quality.
    if (g_hash.find (position, g_depthMax - depth, value, quality)
        && depth > 0)
    {
        if (quality == lesser && value <= alpha) return value;
        else if (quality == exact) return value;
        else if (quality == greater && value >= beta) return value;
    }

    if (depth >= g_depthMax) moveVector = KickingMoveGenerator (position);
    else if (depth > 0) moveVector = BruteForceMoveGenerator (position);
    else moveVector = ExcludeHistoricMoveGenerator (position);

    if (moveVector.size() == 0) return PositionEvaluator (position);

    g_killer.sort(depth, moveVector);

    int valueBest = -infinity;
    for (move in moveVector)      Loop through all moves
    {
        newPosition = Move (position, move);
        int value = -Evaluate (newPosition, depth + 1,
            -beta, -Max(alpha, valueBest));
        if (value > valueBest)
        {
            valueBest = value;
            moveBest = move;
            if (value >= beta) break;
        }
    }

```

```

    }
}

g_killer.add(depth, moveBest);

if (depth >= g_depthMax)      Quiescence search
{
    Store board value in hash table with search depth 0,
    if value <= alpha no statement about the quality is possible
    if (valueBest > alpha)
        g_hash.store(position, 0, valueBest, greater);
}
else if (depth > 0)           Regular search
{
    Determine the value's quality with respect to (alpha,beta)
    if (valueBest <= alpha) quality = lesser;
    else if (valueBest >= beta) quality = greater;
    else quality = exact;
    Store board value in hash table
    g_hash.store (position, g_depthMax - depth, valueBest, quality);
}
else                          depth == 0
{
    g_moveBest = moveBest;
}

return valueBest;
}

```

The two extensions are placed at the top and the bottom of the procedure; `g_hash.store (position, lookaheadDepth, value, quality)` enters or modifies a position in the hash table, `g_hash.find (position, lookaheadDepth, value, quality)` retrieves the value V^s and quality q^s for a given position and the required lookahead depth n . Note that without a brute force search including *all* possible moves, `valueBest` can only be a lower bound to the exact value; this is why an exact result during quiescence search is impossible.

4.3.7 Comparison

Extensive tests have been carried out to measure the efficiency of the proposed algorithms. Table 4.1 summarizes the characteristics of the examined algorithms.

Table 4.1: The examined search algorithms

Abbreviation	Minimax	α - β	Killer	Hash
Algorithms	4.3, 4.2	4.4, 4.2	4.5, 4.6	4.7, 4.6
MALIN family	Caesar	Charles	David	Emil
Symbol	•	×	◦	★
Brute force	yes	yes	yes	yes
Quiescence	yes	yes	yes	yes
α - β	no	yes	yes	yes
Killer move tables	no	no	yes	yes
Iterative pruning	no	no	yes	yes
Hash tables	no	no	no	yes

Several characteristic values are used to evaluate the quality of the algorithms.

Depth n

The “regular” lookahead depth, ignoring quiescence search effects.

Quiescence depth n_Q

Denotes the average search depth including quiescence search ($n \leq n_Q$).

All evaluations e_a

The overall number of position evaluations.

Maximum depth evaluations e_m

Number of position evaluations at maximum depth, $e_m \leq e_a$. If the algorithm is not iterative, $e_m = e_a$. This value corresponds to the theoretical number of terminal positions $I_A(n)$, defined in Section 1.2.

Empirical branching factor b_e

Denotes the number of moves examined per position, excluding quiescence search.

Theoretical branching factor b_t

This value is calculated in imitation of the branching factor R_A used in theoretical analysis of search algorithms, see Section 1.2; it is defined as the n th root of the number of all evaluations, $b_t = \sqrt[n]{e_a}$.

Complexity c

A simple minimax search examines M^n positions, where $M \approx 58$ is the average number of possible moves. We define the complexity c by $M^{n \cdot c} := e_a$, which is equivalent to $c = \log_M b_t$.

Time t

The time elapsing during a full n -ply search, measured in seconds. All test are run on a AMD Pentium 266 Mhz (64MB memory, Windows 98).

Hash h

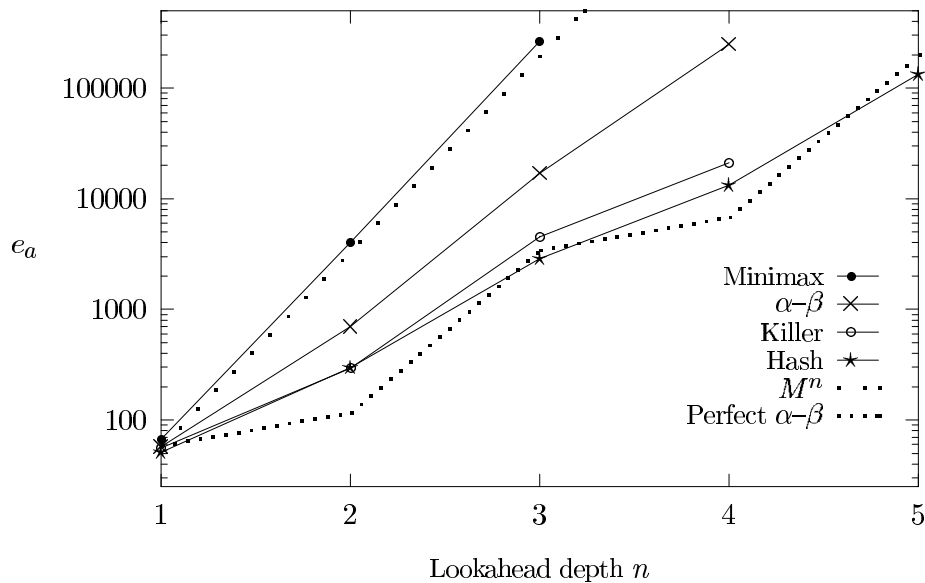
The percentage of positions found in the hash table. Note that the portion of the search tree eliminated as a consequence is necessarily greater (or equal), because all successor nodes of a “found” node will not be evaluated neither.

The following figures are based on the results of a *tournament* of 16 MALIN computer players. Each algorithm competed twice against each algorithm, making a total of $16 \cdot 15 = 240$ games, or 8.3 hours; Table 4.2 (page 70) holds the average characteristic values for each of the 16 players. The tournament winners were 5-ply Hash; 4-ply Hash, 4-ply Killer and 4-ply α - β .

Position Evaluations

Figure 4.5 compares the total number of position evaluations e_a against the theoretical prediction of M^n for a pure minimax algorithm (note that the y axis is logarithmic!). Carrying out a 3-ply Minimax requires about 250000 position evaluations, which corresponds approximately to a 4-ply α - β search, which is again distinctly above a 5-ply Hash search. One of the reasons why the pure Minimax algorithm is *above* M^n is that it includes also evaluations due to quiescence search. The most important improvement is the introduction of α - β cut-offs, followed by the amelioration due to killer move tables. More sophisticated enhancements like hash tables seem less convincing, although they clearly beat every other algorithm.

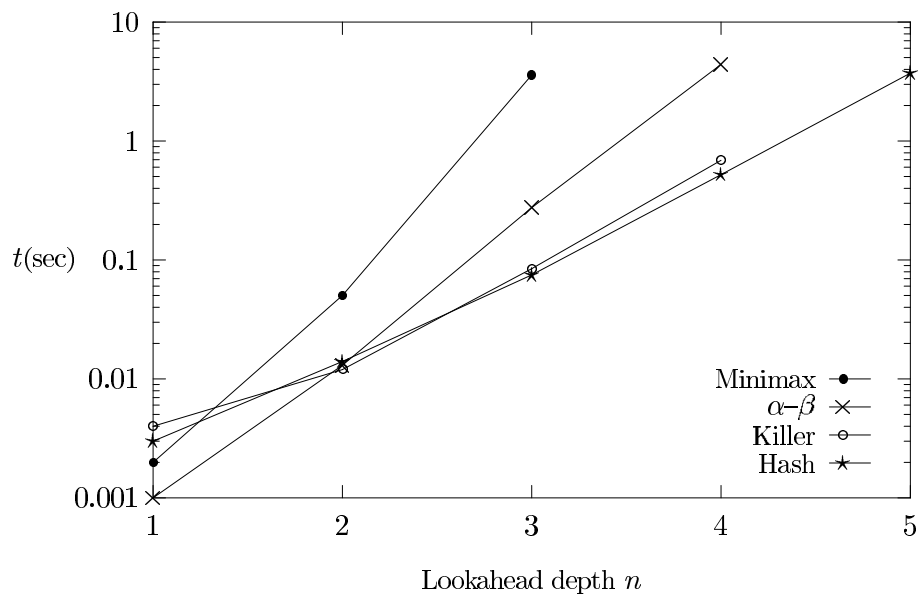
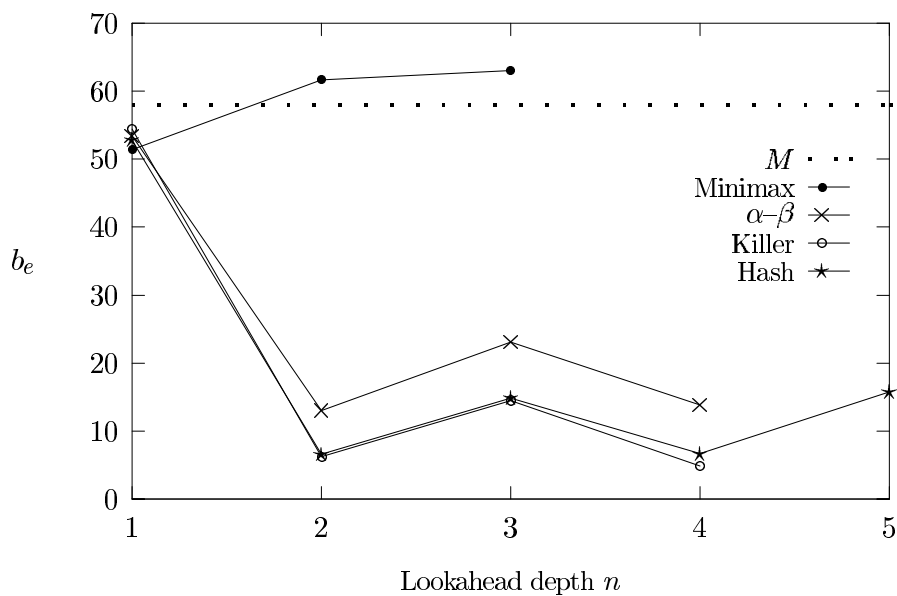
The dotted line denominated “Perfect α - β ” corresponds to $M^{\lfloor n/2 \rfloor} + M^{\lceil n/2 \rceil} - 1$, which is the (theoretical) minimum number of evaluations an α - β algorithm requires (see Section 1.2). Recall that the Hash players do *not* employ a pure α - β algorithm, this is why some outperform Perfect α - β . However, Perfect α - β beats (by definition) all

Figure 4.5: Position evaluations e_a 

“true” α - β algorithms. It is interesting to note that all algorithms descending from α - β , show different behaviour for even or odd n . The reason is that for even n , more cut-offs are possible at the *deepest* search level than for odd n . The game tree in Figure 4.3 can help to understand the difference. 1-ply Perfect α - β will evaluate exactly M positions. For $n = 2$, $M + (M - 1)$ positions will be evaluated; the first M positions are *all* descendants of $m_3 = 12$, which now serves as α -value, cutting down the remaining evaluations from $(M - 1) \cdot M$ to $(M - 1) \cdot 1$. Similarly, in a 3-ply lookahead, a good α -value is soon determined, but at the deepest (and most important) level only β -cutoffs are possible, which can make no use of the α -value. This increases the total number of evaluations to $M^2 + M - 1$ for $n = 3$.

Time per Move

Figure 4.6 plots the the time t required to compute a move. Due to the C++ runtime procedures employed, the standard deviation is quite high; this might explain the very small value of the 1-ply α - β algorithm. The overheads necessary to manage the killer move and hash tables, slow down the corresponding algorithms for low lookahead depth; however for “large” n the Hash algorithm wins again. The same reason makes 5-ply Hash as fast as 3-ply Minimax although it evaluates substantially less positions (see Figure 4.5).

Figure 4.6: Time t per moveFigure 4.7: Empirical branching factor b_e 

Empirical Branching Factor

Figure 4.7 depicts the empirical branching factors b_e . The dotted line at $y = M$ is the average branching factor, which is the average number of possible moves at a given position (ignoring quiescence search). Throughout our investigations we have estimated M by 58, but Figure 4.7 illustrates drastically a problem: M will take a lower value for a *typical* ABALONE position (occurring in a game), than for any *random* ABALONE position. When the Minimax algorithm expands a position by trying all possible moves, it will tend to “randomize” the board position, and therefore increasing the average number of possible moves. $M \approx 58$ has been chosen as a compromise in order to allow comparison with theoretical results (see Section 1.2).

Note the almost identical results for the Killer and Hash algorithm (the small branching factor of 4-ply Killer is probably due to stochastic deviation); this is not surprising – whenever the Hash algorithm does not find a position in its tables, it does effectively employ the Killer algorithm.

Theoretical Branching Factor

Figure 4.8: Theoretical branching factor b_t

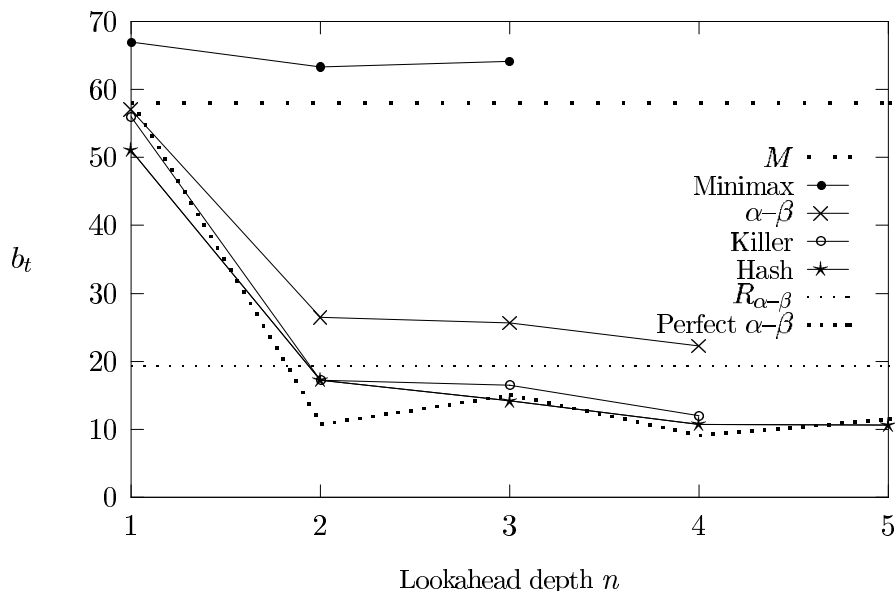


Figure 4.8 plots the theoretical branching factors $b_t = \sqrt[n]{e_a}$, which are derived from the total number of position evaluations e_a . The main difference between the empirical

and theoretical branching factor is that b_e is an essentially local statement, while b_t has much more global expressiveness. For example does b_t reflect the effects of quiescence search (see the Minimax players), the difference between the Killer and the Hash players, but also the overheads due to iterative search.

It is not surprising anymore that the Hash players perform best, further tests with 6-ply Hash (not depicted) confirm that b_t will descend even more with increasing n . The branching factors for the α - β players are bounded by the asymptotical branching factor $R_{\alpha-\beta}$ as defined in Section 1.2; however, due to move ordering and other refinements do Killer and Hash attain substantially smaller branching factors.

Use of Hash Tables

Figure 4.9: Use of hash tables h

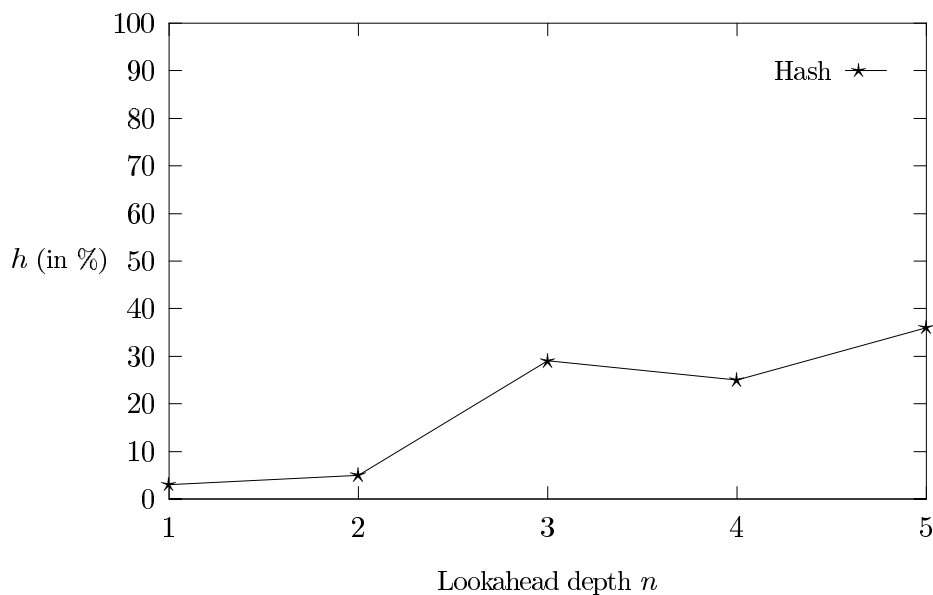


Figure 4.9 shows how the efficiency h of the hash tables increase with n . For $n = 1, 2$ every generated position in the search tree is unique, which makes the hash table no great advantage (see the corresponding b_t values for Killer and Hash in Figure 4.8). But with lookahead depth $n = 3$, move permutation becomes possible and h jumps to more than 25%. Once more the value of the not depicted 6-ply Hash ($h \approx 40\%$) underpins the even/odd hypothesis; h seems only to increase substantially after n being even.

Numerical results

Table 4.2 finally, holds all the numerical values (and more) used in the preceding figures.

Table 4.2: Search algorithm performances

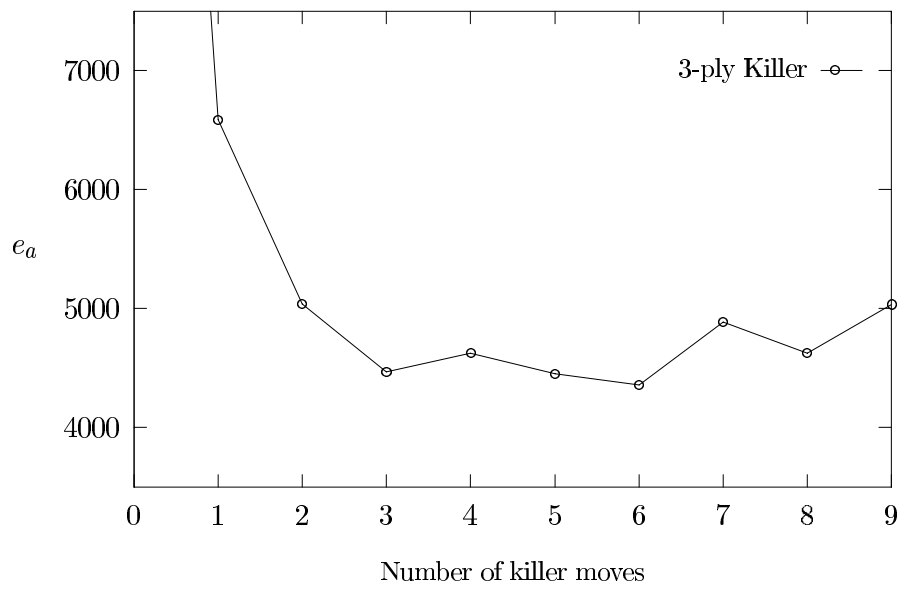
Algorithm	n	n_Q	e_a	e_m	b_e	b_t	c	t	h
Minimax	1	1.92	67	67	51.4	67.0	1.034	0.002	–
	2	2.52	4003	4003	61.7	63.3	1.021	0.050	–
	3	–	263822	263822	63.0	64.1	1.025	3.585	–
α - β	1	1.64	57	57	53.4	57.0	0.994	0.001	–
	2	2.36	700	700	13.0	26.5	0.804	0.013	–
	3	3.73	16905	16905	23.1	25.7	0.779	0.274	–
	4	–	249071	249071	13.8	22.3	0.765	4.429	–
Killer	1	1.63	56	56	54.4	56.0	0.993	0.004	–
	2	2.36	295	241	6.2	17.2	0.700	0.012	–
	3	3.36	4503	4232	14.5	16.5	0.691	0.084	–
	4	4.47	21046	16019	4.9	12.0	0.613	0.687	–
Hash	1	1.54	51	51	52.8	51.0	0.970	0.003	3
	2	2.43	296	243	6.6	17.2	0.701	0.014	5
	3	3.36	2873	2661	14.8	14.2	0.654	0.075	29
	4	4.58	13210	10089	6.7	10.7	0.584	0.519	25
	5	5.41	133830	122022	15.8	10.6	0.581	3.721	36

The definitions of the characteristic values $n_Q, e_a, \dots$ are given at page 64.

Two n_Q values are missing because of overflow.

Killer Moves

In Section 4.3.4 it was already mentioned that the MALIN killer move implementation is based upon 6 killer moves per lookahead stage. Figure 4.10 plots the number of total evaluations e_a depending on the number of used killer moves. 3-ply Killer was used for testing, and 6 turned out to generate the fewest position evaluations.

Figure 4.10: e_a depending on the number of killer moves

Chapter 5

Neuro-Dynamic Game-Playing Algorithms

Chapter 4 introduces a series of ABALONE playing programs, based on sophisticated search algorithms and a hand-crafted position evaluator. In this chapter we will replace the position evaluator by a neural network that trains itself to be an evaluation function by playing against itself and learning from the outcome. Initially, this methodology would appear unlikely to produce any sensible learning, but the rather surprising finding is that a substantial amount of learning actually takes place. The best “bred” neural network outperforms all hand-crafted players not only in quality of play but also in speed!

We now present a brief summary of the implemented reinforcement learning system (see Tesauro [Tes95]). The heart of the reinforcement learner is a *neural network* that is organized as a standard multilayer perceptron (MLP), see Section 1.1. The training procedure is as follows: the network observes a sequence of board positions $j_0, j_1, \dots, j_N$, starting with the opening position j_0 and ending in a terminal position j_N characterized by the score, i.e. 6 : 3 or 5 : 6. The board positions are fed as input vectors $x(j_0), x(j_1), \dots, x(j_N)$ to the neural network, encoded using a representation scheme $x(\cdot)$ that is described later. Each time step in the sequence corresponds to a move made by one side, i.e. a half-move or a *ply*. For each input position j_k there is a neural network output $\tilde{J}(j_k, r)$, where r is the network’s *weight vector*. $\tilde{J}(j_k, r)$ is the neural network’s estimate of the value of the position j_k . At each time step, the TD(λ) algorithm is applied to change the network’s weights. One possible formula for

the weight change is as follows:

$$r_{k+1} - r_k = \gamma \left(\tilde{J}(j_{k+1}, r_k) - \tilde{J}(j_k, r_k) \right) \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_m), \quad (5.1)$$

where γ is a small constant (commonly thought of as a stepsize or *learning rate* parameter), and $\nabla \tilde{J}(j, r)$ is the gradient of network output with respect to the weights r .

The quantity λ is a heuristic parameter controlling the *temporal credit assignment* of how an error detected at a given time step feeds back to correct previous estimates. When $\lambda = 0$, no feedback occurs beyond the current time step, while when $\lambda = 1$, the error feeds back without decay arbitrarily far in time. Intermediate values of λ provide a smooth way to interpolate between these two limiting cases.

At the end of each game, a final reward signal $J^*(j_{N+1})$ is given, based on the outcome of the game. The preceding Equation 5.1 is used to change the weights, except that the difference $\left(J^*(j_{N+1}) - \tilde{J}(j_N, r_N) \right)$ is used in instead of $\left(\tilde{J}(j_{N+1}, r_N) - \tilde{J}(j_N, r_N) \right)$.

During training, the neural network itself is used to select moves for both sides. At each time step during the course of a game, the neural network scores every possible legal move. The move that is selected is then the move with maximum expected outcome for the side making the move. In other words, the neural network is learning from the results of playing against itself. This self-play training paradigm is used even at the start of learning, when the network's weights are random, and hence its initial strategy is a random strategy.

5.1 Theory

Chapter 2 follows Bertsekas and Tsitsiklis [BT96] to develop the notion of a *stochastic shortest path problem* (Section 2.2), respectively its generalization to two-player *dynamic sequential games* (Section 2.2.5). We will now show how the game of ABALONE can be fitted into this framework.

The two principal elements of a problem of dynamic programming (DP) are, (see Section 2.1.1)

1. A discrete-time dynamic system whose state transition depends on a control. We

introduce a state space of the form $\underline{S} \cup \overline{S}$, where $\underline{S}$ are all ABALONE board positions where it is player I's turn, $\overline{S}$ accordingly for player II. At a state (board position) i of $\underline{S}$ only the *Minimizer* (player I) is allowed to choose a control (move) $u_{ij} \in U(i)$ and the system then moves to state j with probability 1. $U(i)$ are all possible moves at the board position i for player I. Note that the choice of a move u_{ij} determines uniquely the next board position, we therefore drop the notion of transition probabilities $p_{ij}(u)$ used in Chapter 2.

Similarly at a state i of $\overline{S}$, only the *Maximizer* (player II) is allowed to choose a control $v_{ij} \in V(i)$ and the system then moves to state j . The sets $\underline{S}$, $\overline{S}$, $U(i)$ and $V(i)$ are all finite, because the number of ABALONE board positions and the number of possible moves at any position are also finite.

2. *A cost that accumulates additively over time* and depends on the states visited and the controls chosen. Effectively, the cost function used in ABALONE is very particular, it does *not* depend on the states visited *nor* on the controls chosen; instead the reward depends uniquely on the final outcome of a game. For this purpose we introduce 12 special states $0_{6:0}$, $0_{6:1}$, $\dots$, $0_{6:5}$, and $0_{0:6}$, $0_{1:6}$, $\dots$, $0_{5:6}$, corresponding to the 12 possible outcomes of an ABALONE game. Every possible game will then start with the opening position j_0 , move to state $j_1, j_2, \dots$ until the last marble is kicked and the systems state becomes $0_{x:y}$, where x is the number of marbles kicked by player I and y accordingly for player II. Suppose the trajectory consists of the states $j_0, j_1, \dots, i_N, i_{N+1}$, then $j_N = 0_{x:y}$, and $j_{N+1} = 0$, the absorbing termination state corresponding to “game over”. As we do not care how the victory is accomplished we set the cost $g(j, u_{jj'})$ to 0 for all intermediate states and do only reward the victory (or the loss). A typical (although not the only used) cost function can then be defined as,

$$g(j, u_{jj'}) = \begin{cases} +1 & \text{if } j = 0_{6:0}, 0_{6:1}, \dots, 0_{6:5}, \\ -1 & \text{if } j = 0_{0:6}, 0_{1:6}, \dots, 0_{5:6}, \\ 0 & \text{otherwise.} \end{cases} \quad (5.2)$$

Unfortunately it is very likely that a deadlock situation occurs in an ABALONE game; put in terms of Section 2.2.1 does this mean that not all policies are *proper*. Even worse, Assumption 2.2 (every improper policy induces infinite cost-to-go) is not fulfilled! The solution adapted is to avoid deadlock situations as discussed in Section 4.1.4, but this provokes a new problem. In order to prevent the repetition of a move at position

j_k , the set of all possible moves $U(j_k)$ is reduced by already exploited moves. Thus $U(j_k)$ does not only depend on the state j_k , but also on its predecessors, and must more correctly be parametrized by $U(j_0, j_1, \dots, j_k)$ – which is beyond the scope of our model (because the state sequence is no Markov chain anymore). However, at the latest when introducing approximating neural networks, we have to leave the safe harbour of exact theory and try to steer clear by using “only” heuristic methods.

If deadlock situations are avoided, *all* policies are proper and the Assumptions 2.1 and 2.2 hold. This ensures that ABALONE will terminate, although the number of moves in a game is not known in advance. According to Chapter 2 ABALONE is therefore a special case of an *infinite horizon* problem, called a *shortest path* problem. Strictly speaking is ABALONE not a *1-player problem* but a *sequential 2-player game*, and because no stochastic elements intervene, do we deal with a *dynamic deterministic sequential shortest path game*.

The *optimal cost-to-go* $J^*(j)$ of the position j is defined as its *game-theoretical value*, see Section 1.3. Due to the nature of $g(j, u_{jj'})$, $J^*(j)$ can only take three values,

$$J^*(j) = \begin{cases} +1 & \text{if player I wins,} \\ 0 & \text{for } j = 0 \text{ (the absorbing terminal position),} \\ -1 & \text{if player II wins.} \end{cases}$$

In order to compute the optimal cost-to-go vector J^* , we start with an arbitrary initial cost-to-go vector J and calculate $\lim_{k \rightarrow \infty} T^k J$, where the mapping T is defined by

$$(TJ)(j) = \begin{cases} \min_{u_{jj'} \in U(j)} (g(j, u_{jj'}) + J(j')), & \forall j \in \underline{S}, \\ \max_{v_{jj'} \in V(j)} (g(j, v_{jj'}) + J(j')), & \forall j \in \overline{S}. \end{cases} \quad (5.3)$$

Dynamic programming theory proves that the limit converges to J^* .

Fortunately it is impossible to calculate $J^*(\cdot)$ exactly (otherwise the game would soon become boring) and we therefore need to employ approximations $\tilde{J}(\cdot, r)$ based on neural networks, where r is a parameter vector (characterizing the neural network). The goal is to adjust the weight vector r in such a way that $\tilde{J}(\cdot, r)$ gets close to $J^*(\cdot)$. This is done by simulation, in much the same way a statistician replaces the theoretical expected outcome $E(X)$ of a stochastic variable X by the mean value $1/n \sum x_i$ of a sample of random drawings x_i of X , $i = 1, \dots, n$; see Section 2.3 for a formal approach.

Of course we cannot suspect $\tilde{J}(j, r)$ to be exactly ± 1 , instead a value close to $+1$ can be interpreted as a high probability that player I will win.

In the following we will use neuro-dynamic algorithms in order to adjust the weight vector r . For example the online variant of approximate optimistic TD(λ) policy iteration (Section 2.3.5) uses the update rule of Equation (2.20),

$$r_{k+1} = r_k + \gamma_k d_k \sum_{m=0}^k \lambda^{k-m} \nabla \tilde{J}(i_m, r_m),$$

where d_k is the temporal difference defined by Eq. (2.19), γ_k a stepsize, and λ a parameter in $[0, 1]$. The next sections will subsequently translate this formalism into pseudo-code algorithms.

5.2 Multilayer Perceptron

A common nonlinear neural network architecture is the *multilayer perceptron* or *feed-forward neural network* with a *single hidden layer*, see Section 1.1 for an introduction. Using a I - H -1 perceptron, $\tilde{J}(j, r)$ evaluates to

$$\tilde{J}(j, r) = \sum_{h=1}^H r(h) \sigma \left(\sum_{i=1}^I r(i, h) x_i(j) \right), \quad (5.4)$$

where the state j is encoded as a vector with components $x_i(j)$, $i = 1, \dots, I$. The coefficients $r(i, h)$, and $r(h)$ are also called the networks *weights* and “connect” the i th *input* with the h th *hidden* unit, and the h th *hidden* with the only *output* unit, respectively. $\sigma(\cdot)$ is a sigmoidal function rendering the transformation nonlinear.

Implementing a multilayer perceptron is straightforward, Algorithm 5.1 gives an idea how this would look like in a object-oriented notation (similar to C++). The defined `Perceptron` object will later on serve as base object for all reinforcement learners. An object contains *member variables* (distinguished by a preceding `m`) and *member functions*; i.e., the member function `propagate` assumes that the input vector $x_i(j)$ is already stored in `m_x`, calculates the neural networks output $\tilde{J}(j, r)$ and stores it in `m_z`.

Algorithm 5.1 (Multilayer Perceptron)

```

class Perceptron
{
    int m_I;                Number of input units
    int m_H;                Number of hidden units
    float m_x[m_I];         The input vector of dimension m_I
    float m_y[m_H];         The hidden vector of dimension m_H
    float m_z;              The output unit
    float m_rih[m_I][m_H];  The weight matrix from input to hidden layer
    float m_rho[m_H];       The weight vector from hidden to output layer

    void propagate ()
    {
        Propagate the input vector m_x through the net
        Input: m_x, m_I, m_H, m_rih, m_rho
        Output: m_y, m_z

        Calculate the activation levels of the hidden layer
        for (h = 0; h < m_H; h++)
        {
            m_y[h] = 0.0;          Sum up inputs * weights
            for (i = 0; i < m_I; i++) m_y[h] += m_rih[i][h] * m_x[i];
            Apply sigmoid function to hidden unit
            m_y[h] = sigma(m_y[h]);
        }

        m_z = 0.0;                Calculate the activation level of the output unit
        for (h = 0; h < m_H; h++) m_z += m_rho[h] * m_y[h];
    }

    float sigma (float x)
    {
        return tanh(x);           Tangent hyperbolic is used as sigmoid function
    }
};

```

5.3 Optimistic TD(λ)

The neural network $\tilde{J}(\cdot, r)$ introduced in Algorithm 5.1 is on principle capable of approximating any bounded function $f(\cdot)$ of bounded variation (i.e. $f(\cdot) = \tilde{J} * (\cdot)$), provided that the number H of units in the hidden layer is large enough (Hornik et al [HSW89]). The typical problem in neural network programming is to determine the a priori unknown weights parameter r in order to approximate $f(\cdot)$. From a mathematical viewpoint this is a (possibly nonlinear) *optimization task*, frequently resolved

by iterative gradient methods. However, in this case we face another difficulty – the “goal function” $f(\cdot) = J^*(\cdot)$ is unknown too! This kind of problem is the subject of *dynamic programming* (DP). The solution for this “double” problem are *neuro-dynamic algorithms* as discussed extensively in Section 2.3. They can be considered as gradient methods where the “goal function” $f(\cdot)$ is continuously altered towards (the unknown) optimal cost-to-go $J^*(\cdot)$. At the same time these techniques can be viewed as dynamic programming algorithms, specialized to the case where the optimal cost-to-go $J^*(\cdot)$ has to be approximated by a less complex function $\tilde{J}(\cdot, r)$.

In the following, we focus on a neuro-dynamic algorithms based on *temporal differences* (TD). Let a game $j_0, j_1, \dots, j_N$ start with position j_0 , and end with j_N , then the networks evaluation $\tilde{J}(j_k, r)$ of the position j_k will become more accurate with growing k ; i.e., $\tilde{J}(j_0, r) = 0$, $\tilde{J}(j_k, r)$ for intermediate k 's is between 0 and ± 1 , and finally $\tilde{J}(j_N, r) = \pm 1$. The idea of TD is to drive $\tilde{J}(j_k, r)$ in the direction of $\tilde{J}(j_{k+1}, r)$,

$$\Delta r = \gamma \left(\tilde{J}(j_{k+1}, r) - \tilde{J}(j_k, r) \right) \nabla_r \tilde{J}(j_k, r),$$

where γ is a stepsize. The temporal difference $\tilde{J}(j_{k+1}, r) - \tilde{J}(j_k, r)$ gives the algorithm its name. In its most general form, the update rule does not only take into account the *last* move, but *all preceding* moves,

$$\Delta r = \gamma \left(\tilde{J}(j_{k+1}, r) - \tilde{J}(j_k, r) \right) \sum_{m=0}^k \lambda^{k-m} \nabla_r \tilde{J}(j_k, r).$$

The quantity λ is a heuristic parameter controlling the *temporal credit assignment* of how an error detected at a given time step feeds back to correct previous estimates. When $\lambda = 0$, no feedback occurs beyond the current time step, while when $\lambda = 1$, the error feeds back without decay arbitrarily far in time. Intermediate values of λ provide a smooth way to interpolate between these two limiting cases.

5.3.1 Online

We start with the online variant of optimistic TD(λ) policy iteration. *Online* signifies that the weight vector r is updated after *every* move; Eqs. (2.19), (2.22), and (2.23)

formalize this algorithm, which we present once more,

$$r_{k+1} = r_k + \gamma d_k e_k, \quad (5.5)$$

$$d_k = g(j_k, u_{j_k j_{k+1}}) + \tilde{J}(j_{k+1}, r_k) - \tilde{J}(j_k, r_k), \quad (5.6)$$

$$e_{k+1} = \lambda e_k + \nabla \tilde{J}(j_{k+1}, r_{k+1}). \quad (5.7)$$

The index $k = 0, 1, \dots$ denotes the k th iteration, γ is a stepsize, and λ the decay parameter. r_k is the parameter vector characterizing the neural network, d_k holds the temporal difference, comparing the current estimate of the cost-to-go $\tilde{J}(j_k, r_k)$ against the “next” estimate $g(j_k, u_{j_k j_{k+1}}) + \tilde{J}(j_{k+1}, r_k)$. When the terminal state j_{N+1} is reached, $\tilde{J}(j_{N+1}, r_N)$ is replaced by 0; this ensures that the terminal state is cost-free (as defined in Section 2.2). e_k is the eligibility vector (with the same dimension as r_k) holding the “direction” into which r_k is to be corrected; e_0 is initialized by $\nabla \tilde{J}(j_0, r_0)$. We add this algorithm to the Perceptron, which results in the following object.

Algorithm 5.2 (Optimistic TD(λ) – Online Variant)

```

class Optimistic_TDlambda extends Perceptron
{
    float m_eih[m_I][m_H];           The eligibility matrix from input to hidden layer
    float m_eho[m_I];                 The eligibility vector from hidden to output layer
    float m_lambda;                   The decay parameter (between 0 and 1)
    float m_step;                     The stepsize (i.e. 1/m_I)
    float m_zOld;                     J(x_k, r_k) from the last iteration

    onlineIterate (float x[m_I], float g)
    {
        Carry out one iteration according to Eqs.(5.5)-(5.7)
        Input:
        x ... The next state x_{k+1}
        g ... The cost associated with the transition from x_k to x_{k+1}

        m_x = x;                      Compute J(x_{k+1}, r_k) = m_z
        propagate ();

        Temporal difference d_k = g(x_k, x_{k+1}) + J(x_{k+1}, r_k) - J(x_k, r_k)
        float d = g + m_z - m_zOld;

        r_{k+1} = r_k + stepsize * d_k * e_k
        iterateWeight (m_rih, m_rho, d);

        propagate ();                  Store J(x_{k+1}, r_{k+1}) in m_zOld
        m_zOld = m_z;
    }
}

```

```

    iterateElig ();           $e_{k+1} = \lambda * e_k + \nabla J(x_{k+1}, r_{k+1})$ 
}

onlineStart (float x[m_I])
{
    Prepare the iteration
    Input: x ... The start state  $x_0$ 

    m_x = x;                Evaluate input  $x_0$  and store
    propagate ();           the result  $J(x_0, r_0)$  in  $m\_zOld$ 
    m_zOld = m_z;

    m_eih.fill(0.0);        Store gradient in eligibilities,
    m_eho.fill(0.0);         $e_0 = \nabla J(x_0, r_0)$ 
    iterateElig ();
}

onlineEnd (float g)
{
    The last iteration step, ending in the absorbing termination state  $x_{N+1} = 0$ 
    Input: g ... The cost associated with the transition from  $x_N$  to  $x_{N+1}$ 

    Temporal difference  $d_N = g(x_N, x_{N+1}) + 0 - J(x_N, r_N)$ 
    float d = g + 0 - m_zOld;

     $r_{N+1} = r_N + \text{stepsize} * d_N * e_N$ 
    iterateWeight (m_rih, m_rho, d);
}
}

```

The member functions `onlineStart`, `onlineIterate` and `onlineEnd` correspond to the Eqs. (5.5)-(5.7), `iterateWeight`, and `iterateElig` are to be defined later (Algorithm 5.3). Beforehand we discuss the computation of $\nabla_r \tilde{J}(j, r)$. Suppose the parameter vector r has the components

$$\begin{array}{cccc}
 r(1,1) & r(1,2) & \dots & r(1,H) \\
 \vdots & & & \vdots \\
 r(I,1) & r(I,2) & \dots & r(I,H) \\
 r(1) & r(2) & \dots & r(H)
 \end{array}$$

then $\nabla_r \tilde{J}(j, r)$ will be a vector with components of the form

$$\frac{\partial \tilde{J}(j, r)}{\partial r(i, h)}, \quad \text{or} \quad \frac{\partial \tilde{J}(j, r)}{\partial r(h)},$$

where $i = 1, \dots, I$ and $h = 1, \dots, H$. We rewrite the neural network output from Eq. (5.4) as

$$\tilde{J}(j, r) = \sum_{h=1}^H r(h) y_h(j, r),$$

where $y_h(j, r)$ is the activation level of the hidden unit h given state j and the network r ,

$$y_h(j, r) = \sigma \left(\sum_{i=1}^I r(i, h) x_i(j) \right).$$

We calculate

$$\frac{\partial \tilde{J}}{\partial r(h)} = y_h, \quad (5.8)$$

$$\begin{aligned} \frac{\partial \tilde{J}}{\partial r(i, h)} &= r(h) \cdot \sigma' \left(\sum_{i=1}^I r(i, h) x_i \right) \cdot x_i \\ &= r(h) \cdot \sigma'(\sigma^{-1}(y_h)) \cdot x_i, \end{aligned} \quad (5.9)$$

where $\sigma'(\xi) = \frac{d\sigma(\xi)}{d\xi}$ and $\sigma^{-1}(\sigma(\xi)) = \xi$. Note that we have suppressed the state j and the weight vector r . If we substitute $\sigma(\cdot)$ by $\tanh(\cdot)$ and make use of the (surprisingly simple) identity

$$\tanh'(\tanh^{-1}(\eta)) = 1 - \eta^2,$$

we obtain

$$\frac{\partial \tilde{J}}{\partial r(i, h)} = r(h) \cdot (1 - y_h^2) \cdot x_i.^1 \quad (5.10)$$

One of the reasons of the huge popularity of the perceptron is the efficient computation of $\nabla_r \tilde{J}(j, r)$ by Eqs. (5.8) and (5.10). We are now ready to deliver the two missing functions of the object `Optimistic_TDlambda`.

Algorithm 5.3 (Optimistic TD(λ) – continued)

```
class Optimistic_TDlambda extends Perceptron (continued)
{
  iterateWeight (var float rih[m_I][m_H], var float rho[m_H], float d)
  {
    Updates the weights according to Eq.(5.5)
     $r_{k+1} = r_k + \text{stepsize} * d_k * e_k$ 
  }
}
```

¹If we use the logistic function $\sigma(\xi) = 1/(1 + e^{-\xi})$ we obtain $\partial \tilde{J} / \partial r(i, h) = r(h) \cdot y_h(1 - y_h) \cdot x_i$

Input:

rih ... The i-h weight matrix r_k , passed by reference (var)

rho ... The h-o weight vector r_k , passed by reference

d ... The temporal difference d_k

Output: rih, rho

Hidden layer

```
for (h = 0; h < m_H; h++)
    rho[h] += m_step * d * m_eho[h];
```

Input layer

```
for (h = 0; h < m_H; h++)
    for (i = 0; i < m_I; i++)
        rih[i][h] += m_step * d * m_eih[i][h];
}
```

iterateElig ()

```
{
    Updates the eligibilities (see Eq.(5.7))
     $e_{k+1} = \lambda * e_k + \nabla J(x, r)$ 
```

Hidden layer

```
for (h = 0; h < m_H; h++)
    m_eho[h] = m_lambda * m_eho[h] + m_y[h];
```

Input layer

```
for (h = 0; h < m_H; h++)
{
    float tempH = m_rho[h] * dSigma_invSigma(m_y[h]);
    for (i = 0; i < m_I; i++)
    {
        m_eih[i][h] = m_lambda * m_eih[i][h] + tempH * m_x[i];
    }
}
}
```

float dSigma_invSigma (float y)

```
{
    return 1.0 - y * y;          Tangens hyperbolical is used as sigmoid function
}
}
```

5.3.2 Offline

We now present the *offline* variant of approximate optimistic TD(λ) policy iteration. While the online algorithm updates the parameter vector r after each state transition (j_k, j_{k+1}) , does the offline variant save computation time by updating r only when the game is over, that is when trajectory $j_0, j_1, \dots, j_N, j_{N+1}$ is completed. According to Section 5.1 is $j_N = 0_{x,y}$, and j_{N+1} the absorbing termination state 0.

Let r_t be the value of the weight vector at the beginning of the t th game. Then, at the end of the trajectory t , we update the parameter vector r by Eq. (2.24),

$$r_{t+1} = r_t + \gamma \sum_{k=0}^{N_t} d_k e_k, \quad (5.11)$$

where the temporal differences d_k and the eligibility vector e_k are defined by

$$d_k = g(j_k, u_{j_k j_{k+1}}) + \tilde{J}(j_{k+1}, r_t) - \tilde{J}(j_k, r_t), \quad (5.12)$$

$$e_{k+1} = \lambda e_k + \nabla \tilde{J}(j_{k+1}, r_t). \quad (5.13)$$

Note that while the online variant (Eqs. (5.5)-(5.7)) requires the computation of $\tilde{J}(j_k, r_k)$ and $\tilde{J}(j_k, r_{k-1})$, does the offline variant only make use of $\tilde{J}(j_k, r_t)$, on that account reducing the number of forward propagations by 50%. We will now extend the `Optimistic_TDlambda` object by the necessary functions.

Algorithm 5.4 (Optimistic TD(λ) – Offline Variant)

```

class Optimistic_TDlambda extends Perceptron (continued)
{
    float m_rihOff[m_I][m_H];    The offline weight r' i-h matrix
    float m_rhoOff[m_I];         The offline weight r' h-o vector

    offlineIterate (float x[m_I], float g)
    {
        Carry out one iteration according to Eqs.(5.11)-(5.13)
        Input:
        x ... The next state xk+1
        g ... The cost associated with the transition from xk to xk+1

        m_x = x;                  Compute J(xk+1, rt) = m_z
        propagate ();

        Temporal difference dk = g(xk, xk+1) + J(xk+1, rt) - J(xk, rt)
        float d = g + m_z - m_zOld;
    }
}

```

```

    Iterate offline weights  $r'_{k+1} = r'_k + \text{stepsize} * d_k * e_k$ 
    iterateWeight (m_rihOff, m_rhoOff, d);

    m_zOld = m_z;           Store  $J(x_{k+1}, r_t)$  in m_zOld

    iterateElig ();          $e_{k+1} = \lambda * e_k + \nabla J(x_{k+1}, r_t)$ 
}

offlineStart (float x[m_I])
{
    onlineStart (x);

    m_rihOff = m_rih;       Initialize offline weights,  $r'_0 = r_t$ 
    m_rhoOff = m_rho;
}

onlineEnd (float g)
{
    The last iteration step, ending in the absorbing termination state  $x_N = 0$ 
    Input: g ... The cost associated with the transition from  $x_N$  to  $x_{N+1}$ 

    Temporal difference  $d_{N-1} = g(x_{N-1}, x_N) + 0 - J(x_{N-1}, r_t)$ 
    float d = g + 0 - m_zOld;

     $r'_N = r'_{N-1} + \text{stepsize} * d_{N-1} * e_{N-1}$ 
    iterateWeight (m_rihOff, m_rhoOff, d);

    m_rih = m_rihOff;       Update weights,  $r_{t+1} = r'_N$ 
    m_rho = m_rhoOff;
}
}

```

5.4 Reinforcement Player

Before we use the $\text{TD}(\lambda)$ algorithms to build an ABALONE learning system, we combine a game-searching algorithm from Section 4.3 with our neural network position evaluator in order to form a complete game-playing object.

Algorithm 5.5 (Reinforcement Player)

```

class ReinforcementPlayer extends Optimistic_TDLambda
{
    Translate a board position  $j$  into an input vector  $x(j)$ . To be defined later on.

```

```

float[m_I] positionToInput (Position position);

float positionEvaluator (Position position)
{
    Calculate the value of position j with respect to the actual player

    Initialize the input vector with x(j), m_z = J(x(j), r)
    m_x = positionToInput (position);
    propagate ();

    Inverse result if it is player II's turn
    if (position.player () == 2) return -m_z;
    else return m_z;
}

Move bestMove (Position position, int depth);
{
    Perform a n-ply search and return the best move
    evaluate(...) performs a 'm_depthMax'-ply search and
    returns the best move in 'm_moveBest' (see Algorithm 4.7)
    m_depthMax = depth;
    evaluate (Position, 0, -infinity, +infinity);
    return m_moveBest;
}
}

```

The function `positionToInput` translates the state (position) j into the input vector $x(j)$, its definition is the subject of Section 5.5. The `positionEvaluator` simply propagates the input $x(j)$ through the neural network r and returns the activation level of the output unit, $\tilde{J}(j, r)$. Note that the neural network always evaluates a position with respect to player I, while `positionEvaluator` returns the value with respect to the actual player. Finally, the function `bestMove` is wrapped around the search algorithm `evaluate` (i.e. Algorithm 4.7) in order to compute the best possible move. `evaluate` makes internally use of the `positionEvaluator` $\tilde{J}(\cdot, r_t)$. In terms of Chapter 2 a call to `bestMove (position, 1)` resulting in the move u , corresponds to

$$u = \arg \min_{u_{jj'} \in U(j)} \left(g(j, u_{jj'}) + \tilde{J}(j', r_t) \right),$$

if it is player I's turn, and accordingly for $v_{jj'} \in V(j)$ if it is player II's turn.

Up until now, we make use of the neural network only to compute the best move u from a given position j . The next algorithm shows how optimistic TD(λ) is used to *train the neural network by playing against itself*.

Algorithm 5.6 (Reinforcement Player – Training)

```

class ReinforcementPlayer extends Optimistic_TDLambda (continued)
{
  void offlineSelfTraining (Position position)
  {
    Play a game starting with the initial position  $j_0$  and use its
    trajectory  $j_0, j_1, \dots, j_N$  to train the neural network  $r$ 

    Translate the position  $j_0$  to the input vector  $x_0$ ,
    initialize  $r'_0 = r$ , compute  $J(x_0, r)$  and  $e_0$ 
    offlineStart(positionToInput (position));

    Play a complete game
    while (position.gameOver () == false)
    {
      Compute the best (or at least a good) move
      Move selectedMove = goodMove (position);
      Play the move
      position.move (selectedMove);

      Carry out an iteration with cost 0,
       $r'_{k+1} = r'_k + \text{stepsize} * (J(x_{k+1}, r) - J(x_k, r)) * e_k, e_{k+1} = \dots$ 
      offlineIterate (positionToInput (position), 0);
    }

    Attribute a reward, depending on who has won
    if (position.winner () == 1) reward = +1.0;
    else reward = -1.0;

    The last iteration step,
     $r'_{N+1} = r'_N + \text{stepsize} * (g - J(x_N, r)) * e_N,$ 
    and finally update the network,  $r = r'_{N+1}$ 
    offlineEnd (reward);
  }
}

```

Note that Algorithm 5.6 contains the statement `selectedMove = goodMove (position)`, where `goodMove` is an undefined function. For the moment we replace it by `bestMove (position, 1)`; Section 5.6 will fill this gap. The training algorithm uses the cost function $g(j_k, u_{j_k, j_{k+1}})$ defined by Eq. (5.2), which assigns the cost 0 to all intermediate state transitions (j_k, j_{k+1}) , and ± 1 when the game is over.

In a typical application, the neural network parameter vector will randomly be initialized to r_0 . Next, `offlineSelfTraining` will be called for i.e. 10000 times with random

start positions j_0 . When waking up the next morning, the resulting network r will possibly beat hand-crafted position evaluators defined in Section 4.2!

The neural network is trained by exclusively playing against itself, no trainer nor supervisor are involved. The knowledge incorporated in the final weight vector r , is therefore acquired by the “machine” itself. In supervised learning approaches, the network is trained to imitate the best available human player strategies, resulting in a playing style very similar to human players. This is not at all the case in Algorithm 5.6, as a matter of principle the resulting network r develops its strategies independently of human playing style (for the moment we neglect the influence of the feature extraction upon which `positionToInput` is based). This can have astonishing consequences; Tesauro [Tes95] reports that after the appearance of his Backgammon playing program TD-Gammon (based on optimistic TD(λ)), human Backgammon world championship players adapted their strategies. The reason was, that in certain situations TD-Gammon *played different than all human players* – and turned out to be right!

5.5 Features

We already mentioned that even the self-training Algorithm 5.6 contains a certain amount of human knowledge, encoded in the translation of a board position j into an input vector $x(j)$. Effectively, the performance of a reinforcement player depends essentially upon the design of the input vector.

The most basic input vector $x(j)$ has the components

$$x_i(j) = o(i), \quad \text{for } i = 0, \dots, 60,^2 \quad (5.14)$$

where $o(\cdot)$ is an occupation function,

$$o(i) = \begin{cases} +1 & \text{if a white marble (player I) is occupying the space } i, \\ 0 & \text{if the space } i \text{ is free,} \\ -1 & \text{if } i \text{ is occupied by a black marble.} \end{cases} \quad (5.15)$$

Theoretically the input vector (5.14) provides all information necessary to make Algorithm 5.6 work, in practice its success will be limited. The main obstacle is the absence of the score of the position in the input vector. By adding enough hidden

²The ABALONE board has 61 spaces

units, the neural network can “count” the white and the black marbles and deduce the score, but this is a highly nonlinear task which is in general difficult to be learned by a network. It is by far more effective to add two new input components which reflect the score,

$$\begin{aligned} x_{61}(j) &= k_1 \\ x_{62}(j) &= k_2, \end{aligned} \tag{5.16}$$

where k_i , $i = 1, 2$ is the number of marbles kicked by player i . This will facilitate the learning task substantially. However, by *extracting this feature* from the position j and integrating it as additional information into the input vector $x(j)$, we provide the learning algorithm with human knowledge. This implies that even the self-training Algorithm 5.6 is not to 100% independent of human knowledge.

Another nonlinear and difficult task is the evaluation of neighbour relationships. In general n adjacent marbles of the same color will form a stronger position than n separated marbles. In order to speed up learning time, this information can be integrated into the input vector as well,

$$x_{63+i}(j) = o(i) \sum_{s \in N(i)} \delta_{o(i)o(s)}, \quad i = 0, 1, \dots, 60, \tag{5.17}$$

where $N(i)$ contains the spaces next to space i , and δ_{xy} equals 1 for $x = y$ and 0 otherwise. In the same fashion numerous features can be integrated into the input vector, for example the number of neighbouring enemy marbles at space i , the total number of lonely marbles, a boolean indicating whether the game is already over, ...

Let us examine the score-feature (5.16) once more; obviously the two components $x_{61}(j)$ and $x_{62}(j)$ should have the same influence on the overall result $\tilde{J}(j, r)$. Put in other words, we want $\tilde{J}(j, r)$ to be symmetric with respect to color reversal,

$$\tilde{J}(j, r) = -\tilde{J}(j', r),$$

where j' is won from position j by changing the marble colors from white to black and reversely. This can for example be ensured by $r_{61h} = -r_{62h}$, $h = 1, \dots, H$; however, our very general multi-purpose neural network and the employed learning algorithms can *not* ensure this symmetry. One solution is to adapt the neural network in order to ensure that $r_{61h} = -r_{62h}$ (see Schraudolph, Dayan and Sejnowski [SDS94] for the game of Go). The other solution is to replace the used features by strictly symmetric

features. x_i , $i = 0, \dots, 60$ are by definition symmetric, x_{61} and x_{62} can be substituted by

$$x_{61} = \text{sgn}(k_1 - k_2) \cdot \max(k_1, k_2), \quad (5.18)$$

$$x_{62} = k_1 - k_2, \quad (5.19)$$

where $\text{sgn}(\cdot)$ is the signum function,

$$\text{sgn}(\xi) = \begin{cases} +1 & \text{if } \xi > 0, \\ 0 & \text{if } \xi = 0, \\ -1 & \text{if } \xi < 0. \end{cases}$$

Features (5.18)-(5.19) are equivalent to the Features (5.16) except for $k_1 = k_2$, and because of $x_i(j) = -x_i(j')$, $i = 0, \dots, 62$, the desired color reversal invariance is achieved, $\tilde{J}(j, r) = -\tilde{J}(j', r)$.

To help the network distinguishing the consecutive phases of a game, the 2-dimensional encoding of the score can be split up into a $(2 \cdot 6)$ -dimensional array,

$$x_{60+i}(j) = \begin{cases} \text{sgn}(k_1 - k_2) & \text{if } \max(k_1, k_2) \geq i, \\ 0 & \text{otherwise,} \end{cases} \quad (5.20)$$

$$x_{66+i}(j) = \begin{cases} \text{sgn}(k_1 - k_2) & \text{if } |k_1 - k_2| \geq i, \\ 0 & \text{otherwise,} \end{cases} \quad (5.21)$$

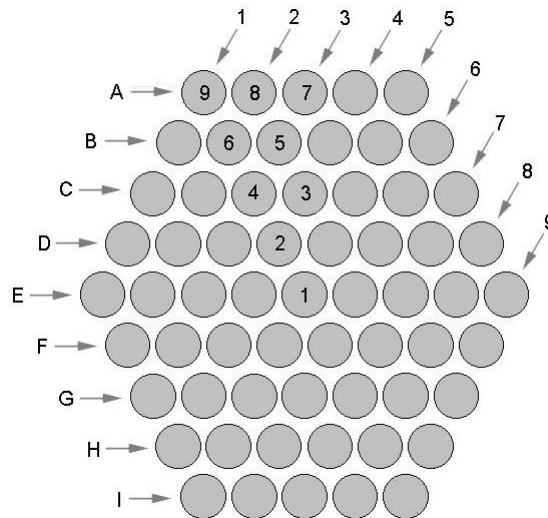
where $i = 1, \dots, 6$. These features are equivalent to the Features (5.18)-(5.19), but the network can more easily adapt its behaviour in function of the kicked marbles.

Another symmetry a priori unknown to the neural network, is the rotation invariance of the hexagonal ABALONE board. Figure 5.1 classifies the 61 spaces on the board into 9 categories, each comprising between 1 and 12 ‘‘symmetric’’ spaces. Instead of encoding each of the 61 spaces separately into the input vector $x(j)$, the position is compressed into an array of dimension 9,

$$x_i(j) = \sum_{s \in C(i+1)} o(s), \quad i = 0, \dots, 8, \quad (5.22)$$

where $C(i)$ contains all spaces of category i . As a consequence, the cost-to-go $\tilde{J}(j, r)$ is invariant against rotations of $m \cdot 60^\circ$ and/or reflection at one of the 6 mirror axis. At the same time, the input vector dimension I is reduced substantially without significant loss of information. Additional features like the neighbour Feature (5.17) can be compressed in the same way.

Figure 5.1: Rotation invariant categories



The 61 spaces on the ABALONE board can be classified into 9 categories.
 E5 is the only space in category 1, the six edges belong to category 9,
 G4 is one of 12 spaces in category 5.

5.6 State-Space Exploration

In deterministic games like Chess, Go and ABALONE the exploration of the state space is a major issue. As explained at the beginning of Section 5.3, can neural network training be viewed as a nonlinear optimization task. Like any other optimization problem, the neural network parameter r can therefore get stuck in a fixed state, resulting in inaccurate position evaluations and thus in a bad playing strategy. Theoretically, when employing color reversal invariant position evaluators this should not happen – the network playing White would be able to predict the idiosyncrasies of the network playing Black, take advantage of them thus changing the outcome, and forcing Black's predictions to change commensurately – but in practice it is a concern. We try to avoid local minimums by applying several techniques.

- One possibility for guaranteeing adequate exploration of the state space is to frequently restart the trajectory and to ensure that the initial states employed form a rich and representative subset of the state space. Although a certain percentage of training games are still started with the standard initial position

from Figure A.1 (page 102), the majority is initialized with random positions. These random positions do as well contain $2 \cdot 14$ marbles, but they are placed anywhere on the board.

- Only color reversal invariant position evaluators are employed. Consider a position evaluator $\tilde{J}(j, r)$ which is *not* color reversal invariant. The training procedure (i.e. `offlineSelfTraining`) is always started with the same initial position j_0 . Then it is likely, that after some training games player I and II settle on a pair of (bad) strategies such that player I always defeats player II. Player I has no incentive to change his strategy because he already wins all the games, player II is stuck in his bad defense strategy. This situation can not arise when $\tilde{J}(j, r)$ is color reversal invariant, because then player I and II employ exactly the same position evaluator, and player II will change strategy.
- Instead of always choosing the *best* move when simulating a trajectory $j_0, j_1, \dots, j_{N+1}$, we pick moves stochastically by *Gibbs sampling* (Geman and Geman [GG84]) in which the probability $P(m)$ of a given move m is exponentially related to the predicted value of the position j^m it leads to, through a “temperature” parameter T that controls the degree of randomness,

$$P(m) = \frac{1}{Z} \cdot \exp \frac{\tilde{J}(j^m, r)}{T}, \quad (5.23)$$

where $Z = \sum_m P(m)$ is a normalizing factor. When $T \rightarrow 0$ the sampling always picks the best move; when $T \gg 0$ the best move has still the highest probability to be chosen, but in general only a “good” move will be chosen; for $T \rightarrow \infty$ P becomes a uniform distribution. This sampling, also known as *simulated annealing*, ensures a better exploration of the state space. The function `goodMove` used in the following self-training algorithm is based on this idea.

Algorithm 5.7 (Reinforcement Player – Gibbs sampling)

```
class ReinforcementPlayer extends Optimistic_TDLambda (continued)
{
    float m_temperature;           The degree of randomness (temperature) T

    Move goodMove (position)
    {
        Compute a 'good' move with Gibbs sampling
```

```

moveVector = ExcludeHistoricMoveGenerator (position)

float probSum = 0;
for (move in moveVector)  Loop through all moves
{
    Generate the position after the move, and evaluate it
    nextPosition = position.move (move);
    float value = -positionEvaluator (nextPosition);

    Compute the probability P
    float prob = exp (value / m_temperature);
    probSum += prob;

    Select a good move according to Gibbs probabilities
    (without storing all values in memory)
    if (rand (0, 1) <= (prob / probSum)) bestMove = move;
}
return bestMove;
}
}

```

Actually, Gibbs sampling is not very well suited to the problem, because typically the values $\tilde{J}(j^m, r)$ competing against each other are very close. An amelioration can be achieved by subtracting a fixed value c in the magnitude of $\tilde{J}(j^m, r)$,

$$P(m) = \frac{1}{Z} \cdot \exp \frac{\tilde{J}(j^m, r) - c}{T}.$$

5.7 Experimentation

The following list summarizes all parameters that will affect the learning behaviour of a reinforcement learner.

Self-play Algorithm 5.6 is based upon self-training, that is, the network plays against itself. However, learning from self-play is sluggish as the network must bootstrap itself out of ignorance without the benefit of exposure to skilled opponents. For that matter, there is nothing to keep us from training the network on moves that are not based on its own predictions – for instance, it can learn by playing against the conventional MALIN players, or even by just observing games between human players.

Offline – Online Instead of using the offline variant of optimistic TD(λ) as in Algorithm 5.6, an online variant will work as well. The offline variant is certainly faster, on the other hand does the online variant tend to converge faster to a good solution.

Feature extracting The employed encoding of a state j to the input vector $x(j)$ is an essential element of success. Important is that the position evaluation $\tilde{J}(j, r)$ is invariant with respect to color inversal and to board rotations. Less sophisticated feature vectors will either learn a lot slower, or even not learn anything meaningful at all.

Network dimensions The number of input units I is determined by the number of features encoded into the input vector. The number of hidden units H is completely arbitrary, ranging from 1 to virtually infinity, limited only by computing resources. Typically $H = 10$ hidden units were used.

Initial position If a training session uses uniquely one initial position j_0 for all games, the chance of getting stuck in a suboptimal fixed state increases. Typically a mix between the standard opening position (Figure A.1) and random positions is used.

Decay parameter λ The parameter λ governs “how far the reinforcement signal is fed back”. Let a state trajectory be $j_0, j_1, \dots, j_{N+1}$, then $\lambda = 0$ means that $\tilde{J}(j_k, r)$ affects only $\tilde{J}(j_{k-1}, r)$, whereas $\lambda = 1$ implies that the value of $\tilde{J}(j_k, r_t)$ influences $\tilde{J}(j_0, r), \dots, \tilde{J}(j_{k-1}, r)$.

Stepsize γ The greater the stepsize γ , the faster the network will bootstrap itself out of the initial random state, but the worse its convergence behaviour will be. When γ and λ are poorly chosen, the network can even diverge, that is, all its weights r_{ih} and r_h tend to ∞ . MALIN effectively uses two different stepsizes, one for the input layer γ_i and one for the hidden layer γ_h . A common choice for γ is the inverse of the number of network units, $\gamma_i = 1/I$, and $\gamma_h = 1/H$. Ideally the stepsize γ decreases in the course of time, which is unfortunately not implemented in the MALIN players.

Number of training games This is obviously important. However, the achieved game-playing quality does not improve monotonously with the number of training games. Reasons are the nature of the reinforcement algorithm, which more

or less randomly “finds” good strategies, but perhaps also the missing moderation of the stepsize γ . Another possible reason is *overtraining* – after a “good” strategy was found, the training games concentrate on a particular area of the state space, “forgetting” about the regions already explored.

Rewards As already suggested, does Eq. (5.2) not define the only possible cost function. For example, we can alternatively use

$$g(j, u_{jj'}) = \begin{cases} x - y & \text{if } j = 0_{x:y}, \\ 0 & \text{otherwise.} \end{cases} \quad (5.24)$$

$\tilde{J}(j, r)$ will then have values in the range of $[-6, +6]$.

Squashing the output unit The neural network (5.4) can be modified to

$$\tilde{J}(j, r) = \sigma \left(\sum_{h=1}^H r(h) \sigma \left(\sum_{i=1}^I r(i, h) x_i(j) \right) \right),$$

which works like the original network but squashes its output unit. For $\sigma(\cdot) = \tanh(\cdot)$ this bounds $\tilde{J}(j, r)$ to $[-1, +1]$. The Eqs. (5.8)-(5.10) for the computation of $\nabla_r \tilde{J}(j, r)$ have to be changed accordingly.

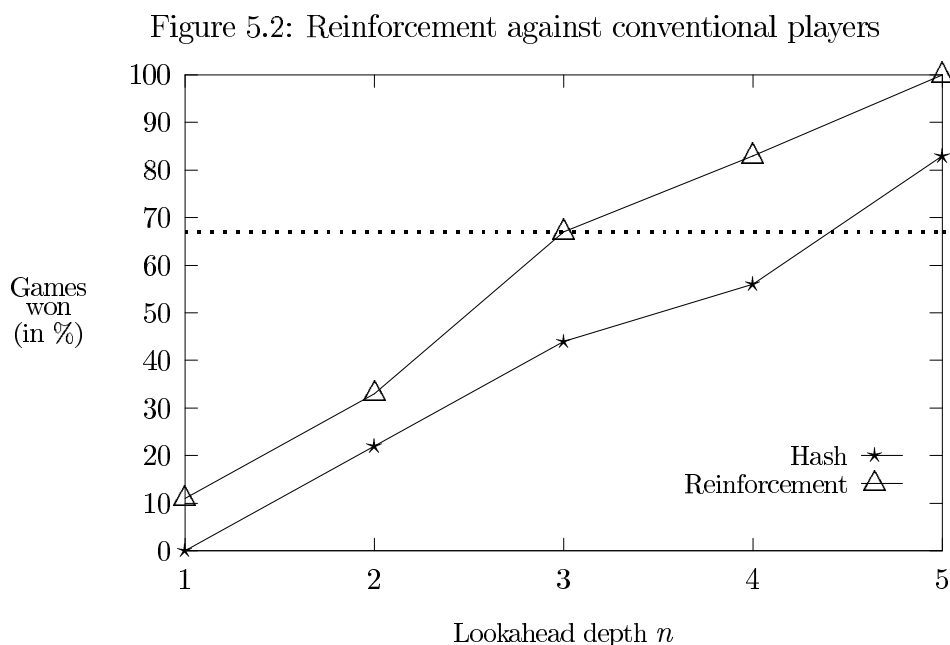
Gibbs sampling – Temperature If Gibbs sampling is used to simulate the trajectory $j_0, j_1, \dots, j_{N+1}$, a certain amount of randomness is voluntarily introduced in order to explore new regions of the state space. Using a low temperature $T \rightarrow 0$ “cools” the moves down and the best move will be picked; raising the temperature $T \gg 0$ gives the moves a lot of energy and the differences disappear in the growing chaos, the chances of the best move to be picked, diminish.

In contrast to the comparison of search algorithms in Section 4.3.7, is the assessment of position evaluation functions not straightforward. How can the quality of a position evaluator be determined? Ideally *all possibly conceivable* players compete in a tournament; if all players use the same search algorithm, the winner must contain the best position evaluator. Obviously, this is not feasible. A realistic approach is to use a set of representative players as opponents for each competing player, and to rank the position evaluators depending on the outcomes of these tournaments. However, if this subset of representative players consists only of computer programs, it can not be guaranteed, that the best ranked player will also be the player performing best against a human player. Finally in a game of ABALONE a lot of randomness is involved, winning only one game does not necessarily prove the superiority of a player.

Results

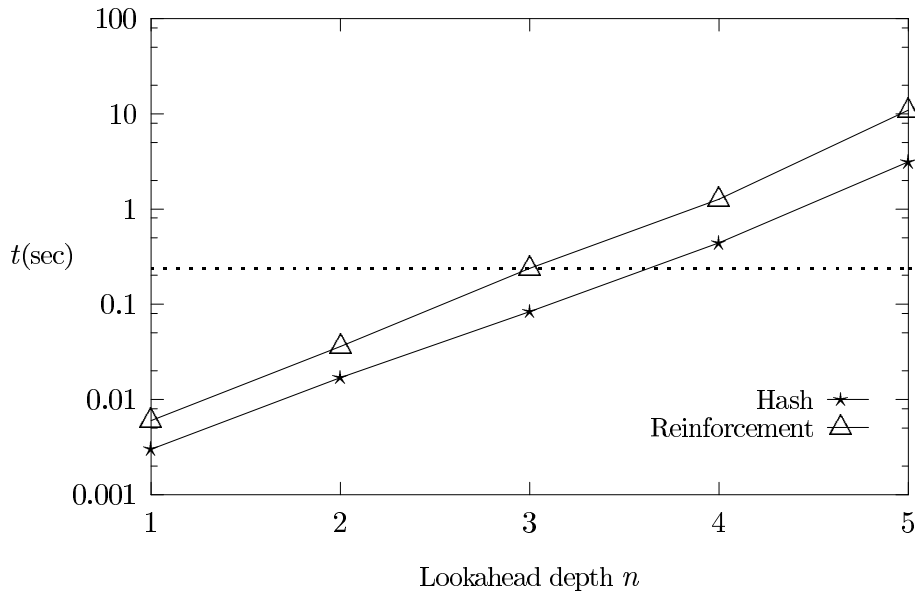
A first series of tests (the **Freud** family) was based on the Features (5.14), (5.16) and (5.17). The number H of hidden units varied between 2 and 20, no Gibbs sampling (5.23) was used. Various training methods were employed, but they all had in common that no success could be achieved by 100% pure self-training. Instead the networks were trained by using conventional MALIN players, i.e. 1-ply Killer, 2-ply Hash, or sometimes even random players. The best of these networks achieved the same quality of play as their trainers, although they were a lot slower.

The next generation of reinforcement players (the **Gunilla** family) play substantially better. They are based on the rotation invariant position Features (5.22), the color inversal invariant score Features (5.20) and (5.21), and Gibbs sampling. These players are able to acquire a remarkable level of play by pure self-training. They play distinctively better than the conventional players introduced in Section 4.3, even beating players benefiting from a deeper lookahead! Figure 5.2 shows the result of a tournament in which the Reinforcement players compete against conventional hand-crafted feature extracting players.



The Reinforcement players are apparently better than the Hash players based on a conventional position evaluator. Consider for example 3-Reinforcement; it not only

beats 3-Hash but also 4-Hash! However, the position evaluator used in the Hash family is conceived to be fast and efficient and is therefore by far less complex than the neural network providing the Reinforcement family with position evaluations. Consequently it is not very astonishing that – same search depth presumed – the Reinforcement players do better than their conventional opponents; at the same time they should also be slower. Figure 5.3 depicts the search time t per move for both families.

Figure 5.3: Time t per move

Surprise, surprise! – 3-Reinforcement does not only play better than 4-Hash, but it is also faster! Motivated by this success, further tests examining the influence of single parameters were carried out.

The single most important factor of success is the feature set employed. Attempts to introduce some sort of asymmetry into the position evaluator $\tilde{J}(\cdot, r)$, almost always degraded the performance. The feature set used in the Reinforcement players in the above tournament, include beside the usual position and score components the number of neighbouring “friend” marbles, the number of neighbouring enemies and the number of “lonely” marbles, all of them “squeezed” into rotation invariant groups (like Feature (5.22)). Its other parameters are for the sake of completeness summarized by Table 5.1.

The influence of the other parameters seem less important. Using the above parameters but varying the decay parameter λ from 0 to 1, shows only abating performance for

Table 5.1: The parameter set of the Reinforcement players

TD(λ)	Output	Initial position	I	H	λ	γ_i	γ_h	T	Games
offline	squashed	60% random	50	8	0.01	0.01	0.125	0.003	26000

All parameters are explained at the beginning of this section.

$\lambda \geq 0.5$. For $\lambda < 0.5$ no clear trend is discernible, the differences seem to be reduced to stochastic noise. Another test series measured the impact of the number of hidden units H on quality of play. Intuitively the performance must improve with increasing H , but the experiment shows that the results for $H = 1, 2, 4, 8, 16, 32$ are basically the same. Perhaps the feature vector is that good, that almost no non-linearities appear – thus rendering additional hidden units useless.

Chapter 6

Conclusion

6.1 Further research

The initial objective of this master’s thesis was the development of *self-learning* ABA-LONE computer players. The (very recent) theory of neuro-dynamic programming proved to yield astonishingly good results, the neural networks employed were able to acquire a high degree of strategic knowledge. However, there is still little understanding of their learning behaviour. The following list proposes some subjects susceptible to further research.

Moderation of the stepsize γ In order to improve the convergence behaviour, the stepsize γ (used in iterative gradient like methods, see Eq. (5.5)) should diminish over time. A popular choice is to let $\gamma = c/(m + d)$ in the course of the m th trajectory (training game), where c and d are some positive constants. MALIN currently only supports fixed stepsizes $\gamma = \gamma_0$.

Variation of the decay parameter λ The quantity $\lambda \in [0, 1]$ controls the temporal credit assignment of how an error feeds back to correct previous estimates. Experiments conducted in Section 5.7 could not register substantial performance differences for $\lambda \leq 0.5$, but it is very probable these results are forged by the “non-moderation” of the stepsize γ .

Radial basis functions Neural networks are not the only possible universal approximators. Radial basis functions could be employed instead.

Number of training games Tesauro [Tes95] reports that his Backgammon neural networks usually learn linear relationships first and only in a second phase start to reflect nonlinear affinities. It would further be very interesting if appropriate moderation of the stepsize γ could effectively avoid overtraining.

Chess The game of ABALONE is somewhat between Backgammon and Chess; while its complexity is rather comparable to Backgammon, does it *not* contain any random elements, thus being a deterministic game like Chess. Tesauro applied reinforcement learning to Backgammon; this work demonstrated that ABALONE as well is amendable to neuro-dynamic programming; the next logical step is to apply this promising methodology to the game of Chess.

6.2 Aftereffects

Artificial intelligence (AI) researchers in the nineteen-sixties and seventies proclaimed in regular intervals that within ten years the world's best Chess player would be a machine; however, it was *only in the nineties* that IBM's "Deep Blue" outperformed virtually all human players (except a handful of international masters). In the last five to ten years (the early nineteen-nineties), neuro-dynamic programming (NDP) has attracted rapidly increasing interest in the machine learning and artificial intelligence communities. Complex problems of planning and sequential decision making, that for a long time were thought to be intractable, are nowadays effectively solved by NDP. Is this the new AI miracle-tool? The development in Chess admonishes us against brash predictions, but it is nevertheless useful to outline some possible evolutions.

This result of this work is "only" a pure software product and will consequently not have great impact on *real-life*. However, it is not very difficult to imagine a neural network that uses the same methods to learn to play *Pacman*¹. At the same time industrial robots build entire cars, independent mobile units explore the Mars, and humanoid robots are already capable of climbing staircases. The combination of such advanced robots with artificial intelligent "brains" will/can/could change our society dramatically! A real-life Pacman can/could learn to "eat" the dust in my apartment, to store and retrieve goods in huge subterranean warehouses, to rescue injured persons

¹A famous computer game, where an *agent* (Pacman!) has to survive in a maze with hidden treasures and hostile monsters.

in radioactive environments, to drive my car, to track and kill enemy soldiers in asian jungles, ...

These perspectives provoke a series of questions. Are these robots *intelligent*? Is a human brain in its complexity and circumstantiality on principle superior to artificial neural networks? Will a robot ever be self-conscious? These questions are of philosophical nature, others focus on more practical issues. Will intelligent robots completely push aside workers in factories? How can we ensure the control over combat-robots? Is it not dangerous to develop intelligent beings missing an inherent control mechanism, some sort of moral sense, a conscience?

Although these questions have no immediate answers, they have to be brought to general public's attention – artificial intelligence is of too serious consequences to be left over to the scientific community.

Appendix A

Abalone – Rules of the Game

The following rules have been copied from the original game manual [LL90].

Object of the Game

To push your opponent's marbles off the hexagonal board.

How to win

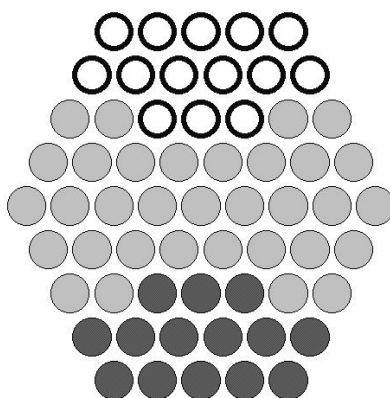
The first player to eject six of the opponent's marbles from the board wins the game.

Start of play

Setup the black and white marbles as in Figure A.1.

- Choose who plays black and who will be white, black begins.
- Players move in turn.
- Once played, a move cannot be changed.

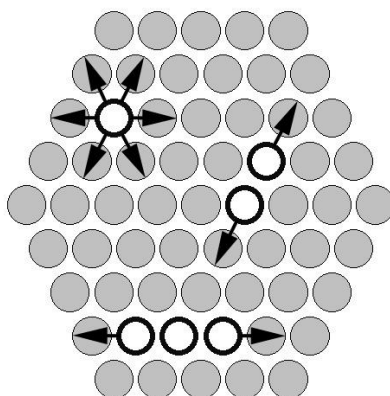
Figure A.1: Starting position



This is the initial position for a 2 player game.

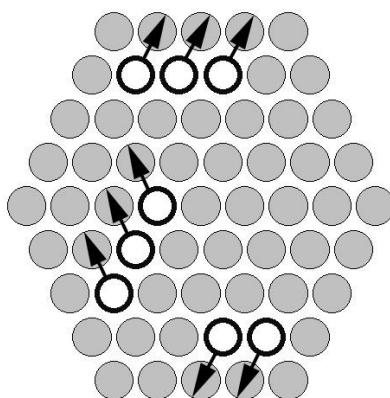
Moving

Two different kind of moves can be distinguished, *inline moves* and *broadside moves*, see Figures A.2 and A.3 respectively.

Figure A.2: *Inline* move

One, two, or three marbles are moved along the direction in which they are aligned.

- At his turn a player may move in any of the six possible directions one, two, or

Figure A.3: *Broadside* move

Two or three marbles are moved “parallel”.

three marbles, provided there is an adjacent free space.

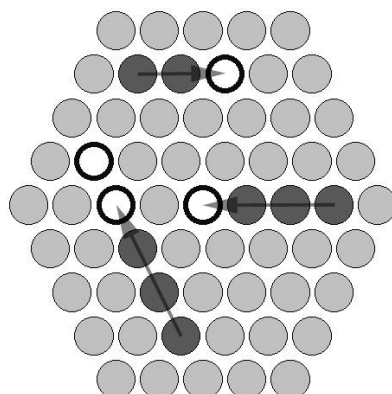
- When two or three marbles of the same colour are moved together, they must be moved in the same direction.
- Not more than three marbles of the same colour may be moved in a single manoeuvre.
- A single move may not be for more than one space at a time.

Sumito

A SUMITO is a numerical superiority allowing to push your opponent’s marbles.

- In order to push your opponent’s marbles, one of the three only SUMITO positions must be set up, see Figure A.4.
- Opponent’s marbles may only be pushed *inline*, when in contact and then only providing there is a free space behind the marble or group of two marbles in the defensive position. See Figure A.5 for some examples of illegal pushes.
- A player is never obliged to push.

Figure A.4: The three only SUMITO positions

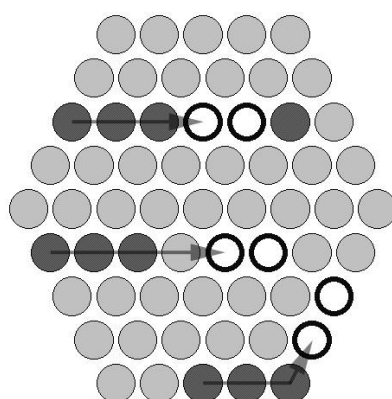


2 Push 1 SUMITO

3 Push 2 SUMITO

3 Push 1 SUMITO

Figure A.5: Examples of impossible pushes



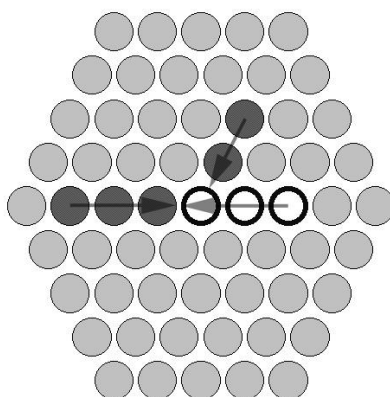
Black cannot push White because there is no free space behind the white group;
because there is an empty space between the black group and the white group;
because the marbles are not in a line.

Pac

A PAC is a position of balanced forces.

- The only three possible PAC positions are the following.
 - 1 to 1 PAC
 - 2 to 2 PAC
 - 3 to 3 PAC
- More than three marbles of the same colour and *in line* do not change the 3 to 3 score.
- In a PAC position neither side can push. The PAC must be broken along a different line of attack, see Figure A.6.

Figure A.6: How to break a PAC

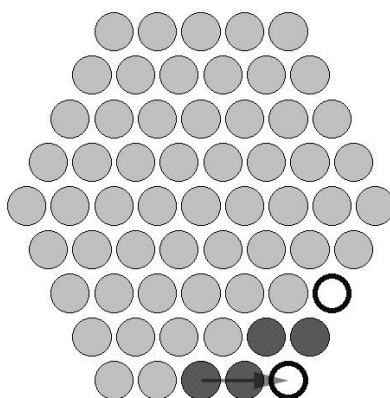


The horizontal PAC can only be broken by the two black marbles.

Ejection

A marble is ejected when it is pushed off the board, see Figure A.7. The first player to eject six of the opponent's marbles wins the game.

Figure A.7: Eject a marble



Black can push a white marble off the board.

Key Words

Move Moving a group of marbles without pushing your opponent's.

Sumito Numerical superiority in line: *to set up a SUMITO* (Figure A.4).

Push The action of pushing one or two of your opponent's marbles. This is only possible in a SUMITO position.

Line Two or three marbles that move along the direction in which they are aligned: *inline move*.

Broadside A sideways escape move by two or three marbles together: *broadside move*.

Pac A position of numerical equality in line: *PAC position*.

Eject Pushing a marble off the board, see Figure A.7.

Gambit A voluntary sacrifice of a marble to achieve a later gain.

Covered A position in which a marble is surrounded or the surrounded marble itself: *covered ball, covered position*.

Det A marble in a position from which it can be ejected and having no escape route: *DET marble* (Figure A.7).

Appendix B

Malin – Detailed Design

MALIN 3.0 is an ABALONE playing computer program developed for Microsoft Windows platforms, see Chapter 3 for an introducing user guide. This appendix is not a fully detailed design document, but gives a rough overview over the program architecture.

MALIN 3.0 is entirely written in C++ as it comes with Microsoft Visual C++ 6.0. It makes use of a lot of advanced features of C++, like *virtual base classes*, *polymorphism*, *operator overloading*, *exception handling*, *smart pointers* and *templates*. The program (approximately 350kB in 100 files) is structured in two modules.

Kernel The Kernel module comprises all ABALONE specific objects, see Figures B.1 and B.2. For example the Player object is able to serialize itself to and from the disk, to allocate the necessary objects and to compute a move for a given position. The Game object manages its players, checks if all moves are legal and keeps the game history in memory. All Kernel objects are written in pure ANSI C++ in order to facilitate possible portation to other operating systems. Extensive use is made of the standard C++ libraries, particularly of the new C++ Standard Template Library (STL).

Framework This module contains all operating-system dependent objects, see Figure B.3. To a large extent these objects drive the user interface, but others manage the worker threads performing a best move search. MALIN 3.0 is multithreaded in a way that the user interface is always served by the main thread. This ensures that the program can be controlled by the user at any time. MALIN is based on

the MFC 6.0 (Microsoft Foundation Class Library) single document architecture.

Figure B.1: Kernel objects

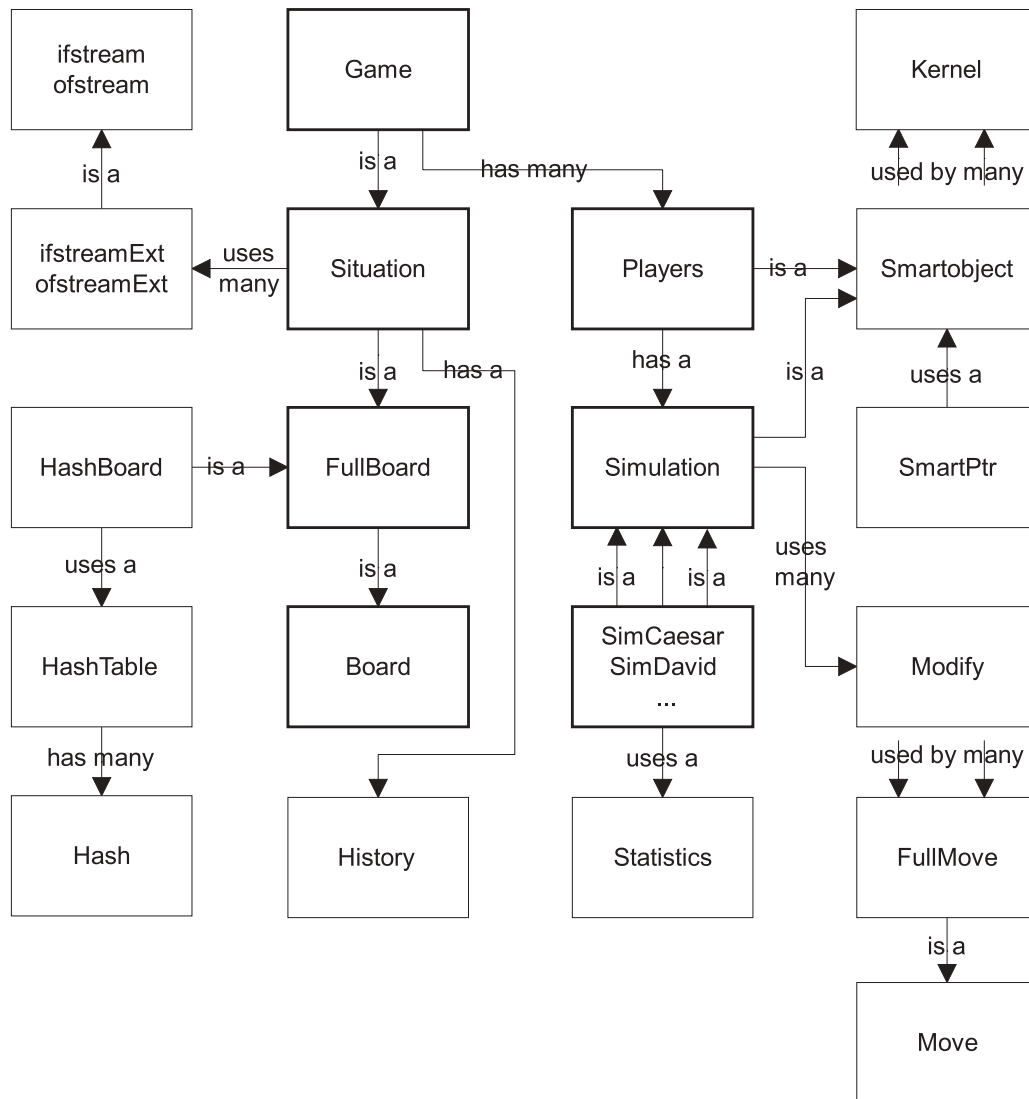


Figure B.2: Player-family specific objects

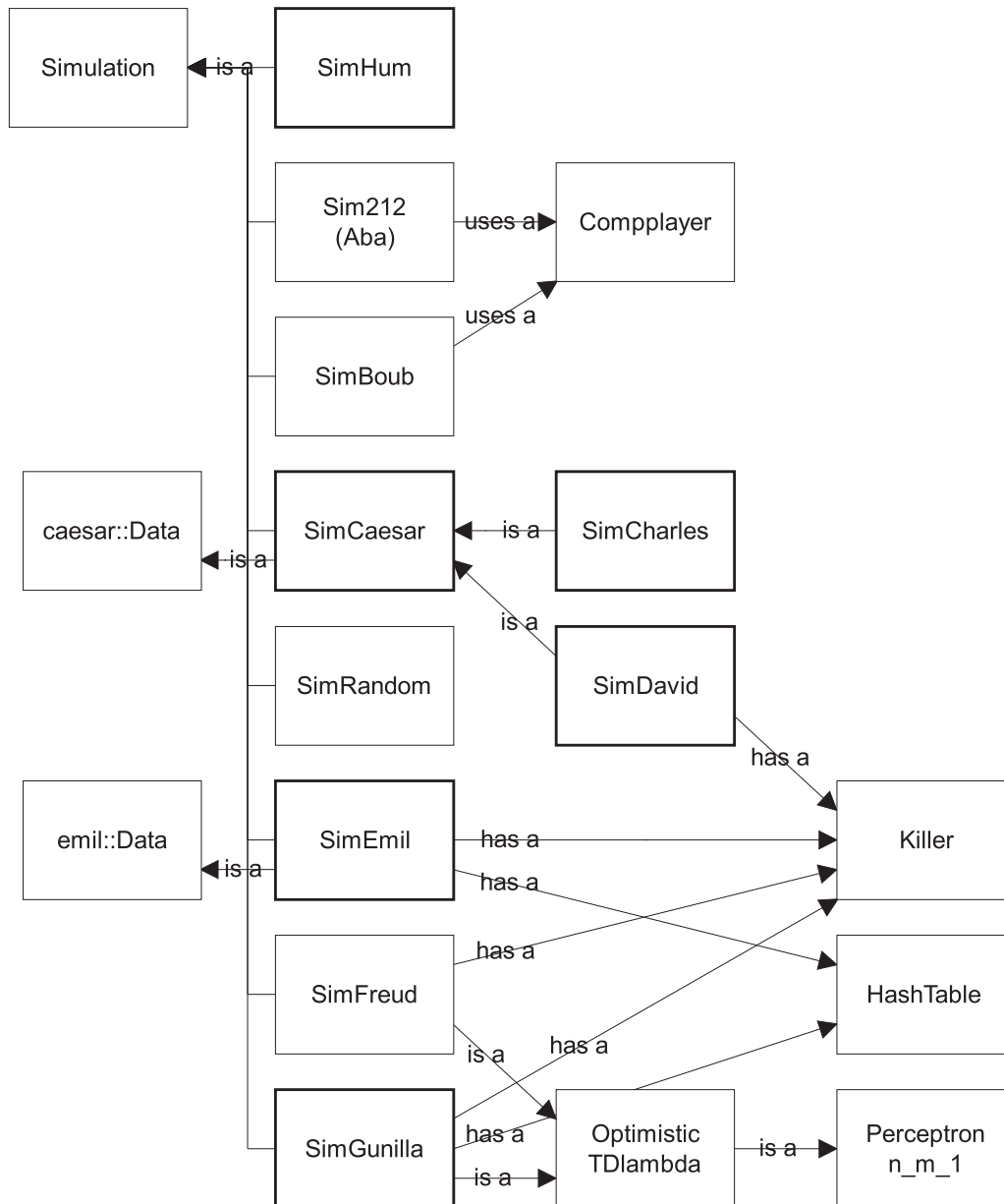
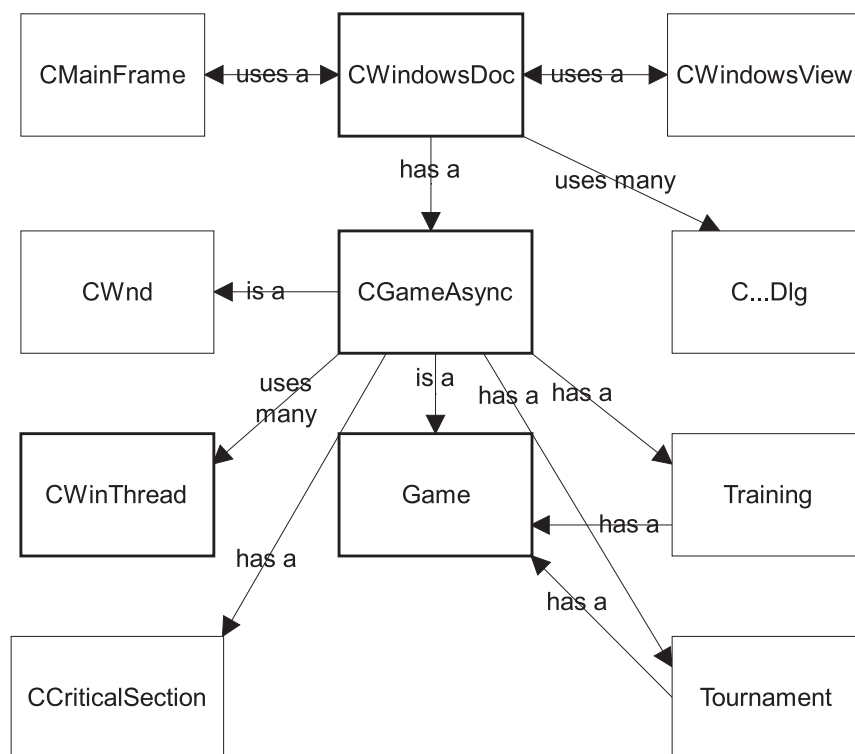


Figure B.3: Framework objects



Bibliography

- [Atk93] George Atkinson. *Chess and Machine Intuition*. Ablex Publishing Corporation, Norwood, New Jersey, 1993.
- [BCG82] E. R. Berlekamp, J. H. Conway, and R. K. Guy. *Winning Ways*. Academic Press Inc, London, 1982.
- [Ber90] Special issue on computer chess. In H. J. Berliner and D. F. Beal, editors, *Artificial Intelligence*, volume 43. 1990.
- [Ber95] D. P. Bertsekas. *Dynamic Programming and Optimal Control*, volume I and II. Athena Scientific, Belmont Mass. USA, 1995.
- [Bin92] Ken Binmore. *Fun and Games – A Text on Game Theory*. D. C. Heath and Company, Lexington, MA, 1992.
- [Bra83] M. A. Bramer, editor. *Computer Game-Playing. Theory and Practice*. Ellis Horwood Limited, Chichester, 1983.
- [BS93] Allen Bonde and Richard Sutton. *Nonlinear TD/Backprop pseudo C-code*. GTE Laboratories Incorporated, <ftp://ftp.cs.umass.edu/pub/-anw/pub/sutton/td-backprop-pseudo-code> (visited 11/98), 1993.
- [BT96] D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, Belmont Mass. USA, 1996.
- [Daw96] Herbert Dawid. *Adaptive Learning by Genetic Algorithms. Analytical Results and Applications to Economic Models*. Springer, Berlin, 1996.
- [Elm98] Wilfried Elmenreich. *Anwendung einer Alpha-Beta-Suchroutine mit Hash-tabelle für ein Abalone-Spielprogramm*. Master's thesis, Technische Universität Wien, 1998.

- [Eps94] Susan L. Epstein. Toward an ideal trainer. *Machine Learning*, 15:251, 1994.
- [Fre83] P. W. Frey. The α - β algorithm: Incremental updating, well-behaved evaluation functions, and non-speculative forward pruning. In M. A. Bramer, editor, *Computer Game Playing. Theory and Practice.*, pages 285–289. Ellis Horwood Limited, Chichester, 1983.
- [Gal96] *GAlib. A C++ Genetic Algorithms Library*. Massachusetts Institute of Technology, <http://lancet.mit.edu/ga/> (visited 09/1998), 1996.
- [GG84] S. Geman and D. Geman. Stochastic relaxation, gibbs distribution, and the bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6, 1984.
- [Gör95] G. Görz, editor. *Einführung in die Künstliche Intelligenz*. Addison-Wesley, Bonn, Paris, Reading Mass., 1995.
- [HSW89] K. Hornik, M. Stinchcombe, and H. White. Multilayer Feedforward Networks are Universal Approximators. *Neural Networks*, 2:359–366, 1989.
- [Kae96] Special issue on reinforcement learning. In L. P. Kaelbling, editor, *Machine Learning*, volume 22. 1996.
- [Kai89] Herman Kaindl. *Problemlösen durch heuristische Suche in der Künstlichen Intelligenz*. Springer, Wien - New York, 1989.
- [KLM96] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.
- [KR89] Janet Kolodner and Chris Riesbeck. Case-based reasoning. In *Tutorial Notes at the 11th International Joint Conference on Artificial Intelligence (IJCAI-89)*, 1989.
- [Lin92] L. J. Lin. Self improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–322, 1992.
- [LL90] Laurent Lévi and Michel Lalet. *The Game Abalone, Rules of the Game*. Abalone SA, Paris, France, 1990.

- [LM90] Kai-Fu Lee and Sanjoy Mahajan. The development of a world class othello program. *Artificial Intelligence*, 43:21–36, 1990.
- [Meh97] A. Mehlmann. *Wer Gewinnt das Spiel?* Vieweg, Braunschweig/Wiesbaden, 1997.
- [Mic96] Z. Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Springer, Berlin, 3. rev. and extended edition, 1996.
- [Par93] E. M. Parra. *Design und Implementierung einer Suchstrategie für das 2-Personen-Nullsummenspiel Abalone*. Master’s thesis, Universität Wien, 1993.
- [Pea84] Judea Pearl. *Heuristics. Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [Rei94] C. Reiser. *Case-Based Reasoning für mehrstufige Probleme und die Integration mit heuristischer Suche*. PhD thesis, Technische Universität Wien, 1994.
- [Sch96] Herbert Schildt. *Schildt’s Expert C++. Harness the power of C++*. Osborne McGraw-Hill, Berkely California, 1996.
- [SCT⁺92] J. Schaeffer, J. Culberson, N. Treloer, B. Knight, P. Lu, and D. Szafron. A world championship caliber checkers program. *Artificial Intelligence*, 53:273–289, 92.
- [SDS94] Nicol N. Schraudolph, Peter Dayan, and Terrence J. Sejnowski. Temporal difference learning of position evaluation in the game of go. In J. D. Cowan, G. Tesauro, and J. Alspecter, editors, *Advances in Neural Information Processing 6*, pages 817–824. Morgan Kaufmann, San Francisco, 1994.
- [Sed92] Robert Sedgewick. *Algorithmen in C*. Addison-Wesley, 1992.
- [Sin92] S. P. Singh. Transfer of learning by composing solutions of elemental sequential tasks. *Machine Learning*, 8:323–340, 1992.
- [SS96] S. P. Singh and R. S. Sutton. Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22:123–158, 1996.

-
- [Sto79] G. Stockman. A minimax algorithm better than alpha-beta? *Artificial Intelligence*, 12:179–196, 1979.
- [Sut92] Special issue on reinforcement learning. In R. Sutton, editor, *Machine Learning*, volume 8. 1992.
- [Tes92] Gerald Tesauro. Practical issues in temporal difference learning. *Machine Learning*, 8:257–277, 1992.
- [Tes94] Gerald Tesauro. TD-Gammon, a self-teaching Backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.
- [Tes95] Gerald Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, March 1995.
- [TS89] Gerald Tesauro and Terrence J. Sejnowski. A parallel network that learns to play backgammon. *Artificial Intelligence*, 39:357–390, 1989.
- [Vie97] R. Viertl. *Einführung in die Stochastik. Mit Elementen der Bayes-Statistik und Ansätzen für die Analyse unscharfer Daten*. Springer, Wien, 2. edition, 1997.

List of Figures

1.1	Several activation functions	3
1.2	A multilayer perceptron with one hidden layer	4
1.3	A $(2, 3, F)$ -tree	5
1.4	Components of a case-based reasoning system	10
3.1	MALIN in action	33
3.2	The Game menu	35
3.3	The New Game dialog box	36
3.4	The Players pane	37
3.5	MALIN board notation	37
3.6	The Move menu	38
4.1	The minimax principle	44
4.2	An unstable position	51
4.3	The α - β algorithm in a minimax formulation	53
4.4	The α - β algorithm in a negamax formulation	55
4.5	Position evaluations e_a	66
4.6	Time t per move	67
4.7	Empirical branching factor b_e	67
4.8	Theoretical branching factor b_t	68

4.9	Use of hash tables h	69
4.10	e_a depending on the number of killer moves	71
5.1	Rotation invariant categories	90
5.2	Reinforcement against conventional players	95
5.3	Time t per move	96
A.1	Starting position	102
A.2	<i>Inline</i> move	102
A.3	<i>Broadside</i> move	103
A.4	The three only SUMITO positions	104
A.5	Examples of impossible pushes	104
A.6	How to break a PAC	105
A.7	Eject a marble	106
B.1	Kernel objects	108
B.2	Player-family specific objects	109
B.3	Framework objects	110

List of Tables

4.1	The examined search algorithms	64
4.2	Search algorithm performances	70
5.1	The parameter set of the Reinforcement players	97

List of Algorithms

4.1	Brute Force Negamax Search	48
4.3	Brute Force Negamax Search – Initialization	50
4.3	Quiescence Search	51
4.4	α - β Search	55
4.5	Killer Moves	57
4.6	Iterative Search – Initialization	59
4.7	Hash Table	62
5.1	Multilayer Perceptron	76
5.2	Optimistic TD(λ) – Online Variant	79
5.3	Optimistic TD(λ) – continued	81
5.4	Optimistic TD(λ) – Offline Variant	83
5.5	Reinforcement Player	84
5.6	Reinforcement Player – Training	85
5.7	Reinforcement Player – Gibbs sampling	91

Index

- α - β algorithm, 6, 53
- Abalone
 - board notation, 37
 - Det, 106
 - Gambit, 106
 - Pac, 105
 - rules, 101
 - Sumito, 103
- Activation function, 3
- Activation level, 3
- Bellman
 - equation, 14
- Branching Factor, 5, 68
- Brute force algorithms, 43
- Case-based reasoning, 9
- Color reversal invariance, 88
- Control, 13
- Cost, 13
 - average per stage, 15
 - of a stationary policy, 15
- Cost-to-go
 - approximate, 24
 - optimal, 14
 - vector, 15
- Curse of dimensionality, 12
- Curse of modeling, 12
- Deadlock situation, 75
- Deadlock situations, 44
- Deterministic shortest path problem, 16
- Discount factor, 13
- Distance-to-center, 45
- Dynamic programming, 13
- Eligibility vector, 31
- Features, 87
- Feedforward networks, 3
- Finite horizon problems, 14
- Game
 - approximation, 32
 - dynamic sequential, 23
- Game theory, 6
- Game-playing algorithms, 41
 - neuro-dynamic, 72
- Genetic algorithms, 7
- Gibbs sampling, 91
- Hash table, 60
- Heuristic move ordering, 56
- Incremental gradient iteration, 27
- Infinite horizon problems, 15
 - average cost per stage problems, 15
 - discounted problems, 15
- Input vector, 87
- Iterative search, 58
- Killer moves, 56

- Lookup table representation, 25
- Malin
 - detailed design, 107
 - User guide, 33
- Minimax principle, 43
- Move generator, 42
 - brute force, 43, 48
 - heuristic selection, 43
- Negamax principle, 48, 54
- Neighbours, 46
- Neural network, 2
- Neuro-dynamic programming, 12
- Neuron, 2
- Non-speculative forward pruning, 59
- Optimistic $TD(\lambda)$, 29, 77
 - offline, 31, 83
 - online, 30, 78
- Overtraining, 94
- Perceptron, 2, 76
- Perfect information, 6
- Policy, 14
 - improper, 16
 - proper, 16
 - stationary, 14
- Policy evaluation step, 19
 - approximate, 26
 - approximate $TD(\lambda)$, 27
- Policy improvement step, 19
- Policy iteration, 19
 - λ , 22
 - approximate, 29
 - modified, 20
 - optimistic $TD(\lambda)$, 29
 - temporal difference-based, 21
- Position evaluator, 41
 - feature extracting, 45
 - neural network, 76
- Quiescence search, 50
- Reinforcement learning, 12
- Rotation invariance, 89
- Sigmoidal function, 3
- Simulated annealing, 91
- Simulation, 24
- State-space exploration, 90
- Stochastic shortest path problems, 16
- Strictly competitive game, 6
- Supervised learning, 1
- Synapse, 2
 - excitatory, 3
 - inhibitory, 3
- TD-Gammon, 1, 87
- Temporal credit assignment, 1, 78
- Temporal differences, 21, 28, 30, 78
- Termination state, 16
- Tesauro, 1, 87
- Training, 24, 86
- Transition probability, 13
 - graph, 18
- Universal approximator, 24
- Unstable position, 50
- Value
 - game-theoretical, 6, 23, 75
- Value iteration, 18
 - asynchronous, 19